SWE5001 · ARCHITECTING SCALABLE SYSTEMS · FINAL EXAM

考场答题手册
中英对照 • 按题型查

Exam Answer Handbook — read the question, find the type, copy the structure

2026-11-03（周二）18:30–22:00 · NUS MPSH 2 · 开卷纸质 · 总分 50，及格 20
阅读时间 30 分钟（只读不写）+ 作答 3 小时

① 读题 → 认题型
PART 0 的"题型速认表"：题干里的英文触发词 → T1–T17 → 翻到题型卡。

② 题型卡 → 下笔骨架
PART 1：每张卡 = 怎么认 · 判断表 · 按分值的骨架 · 中英句子 · 易错 · 真题位置。

③ 样卷范文 → 照着换数字
PART 2：三套样卷逐题的简单英文范文 + 中文对照 + 得分点。

④ 不认识的词 → 书末索引
PART 5：题干关键词索引（英文词 → 意思 → 题型 → 页）+ 术语 A–Z 索引。

先做每卷必考的 **T4 找错、T1/T2 领域模型、T3 API、T5/T6 架构图**（约 31 分，已超过及格线 20 分），再做 2 分小题。

整理依据：按章复习笔记与考场策略卡、考场答题工具包（题型卡、补充卡 T13–T17、复习书第 9、13、15–17 章）、课堂转写与答疑要点。PAST 有官方答案；PAST2、PAST3 没有官方答案，得分点为按 PAST 评分方式推断。整理日期 2026-11-03。

github.com/bibi45c · 免费分享，禁止售卖

HOW TO USE

使用说明：考场上怎么查这本手册

三种查法

你遇到的情况	怎么查	去哪
读完题，不知道这是哪类题	找题干里的**英文触发词**（如 **promptly**、**reuse**、**RMM**、**metric**、**shard**）	PART 0 题型速认表 p.**5**
题目是样卷里没见过的类型	四步下笔 + 8 张预备卡（LLM 延迟、四框、迁移、存储选型、网关……）	超纲·重点 p.**8**
需要引用数字、公式、默认值	案例数字表、指标定义、几个 9、换算、默认值	常用数据 p.**67**
知道题型，不知道怎么写	题型卡：**判断表 → 按分值的骨架 → 中英句子**	PART 1，从 p.**13** 开始
想看"标准答案长什么样"	同题型的样卷范文，**把名字和数字换成新案例的**	PART 2，从 p.**43** 开始
题干里有不认识的英文词	题干关键词索引：英文词 → 中文意思 → 题型 → 页	书末 PART 5 p.**79**
想写英文但不会表达	按功能找句型（表态、原因、措施、代价、假设）	PART 3 p.**58**
某个概念忘了	术语 A–Z 索引 → 学习笔记那一页	书末 PART 5

每张题型卡的固定结构

栏目	作用
怎么认	题干里出现哪些词就是这类题
判断表 / 判断链	从案例里找什么、按什么顺序判断
骨架（按分值）	每一段对应一个得分点，照着顺序写，不会漏
中英句子	_____ 处填案例里的服务名、实体名、数字；英文下面一行是中文
易错	老师点名或练习中常见的丢分点
真题	三套样卷里哪道题是这类，范文在第几页

考场 7 步

步	做什么	时间
1	阅读时间（30 分钟，**不能写字**）：先读全部题目，每题记住 **指令词 / 要交几个 / 几分 / 题型**	8 分钟
2	读附录 **§4 数字**（并发、P90、更新时限、是否多租户）→ **§7 领域模型**（哪些连线跨框）→ **§8 打包部署图和图注**（PAST3 类：§3 是数字，没有 §7 / §8）	17 分钟
3	剩下时间读用例（PAST / PAST2：§1 Phase 1、§2 Phase 2；PAST3：§5）：哪个用例**本地存储**、哪个**发通知**、哪个**实时汇总**	5 分钟
4	开考：先做 **T4、T1/T2、T3、T5/T6**（约 31 分）	约 100 分钟
5	再做 2 分小题（T7、T9、T10、T12、T11），**每题不超过 7 分钟**	约 50 分钟
6	写不出来先跳，最后回来补，**不留空白**	—
7	留 **20 分钟**复查：每个要求都答了吗？图上要求的元素画齐了吗？数字抄对了吗？	20 分钟

◆ 要点｜必背结论 **英文写不出来时的三条底线**

1. **短句**：一句话只说一件事。主语 + 动词 + 宾语，然后用 **because / so** 接下一句。
2. **抄题目和附录的原词**：服务名、实体名、用例名、数字一律照抄，不要自己翻译。
3. **结构比文采重要**：老师按得分点给分。用 **Cause: / Countermeasure: / Drawback:** 这类标签开头，阅卷人一眼能找到分。

✕ 常见误区｜这样答会丢分 **本手册的可信度**

- PAST 有**官方答案和评分**；PAST2、PAST3 **没有官方答案**，标"推断"的得分点是按 PAST 的评分颗粒度推出来的。
- PART 4 学习笔记按章整理题型卡之外的知识点，并附"易错回顾"（常见错答 → 正确要点）。
- 第 7、9、13 章的考点也并入了题型卡 T10、T12、T13–T17。

github.com/bibi45c · 免费分享，禁止售卖

CONTENTS

目录

先找题型卡（PART 1），再看同题型的样卷范文（PART 2）；按题干单词查页见最后的索引

PART 0 · 考场总纲

考场总纲：规则 • 读题 • 评分 • 时间 5

考试规则与时间分配 5
读题：指令词 → 要交什么 5
题型速认表 5
老师讲过的评分规则 6
万能答题框架 7
拿到新案例：去附录哪一节找材料 7
开考前自检 7

超纲题型与重点判断：老师点名 • 新题四步 • 8 张预备卡 8

怎么判断重点：证据从强到弱 8
遇到没见过的题：四步下笔 8
预备卡 A：第三方 API / LLM 调用延迟 9
预备卡 B：NFR 归到哪一框（四框） 9
预备卡 C：上云迁移策略怎么选 10
预备卡 D：多个用例各选一种存储 10
预备卡 E：API 网关做什么 11
预备卡 F：托管服务还是自建 11
预备卡 G：单体还是微服务（什么时候拆） 11
预备卡 H：怎么证明可维护性 / 质量 11
实体标签建议（按这个贴，和每页右上角的黑色标签同名） 12

PART 1 · 题型卡 T1–T17（怎么认 · 怎么写）

T4 打包 / 部署模型找错（每卷必考 6 分） 13

先懂两条规则：服务 × 层 = 盒子，部署是整盒搬 13
找错清单：8 项逐条扫 13
下笔骨架：三段式（每段 1 分） 13
真题：PAST Q3(c) 官方答案的三个错 14

T1 领域模型诊断 • T2 画类图与共享服务 16

T1 判断总纲：五类症状 16
T1 下笔骨架与句子 16
T2 画类图与共享服务 18
真题位置 19

T3 API 设计：RMM • 版本 • 性能重构 • Level 3 20

必备表 1：HTTP 方法与状态码 20
必备表 2：RMM 四级（从 Level 0 编号） 20
必备表 3：版本 20
问法 ①：改错表（PAST Q1(c)） 20
问法 ②：API 性能题的判断链（PAST2 Q1(c)） 21
问法 ③：设计 Level 3 API（PAST3 Q1(c)(ii)） 21

T5 云原生参考架构 + K8s 部署计划 23

第 1 步：按负载给每块功能选风格（混合架构） 23
第 2 步：照模板画组件（图 8-1，AWS 为例） 23
第 3 步：四个小问的答法 23
第 4 步：K8s 部署计划（3 分） 24
改造本地部署架构（PAST2 Q2(a) 问法） 25

T6 推荐云服务 • AWS 物理部署图 27

按需求选存储（对号入座） 27
逻辑节点 → AWS（画物理部署图用） 27
写"架构决策"的格式：选了什么 + 为什么 + 满足哪个需求 28
托管 ML 架构（PAST2 Q2(c) 问法） 28
三厂商对照（最常用的一行） 28

T7 质量属性 • T9 平台思维 30

T7 套路：选属性 → 引数字 → 说后果 → 说为什么比别的更关键 30
T9 平台思维 31

T8 指标设计（评价型 / 设计型） 33

第 1 步：指标要测"结果"，不是"用了多少" 33
第 2 步：直接采集，还是计算得出? 33
第 3 步：七个字段逐项填（不会漏分） 33

T10 二选一决策 • T12 服务共置 35

常考的四组二选一 35
T12 服务共置：哪些服务放进同一个容器 36

T11 性能 • 负载测试 • 分片 38

先判断在问哪一类 38
负载测试：工作负载模型（PAST Q1(a)(i)） 38
写吞吐瓶颈 → 分片（PAST Q3(d)，6 分） 38
性能论证用的数字和公式 39

T13–T17 可用性 • 容量 • ISO 25010 • 安全 • 韧性迁移 40

T13 可用性计算与高可用设计 40
T14 容量规划与估算 41
T15 质量属性理论（ISO 25010 / 效用树 / ASR） 41
T16 安全取舍 42
T17 韧性、发布与迁移 42

PART 2 · 样卷逐题范文（简单英文 + 中文）

PAST Vote Anywhere 投票平台（有官方答案） 43

案例速读：Vote Anywhere 43
Q1(a)(i) 性能测试结果不稳定 · Q1(a)(ii) 队列还是事件流 43
Q1(b) 领域模型诊断（9 分） 44
Q1(c) API 表：RMM、版本、重构（6 分） 44
Q1(d) 易用性是不是最重要的非功能需求 45
Q1(e) "每日处理选民数"能不能当指标 45
Q2(a) 单体 → 混合云原生参考架构 + K8s（10 分） 46
Q2(b) 原生 App 还是 PWA 46
Q3(c) 打包部署模型找错（6 分） 47
Q3(d) 数据库写吞吐瓶颈（6 分） 47

github.com/bibi45c · 免费分享，禁止售卖

PAST2 MLTTP 公交遥测平台 48
案例速读：MLTTP 公交遥测平台 48
Q1(a) 平台成功的两个关键架构方面 48
Q1(b) 领域模型：算得慢 / 内存高（9 分） 48
Q1(c) 大批量并发更新时 API 延迟 20–30 秒（6 分） 49
Q1(d) 存储最重要的两个质量属性 · Q1(e) 两个云数据库 50
Q1(f) 哪些服务放进同一个容器 50
Q2(a) 本地部署 → 云原生架构（7 分） 51
Q2(b) 虚拟机还是容器 51
Q2(c) 托管 ML 预测到站时间 51
Q3(a) 第二阶段逻辑架构：找一个错 + 补全打包（6 分） 52
Q3(b) 预测准确度指标（6 分） 52

PAST3 ONE Health 医疗健身平台 54
案例速读：ONE Health 医疗与健身平台 54
Q1(a) 平台成功的一个关键架构方面 54
Q1(b) 用 DDD 画类图 + 推荐共享服务（9 分） 54
Q1(c) 开放哪两个 API · 设计 Level 3 API（6 分） 55
Q1(d) 通信亲和度高的服务共置 56
Q2(a) 逻辑详细部署图：找三个错（6 分） 56
Q2(b) AWS 物理部署图 + 三个架构决策（9 分） 57

PART 3 · 英文写作工具箱

英文写作工具箱：规则 • 万能句型 • 中式英语改错 58
写英文答案的 10 条规则 58
答案骨架标签（直接抄） 58
万能句型（按功能查） 58
中式英语改错表（常见错误） 59
常用动词（答题够用的 30 个） 60
常用名词短语（也收进书末术语索引） 60
连接词与数字写法 61

PART 4 · 学习笔记精编（知识层）

学习笔记回忆卡：13 章要点 + 易错回顾（一行一题） 62

PART 5 · 速查与索引

常用数据与性能指标：案例数字 • 公式 • 默认值 67
三套样卷的关键数字（照抄用） 67
性能与可用性指标：定义和写法 67
利用率 → 响应时间（R = S / (1 − U)） 68
可用性：几个 9 → 允许停机多久 68
换算：人数和时间 → 每秒多少请求 68
延迟量级：为什么"跨服务就慢" 69
答题默认数字（案例没给时用，写 Assume） 69
HTTP 状态码全表 69
云服务常用数字 70

术语词典：题干难词 + 14 个主题 71
题干常见英文词（非技术） 71
架构基础与质量属性 72
逻辑 / 物理架构与打包部署 72
平台思维与平台工程 73
领域驱动设计 DDD 73
API 设计 74
云原生与架构风格 74
容器与 Kubernetes 75
数据与存储 75
通信与消息 76
性能、容量与测试 76
可用性、韧性与发布 77
安全 77
可观测性与指标 78
AWS 常用产品：名字 → 干什么 78

索引：题干关键词 • 中文拼音 • 英文 A–Z 79
题干关键词索引：看到这个英文词 → 是什么题 → 翻到哪页 79
中文术语索引（按拼音首字母） 82
英文术语 A–Z 索引 84

github.com/bibi45c · 免费分享，禁止售卖

PART 0 考场总纲

考场总纲：规则 • 读题 • 评分 • 时间

考试规则与时间分配

项目	内容
时间地点	2026-11-03（周二）18:30–22:00，NUS MPSH 2；以考试信为准
形式	开卷，**只能带纸质资料**（已确认可以带）；不能用电子设备
阅读时间	**30 分钟，只能读、可以举手提问，不能写字**（草稿也不行）；能不能翻资料没写明，可当场问
作答	3 小时，顺序随意，直接写在试卷空白处；不够写申请额外答题纸，交卷时订在对应 section 后面
文具	文字用笔（**不要用红笔**），画图可用铅笔；不需要写代码，不需要计算器
结构	通常三个 section、总分 50，各 section 分值不固定，不保证一个 section 对应一门课

按分值分配时间： 留 20 分钟复查，160 分钟答 50 分 → **每 1 分约 3 分钟。**25 / 13 / 12 的结构约 80 / 42 / 38 分钟。**2 分小题不超过 7 分钟。**

读题：指令词 → 要交什么

指令词	中文意思	要交什么	开头可以这样写
identify	指出、找出	点名**具体**的服务、实体（entity）、连线、决策 + 它在图上的位置	**The undesirable decision is that _____.**
explain / analyse	解释、分析	**机制**：为什么会这样，一句因果链，引用案例	**This happens because _____, so _____.**
propose / suggest	提出	**具体改什么、怎么改**：动作 + 对象 + 结果	**I propose to move _____ from _____ to _____.**
justify	论证（justification）	为什么这样改能满足要求，**引用 §4 数字或用例**	**This meets the requirement in Section 4 that _____.**
drawback / side effect	缺点、副作用	措施的代价（trade-off）；**没有也要写 no major drawback + 原因**（cause）	**The drawback is that _____.**
Do you agree…?	你同意吗	**先表态，再给理由**（reason）（通常两个）	**I disagree, for two reasons.**
draw / illustrate	画、举例说明	**必须画图或给具体例子**，要求的元素和连线都画出、都标名	**The diagram below shows _____.**
recommend (cloud) services	推荐云服务	**具体厂商的具体产品名** + 它满足哪个需求	**I recommend Amazon _____, because _____.**
most likely cause	最可能的原因	一个最主要的原因，点名实体和服务	**The most likely cause is that _____.**
countermeasure	对策	改领域模型（domain model）/ 改设计的具体动作	**Countermeasure: _____.**
two / three / at least two	两个 / 三个 / 至少两个	**要几个写几个，编号写**；多写一个保险也可以	**(1) _____. (2) _____.**

题型速认表

题干里出现	题型	常见分值	去哪页
packaging and deployment model, subsystem, manifest, deploy, undesirable decision, complete the diagram	**T4 打包**（packaging）**部署**（deployment）**找错**（error finding）	6	p.13
domain model, most likely cause, countermeasure, drawback; 症状 promptly / reuse / memory / scale independently / consistency	**T1 领域模型**（domain model）**诊断**	4–5 ×2	p.16
DDD, UML class diagram, domain objects, shared services, use case flows	**T2 画类图**（class diagram）**+ 共享服务**（shared service）	9	p.18
API, URI, HTTP verb, RMM Level 2/3, versioning, latency of API, refactor the API, outside-in	**T3 API 设计**	6	p.20
reference architecture, cloud-native, hybrid, microservices, scheduled jobs, serverless, Kubernetes deployment plan, stateful	**T5 云原生**（Cloud-Native）**架构 + K8s**	7–10	p.23
cloud storage / database services, physical deployment diagram, AWS services listed, managed ML service	**T6 云服务 / 物理部署**	3–9	p.27
most important non-functional requirement / quality attributes, Do you agree… usability…	**T7 质量属性**（quality attribute）	2	p.30
metric, measure and track, application metric, captured, frequency, retention	**T8 指标**（metric）**设计**	4–6	p.33
seed, interaction, producer, consumer, crucial architectural aspect, expose API to	**T9 平台思维**（platform thinking）	2	p.31
A or B? message queue vs event streaming, native vs PWA, VM vs container	**T10 二选一**（either-or decision）**决策**	2–3	p.35

github.com/bibi45c · 免费分享，禁止售卖

题干里出现	题型	常见分值	去哪页
performance test results vary, workload, throughput, write bottleneck, read replica, shard	**T11 性能**（performance）**/ 测试 / 分片**（sharding）	2–6	p.38
collocate, same container, licence constraint, communication affinity	**T12 服务共置**（collocation）	2	p.36
availability, nines, downtime, MTBF, MTTR, single point of failure, redundancy	T13 可用性（availability）（样卷（sample paper）未考）	—	p.40
capacity, sizing, storage for N years, growth, forecast	T14 容量规划（capacity planning）（未考）	—	p.41
ISO 25010, utility tree, quality attribute scenario, architecturally significant	T15 质量属性理论（未考）	—	p.41
threat, vulnerability, risk, STRIDE, control, defence in depth	T16 安全（security）（未考）	—	p.42
resilience, cascading failure, circuit breaker, zero downtime, rollback, schema change, migrate to cloud	T17 韧性 / 发布 / 迁移（migration）（未考）	—	p.42

三套样卷的题型（question type）分布

题型	PAST（Vote Anywhere）	PAST2（MLTTP 公交）	PAST3（ONE Health）
T1 领域模型诊断	Q1(b) 9	Q1(b) 9	—
T2 类图与共享服务	—	—	Q1(b) 9
T3 API 设计	Q1(c) 6	Q1(c) 6	Q1(c)(ii) ≈4
T4 打包部署找错	Q3(c) 6	Q3(a) 6	Q2(a) 6
T5 云原生架构	Q2(a) 10	Q2(a) 7	—
T6 云服务 / 物理部署	—	Q1(e) 4、Q2(c) 3	Q2(b) 9
T7 质量属性	Q1(d) 2	Q1(d) 2	—
T8 指标	Q1(e) 4	Q3(b) 6	—
T9 平台思维	—	Q1(a) 2	Q1(a) 2、Q1(c)(i) ≈2
T10 二选一	Q1(a)(ii) 2、Q2(b) 3	Q2(b) 3	—
T11 性能 / 分片	Q1(a)(i) 2、Q3(d) 6	—	—
T12 服务共置	—	Q1(f) 2	Q1(d) 2

每套必考的四类：T1/T2、T3、T4、T5/T6，合计约 31 分，已超过及格线 20 分。开考先做这四类。

老师讲过的评分规则

#	规则	对你意味着什么
1	**答案必须贴合案例**，通用笔记照抄不得分	每段都要出现案例里的角色、用例、实体（entity）、服务、数字
2	**题目要求的每一部分都要答**	原因（cause）、措施、理由（reason）、缺点（drawback）是不同的得分点（marking points），缺一个丢一份
3	**在题目指定的层面答**	领域模型（domain model）题写"加缓存（cache）、加负载均衡（Load Balancing）"是跑题；题目说刻意省略（deliberately omitted）的部分不能当缺陷
4	**用完整句子解释机制**	只列关键词说明不了"为什么"；短句也行，但要有因果
5	**合理的替代答案也给分**	质量属性（quality attribute）、指标（metric）、架构取舍（trade-off）可以不同，但必须论证（justification）；图中元素、归属、连线评分较严
6	**要求两个，不代表只有两个**	从多个问题里选两个答对即可
7	**云服务写具体产品名**	写 Amazon DynamoDB，不写"NoSQL 数据库"
8	**改图只画改动部分**	不用重画整张图；但要补的连线、元素**一定要画出来**
9	安全（security）一般不是重点，论证合理也给分	可作补充论据；不要用笼统的"安全性不好"代替根因

官方评分颗粒度（marking scheme）（PAST）

题	分数怎么给
T1 领域模型（5 分）	原因 2 + 对策（countermeasure）两部分各 1.5 + 缺点 0（但要写）
T1 领域模型（4 分）	原因 1.5 + 对策 1.5 + 缺点 1
T3 API（6 分）	三个小问各 2 分（解释 + 改进设计；官方没拆，各约 1 分是推断）
T4 找错（error finding）（6 分）	**每个决策：识别**（identify）**1 + 纠正**（correction）**1 + 论证 1，答两个**
T5 架构图（10 分）	参考架构（Reference Architecture）4 + 通信组件 2 + 缓存 1 + K8s 部署 3（Pod、Deployment、Service）
T8 指标（4 分）	每个指标连同理由 2 分
T7 / T10 小题（2 分）	每个有说服力的理由 1 分
T11 分片（sharding）（6 分）	分片方式 2 + 受影响组件及理由 4

github.com/bibi45c · 免费分享，禁止售卖

"5 分写 5 点"不是通则：**按题目的要求项逐项覆盖**才是关键。

万能答题框架

表态 / 现象 → 原因（cause）**（引用案例）→ 措施 → 理由**（reason）**与代价**（trade-off）

❝ 可用句式 | ___ 处填案例内容 **四段骨架（几乎所有论述题都能套）**

Position: I agree / I disagree with _____.
表态：我同意 / 不同意 _____。

Cause: According to Section 4, the platform must support _____ concurrent _____ . Because _____ is in _____ and _____ is in _____, every request needs _____ , which is slow.
原因：按第 4 节，平台要支持 _____ 个并发 _____。因为 _____ 在 _____ 里、_____ 在 _____ 里，每个请求都要 _____，所以慢。

Countermeasure: I propose to _____ . (Only the changed part of the diagram is drawn below.)
措施：我建议 _____。（下面只画改动的部分。）

Justification: This works because _____ ; as a result, _____ can be done within _____ seconds.
理由：这样有效是因为 _____；结果 _____ 能在 _____ 秒内完成。

Drawback: The trade-off is that _____ . It can be reduced by _____ . / There is no major drawback, because _____ .
代价：代价是 _____，可以用 _____ 缓解。/ 没有大的缺点，因为 _____。

拿到新案例：去附录哪一节找材料

附录小节	内容	哪些题要用	抄什么
§1 Phase 1	背景、角色、系统上下文（System Context）、**Phase 1 用例表**	T9、T2、T4、T8、T5	种子（seed）、生产者（producer）、消费者（consumer）；**本地保存、通知、实时汇总**（aggregation）的用例
§2 Phase 2	新角色、Phase 2 用例（分析、预测、扩展）	T8、T5、T6	**批处理、分析**的用例
§3 可扩展性（scalability）	公共服务和库、未来扩展方向	T2、T9	"新服务 / 新角色"的原文要求
§4 性能（performance）与扩展	**并发数**（Concurrency）、**响应百分位**（percentile）、**更新时限、多租户**（multitenancy）	T7、T6、T5、T10、T11	**所有数字原样照抄**
§5 安全（security）	DMZ、加密（encryption）、GDPR、PCI-DSS	T6、T4 的补充论据	一两句即可
§6 初始架构	分层（层名）、前后端（backend）部署（deployment）层	T4、T5	**层名写法要和附录一致**
§7 领域模型（domain model）	`<<service>>` 框、实体（entity）、聚合（aggregate）（红圈）	T1、T4、T12、T5	服务名、实体名、**跨框连线**
§8 打包（packaging）部署图	子系统（subsystem）、manifest、deploy、图注	T4	**每条线逐一核对**；读图注里的"省略"

注意：PAST、PAST2 写明**不预期多租户**，PAST3 要求多租户与分片（sharding）；§6 的层名在不同案例里不一样（PAST2 有 Platform Service、Enterprise Integration 层）。**PAST3 的编号不同**：§1 需求与用例、§2 扩展性、§3 性能、§4 安全、§5 用例流程、§6 架构，没有 §7、§8。**先看附录目录再引用 Section 号**，句子库里的 **Section 4** 要换成实际编号。

开考前自检

✎ 考试怎么考 | 题型·分值·套路 **开考前 30 秒自检**

- [ ] **每道题以附录原图为准**，不把前一题的改造当后一题的前提（assumption）；两小题的对策（countermeasure）不能互相矛盾
- [ ] 先做 **T1/T2、T3、T4、T5/T6**，再做小题
- [ ] 每道题：指令词（command word）/ 要几个 / 几分 / 题型（question type）
- [ ] **数字原样照抄**附录 §4
- [ ] T4：每个错 = **识别**（identify）**+ 纠正**（correction）**+ 论证**（justification）；**盒子位置 + 盒子里装的东西**都要看
- [ ] T3：每个例子都有 **方法 + URI + 请求体**（request body）**+ 返回码**
- [ ] drawback 没有也写 **no major drawback**
- [ ] 画图题：**只画改动部分**，新增的连线必须画出
- [ ] 2 分题不超过 7 分钟，写不出来先跳；留 20 分钟复查

github.com/bibi45c · 免费分享，禁止售卖

PART 0 考场总纲

超纲题型与重点判断：老师点名 • 新题四步 • 8 张预备卡

✕常见误区｜这样答会丢分 **先说清楚**

老师只说正式考试和样卷（sample paper）**难度相近**，没说题型（question type）一样。三套样卷覆盖了 T1–T12，但课程还教了很多没考过的内容。本章三件事：**① 按证据判断重点 ② 遇到没见过的题怎么下笔 ③ 8 张"可能考、样卷没考"的预备卡。**下面的"老师说过"来自课堂转写和答疑，ASR 有错字，属于高概率线索，不是承诺。

怎么判断重点：证据从强到弱

证据强度	来源	结论
★★★★	三套样卷（sample paper）每卷都考的题型（question type）	**T1/T2、T3、T4、T5/T6 必须拿分**（约 31 分）
★★★	老师在课上**点名**"会考 / 考过"的题	见下表，按预备卡准备
★★	Workshop、作业、课堂练习做过的题	可用性（availability）计算、存储选型（technology selection）、迁移策略（migration strategy）、日志（logs）查询
★	课件里讲了但没人提	认识名词，用到时查术语词典

老师说过的话（转写，可能有误）

课	老师说过	你要准备	页
ArchSS	他负责的部分约 **10 分**，在 **Q1(a) 和 Q1(c)**，小问各 **2 分**；评分偏宽松	性能（performance）测试（T11）、API 版本（version）与接口表（T3）	p.**38**
ArchSS	讲**第三方 LLM 调用延迟**（latency）时说 **"This question will come out during the exam"**	预备卡 A	p.**9**
ArchSS	**API 网关**（gateway）**题曾是考题**，可出类似 Q1 的题，理由（reason）要扎实	预备卡 E	p.**11**
ArchSS	可用性 workshop：从 NFR 找约束（constraint） → 算串联（in series）可用性 → 改到 98% / 99% → 重算	T13 卡	p.**40**
ArchSS	**排队**（queueing）**论数学不考**；看 P95 / P99 不看平均	只用公式写论证（justification）	p.**68**
CNSD	NFR 归类 **"Remember it's always one of the four boxes"**，题目会 tricky	预备卡 B	p.**9**
CNSD	托管 vs 自建（self-hosted）是成本判断核心；托管的宕机怎么算写在细则里	预备卡 F	p.**11**
CNSD	迁移策略：课堂讲六种、课件 7R（TickIt 案例）；存储选型靠四个问题（购物车作业）	预备卡 C、D	p.**10**
CNSD	微服务（microservices）是"用复杂度换扩展"，简单场景（scenario）单体（monolith）更好	预备卡 G	p.**11**
CNSD	常见错误：把 GitOps / CI-CD 当应用级选择（其实是**平台**关注点）；看到 data 就说 storage	预备卡 B	p.**9**
ArchSS	质量要给**证据**：覆盖率、每服务测试数、静态扫描	预备卡 H	p.**11**
通用	安全（security）一般不是重点，论证合理给分	T16 卡	p.**42**

遇到没见过的题：四步下笔

步	做什么	例
1 定位	这题在考**哪门课的哪个知识**？用题干（question stem）单词查书末索引	"circuit breaker" → T17
2 找材料	附录哪一节有用？§4 数字、§2 用例、§7 模型、§5 安全（security）	"外部支付网关（gateway）慢" → §6 集成层、§4 时限
3 套骨架	用总纲的四段骨架：**表态 / 现象** → **原因**（cause）**（引用案例）** → **措施** → **理由**（reason）**与代价**（trade-off）	每段一个得分点（marking points）
4 数要求	题目要几个（two / three / at least）、要不要图、要不要例子、要不要缺点（drawback）	数够了再写下一题

github.com/bibi45c · 免费分享，禁止售卖

“ 可用句式 | ___ 处填案例内容 **没见过的题也能用的四句**

The key requirement here is _____ in Section _____, which needs _____.
| 这里的关键要求是第 _____ 节的 _____，它需要 _____。

The problem is caused by _____, because _____.
| 问题由 _____ 引起，因为 _____。

I propose _____. It meets the requirement because _____.
| 我建议 _____。它能满足要求，因为 _____。

The trade-off is _____, which is acceptable because _____.
| 代价是 _____，这可以接受，因为 _____。

预备卡 A：第三方 API / LLM 调用延迟

怎么认： 平台调用外部系统（LLM、支付网关（gateway）、地图、短信），对方**慢或不稳定**，问怎么设计 / 怎么保证响应时间（response time）。老师说这是**不可控的延迟**（latency）**约束**（constraint），要按约束设计，不能按理想值设计。

手段	做法
异步化（asynchronous）	请求先回 202，后台调用外部 API，结果推送（push） / 轮询（polling）（WebSocket、通知）
超时（timeout） + 重试（retry）	设超时；指数退避重试（Retry with Exponential Backoff）；只重试幂等（idempotent）请求
熔断（circuit breaking） + 降级（degradation）	失败率高时熔断，返回缓存（cache）值 / 通用答案 / 提示稍后
缓存	相同问题的结果缓存（Redis），命中时不调外部
流式返回（streaming response）	LLM 分段返回（streaming），用户先看到前几句【通用知识】
队列 + 限流（rate limiting）	进队列削峰（peak shaving）；遵守外部 API 的速率限制，避免 429
集成层隔离（isolation）	外部调用放 External Integration 层，外部变化不影响业务
度量	单独监控（monitoring）外部调用的 P95 / P99 和失败率

“ 可用句式 | ___ 处填案例内容

The LLM API is outside our control, so its latency is a constraint. The request returns 202 Accepted at once, and a background worker calls the LLM with a timeout and retries; the answer is pushed to the user when ready.
| LLM API 不在我们控制范围内，它的延迟是一个约束。请求立即返回 202 Accepted，后台工作者带超时和重试去调用 LLM；结果准备好后推送给用户。

A circuit breaker stops calls when the failure rate is high and returns a cached or generic answer, so the platform stays responsive. We monitor the P95 latency of the external call separately.
| 失败率高时熔断器停止调用，返回缓存或通用答案，让平台保持响应。我们单独监控外部调用的 P95 延迟。

“ 可用句式 | ___ 处填案例内容 **预备卡 A 补充：逐条对上题目的时限要求**

Decision 1 – Asynchronous request: the app sends POST / _____ /questions and gets 202 Accepted with a questionId at once, so 90% of questions are acknowledged within _____ seconds.
| 决策 1（异步请求）：App 发送 POST / _____ /questions，立即收到 202 Accepted 和 questionId，所以 90% 的问题能在 _____ 秒内确认收到。

A background worker calls the LLM API with a timeout of about _____ seconds and at most one retry, so the answer, or a fallback answer, is shown within _____ seconds.
| 后台 worker 调用 LLM API，超时设为约 _____ 秒、最多重试一次，所以答案（或降级答案）能在 _____ 秒内显示。

Decision 2 – Push the answer: when the answer is ready it is pushed to the app through WebSocket or a push notification, so the app does not wait or poll.
| 决策 2（推送答案）：答案准备好后通过 WebSocket 或推送通知发给 App，App 不用干等，也不用反复轮询（polling）。

Decision 3 – Circuit breaker and fallback: about _____ % of LLM calls fail. When the failure rate passes a threshold, the circuit breaker opens and the app gets a fallback answer (e.g. a rule-based list of _____), so it stays responsive.
| 决策 3（熔断与降级）：约 _____ % 的 LLM 调用会失败。失败率超过阈值时熔断器打开，App 拿到降级答案（如按规则算出的 _____ 列表），保持响应。

Common questions are cached in Redis, and the LLM call sits in the External Integration layer, so changes in the provider's API do not affect the other services.
| 常见问题的结果缓存在 Redis；LLM 调用放在 External Integration 层，供应商 API 变化不影响其他服务。

预备卡 B：NFR 归到哪一框（四框）

怎么认： 给一串需求，问属于哪一层 / 谁负责 / 架构师怎么处理。老师的"四框（four boxes）"：**基础设施**（infrastructure）**（计算、存储、网络）/ 平台 / 应用架构**（application architecture）**/ 业务功能**（business function）。业务需求来自干系人，其余三框靠架构师判断。

需求例子	归到	理由
要多少 CPU、磁盘、网络带宽	基础设施	算、存、网
数据库、消息队列（message queue）、CI/CD、GitOps、监控（monitoring）、身份	**平台**	多个应用共用的服务（老师：GitOps 是平台关注点，常被答错）

github.com/bibi45c · 免费分享，禁止售卖

需求例子	归到	理由
微服务（microservices）拆分、同步（synchronous）/ 异步（asynchronous）、缓存（cache）、状态管理	应用架构	系统怎么搭
下单、退款规则、通知内容	业务功能	来自干系人
"要存数据"	先问是**数据库服务（平台）**还是**磁盘（基础设施）**	老师：data ≠ storage

❝ 可用句式｜___ 处填案例内容

_____ belongs to the platform box, because it is a shared service used by every application, not a choice of one application.
_____ 属于平台这一框，因为它是所有应用共用的服务，不是某个应用自己的选择。

预备卡 C：上云迁移策略怎么选

怎么认： 传统本地系统要上云，问用哪种迁移策略（migration strategy）、为什么（TickIt 票务案例）。课堂讲六种（Retire、Rehost、Replatform、Re-architect、Rebuild、Repurchase），课件 7R 再加 Retain。

策略	做法	适合
Retire	下线	没人用的系统
Retain	先不动	合规或刚升级过
Rehost（lift & shift）	原样搬到 VM	要快、风险（risk）最低；只是"一对一替换"，不利用弹性（Elasticity）
Replatform	搬到容器（Container）/ 托管数据库，少量修改	想省运维、要一定弹性
Rearchitect	改架构（微服务（microservices）、无服务器（serverless））	要扩展、要敏捷，成本和风险最高
Rebuild	重写	旧代码不值得改
Repurchase（= Replace）	换成买来的 SaaS	有成熟的现成产品

判断： 成本、复杂度、风险优先 → Rehost / Replatform；敏捷、创新、海外扩张要弹性 → Rearchitect。评估五维：价值、见效时间、业务中断、变更成本、风险。

❝ 可用句式｜___ 处填案例内容

I recommend replatforming: the web and app tiers move to containers on Amazon EKS and the database to Amazon RDS. This gives auto-scaling for the new overseas customers with moderate cost and risk; a full re-architecture would be more agile but slower and riskier.
我建议 replatform：Web 和应用层搬到 Amazon EKS 上的容器，数据库搬到 Amazon RDS。这样能为新的海外客户提供自动扩缩，成本和风险适中；完全重构更敏捷，但更慢、风险更高。

❝ 可用句式｜___ 处填案例内容 **预备卡 C 补充：直接搬迁的缺点**（drawback）**• 封装**（Encapsulation）**• 先迁什么**

Rehosting (lift and shift) is the fastest and least risky option, but the application cannot use cloud-native features such as auto-scaling and managed services, and it may cost more to run in the cloud than on-premises.
重新托管（直接搬迁）最快、风险最低，但应用用不上自动扩缩、托管服务这些云原生特性，在云上运行还可能比本地更贵。

The legacy _____ can be encapsulated: its data and functions are exposed as APIs through Amazon API Gateway, so new cloud services can reuse it without rewriting it now.
遗留的 _____ 可以封装（Encapsulate）：通过 Amazon API Gateway 把它的数据和功能开放成 API，新的云服务就能复用它，暂时不用重写。

I would not migrate _____ first, because it is the most critical and complex system; we start with _____, which is less critical, to build skills and a landing zone.
我不会先迁 _____，因为它最关键、最复杂；先迁不那么关键的 _____，积累经验并搭好着陆区。

预备卡 D：多个用例各选一种存储

怎么认： 给 3–5 个用例，每个选一种数据库并给一句理由（reason）（购物车作业的问法）。老师说**没有唯一答案，评的是论证**（justification）。模板：**用例 → 数据特征 → 存储类型 → 为什么匹配 → 替代为什么不够。**

用例	选	理由
登录（用户名唯一、密码哈希）	RDBMS	唯一约束（constraint）、每次登录更新要正确
页头（积分、上次登录）	键值 / Redis	已知 ID 直接取，处处显示
商品列表（品类字段不同）	文档	每件商品字段各异，schema 灵活
下单（订单 + 明细）	RDBMS	必须一起保存，事务（transaction）保证不留半单
推荐	图	依赖"买过同一商品的人"等关系遍历
时间线、消息、传感器	宽列（Wide-Column）/ 时序	写一次读多次、写密集
日志（logs）分析	对象存储（object storage）+ 搜索	量大、长任务

老师的四问： 数据形态、量级、关系复杂度、一致性要求。**财务账本要强一致**（strongly consistent）。

github.com/bibi45c · 免费分享，禁止售卖

预备卡 E：API 网关做什么

怎么认： 问 API 网关（gateway）的作用、为什么需要、和服务网格（service mesh）/ 反向代理（reverse proxy）有什么区别。老师说**网关题曾是考题。**

网关六件事	一句话
路由与服务发现（service discovery）	把请求转给正确的服务
认证（Authentication）与授权（Authorisation）	统一校验令牌（OAuth 2.0 / JWT）
限流（rate limiting）与配额（Quota）	挡住过载，按套餐限流
协议转换	如 REST ↔ gRPC / SOAP
响应聚合（aggregate）	一次请求合并多个服务的结果
监控（monitoring）与日志（logs）	统一记录每个请求

区别： 网关管**南北向**（客户端 ↔ 平台），服务网格管**东西向**（East-West）（服务 ↔ 服务）；反向代理主要做转发、负载均衡（Load Balancing）、TLS 终止、隐藏内部服务器；API 网关在此之上做 **API 管理**：认证授权、限流配额、版本（version）、协议转换、聚合、监控。老师：**限流和熔断**（circuit breaking）**是韧性手段，不是性能**（performance）**优化**；不要用服务网格回答"网关做什么"。

❝ 可用句式｜___ 处填案例内容

The API gateway is the single entry point. It routes each request to the right service, checks the token, applies rate limits, translates protocols and combines results, so each service only handles business logic.

> API 网关是统一入口。它把每个请求路由到正确的服务、校验令牌、执行限流、转换协议、合并结果，这样每个服务只处理业务逻辑。

预备卡 F：托管服务还是自建

见 T10 卡的判断表 p.**35**。补两句老师的话：托管 = 订阅（subscribe）+ 厂商负责全生命周期 + SLA，但**宕机怎么算写在细则里**；**多云**（Multi-Cloud）**或强可移植**时，厂商专属方案（如厂商 Operator）是陷阱。

❝ 可用句式｜___ 处填案例内容

I choose a managed service because the traffic is bursty and the team is small: we pay per use and the vendor runs upgrades and scaling under an SLA. The risk is vendor lock-in, so we keep the data in a portable format.

> 我选托管服务，因为流量突发、团队小：按用量付费，升级和扩缩由厂商按 SLA 负责。风险是厂商锁定，所以数据保持可移植的格式。

Moving from IaaS (EC2) to CaaS (Amazon EKS) shifts OS patching and cluster management to AWS, so the small team only builds container images and Kubernetes manifests; the trade-off is less control and some vendor lock-in.

> 从 IaaS（EC2）换到 CaaS（Amazon EKS），操作系统补丁和集群管理交给 AWS，小团队只需构建容器镜像和 K8s 配置；代价是控制更少、有一定厂商锁定。

For short, event-triggered tasks such as _____, I would use FaaS (AWS Lambda): we only write the function, pay per invocation and pay nothing when idle; the drawback is cold-start latency.

> 对于 _____ 这类短小、事件触发的任务，我会用 FaaS（AWS Lambda）：只写函数，按调用付费，闲时不花钱；缺点是冷启动延迟。

预备卡 G：单体还是微服务（什么时候拆）

怎么认： 问"该不该改成微服务（microservices）""先拆哪个"。老师：微服务是**用复杂度换扩展**；简单应用单体（monolith）更高效（Fowler："系统复杂到无法当单体管理，再考虑微服务"）。

拆的理由	不拆的理由
不同部分负载差别大，要独立扩展（scale independently）	团队小、功能简单、要快速上线
发布节奏不同，要独立部署（deployment）	网络调用带来延迟（latency）和运维成本
团队多，要各自负责一个限界上下文（bounded context）	分布式数据、事务（transaction）、排障更难

拆法： 按限界上下文；先拆最常用、负载最大的功能（Netflix：先按计费、流媒体、趋势分成独立技术栈）。

预备卡 H：怎么证明可维护性 / 质量

怎么认： 问怎样提高或证明可维护性（maintainability）、怎么保证改动不出错。老师：维护成本里**改进型维护占 70–75%**；要给**证据**：覆盖率、每服务测试数、静态扫描（SonarQube）结果。

手段	要点
四个战术	拆分模块、提高内聚（High Cohesion）、降低耦合（Low Coupling）、推迟绑定
架构层	限界上下文（bounded context）拆服务；事件驱动（新功能只加订阅（subscribe）者）；API 版本（version）管理；外部化配置（Externalised Configuration）
可靠变更	CI/CD、自动化测试、IaC、能回滚（rollback）；**适应度函数**（Fitness Function）不过不许上生产
可分析性（Analyzability）	集中日志（logs）+ 关联（Association）ID、追踪（tracing）、指标（metric）

github.com/bibi45c · 免费分享，禁止售卖

❝ 可用句式 | ___ 处填案例内容

Maintainability is shown with evidence: test coverage per service, static-analysis results from SonarQube and fitness functions in the CI/CD pipeline; a change cannot reach production unless they pass.

可维护性要用证据证明：每个服务的测试覆盖率、SonarQube 静态扫描结果、CI/CD 流水线里的适应度函数；它们不通过，改动就不能上生产。

The producer only publishes events and does not know who consumes them, so a new feature such as _____ is added as a new subscriber without changing the existing services; a schema registry versions the event format.

生产者只发布事件，不知道谁在消费，所以像 _____ 这样的新功能只需新增一个订阅者，不用改现有服务；schema 注册表负责管理事件格式的版本。

实体标签建议（按这个贴，和每页右上角的黑色标签同名）

标签	起始页	标签	起始页
总纲·速认	p.**5**	T13–17	p.**40**
超纲·重点	p.**8**	PAST 范文（model answer）	p.**43**
T4 找错（error finding）	p.**13**	PAST2 范文	p.**48**
T1/T2 领域	p.**16**	PAST3 范文	p.**54**
T3 API	p.**20**	英文句型	p.**58**
T5 架构+K8s	p.**23**	笔记回忆	p.**62**
T6 云服务	p.**27**	常用数据	p.**67**
T7–T12 小题	p.**30**	术语词典	p.**71**
T11 性能（performance）	p.**38**	索引	p.**79**

github.com/bibi45c · 免费分享，禁止售卖

PART 1 题型卡 T1–T17（怎么认・怎么写）

T4 打包 / 部署模型找错（每卷必考 6 分）

✎ 考试怎么考 | 题型·分值·套路 **怎么认・分值**

题干（question stem）出现 **logical packaging and deployment model / subsystem / manifest / deploy / undesirable architectural decision / complete the packaging**，附一张"不完整、可能有错"的图。PAST Q3(c)、PAST2 Q3(a)、PAST3 Q2(a)，**每卷 6 分。** 评分：**每个错 = 识别**（identify）**1 + 纠正**（correction）**1 + 论证**（justification）**1。**要两个就写两个（PAST），要三个写三个（PAST3），"找一个 + 补全打包（packaging）"各约 3 分（PAST2，推断）。

先懂两条规则：服务 × 层 = 盒子，部署是整盒搬

规则	白话	例子
子系统（subsystem）**名 = 服务名 + 层名**	一个盒子只装"同一个服务、同一层"的东西	Voting Manager Control、Notification Manager Data Access
BCE 放哪层	boundary（界面 / 外部接口）→ Presentation 或 External Integration；control（流程）→ Control；entity（数据）→ Data Access	VoteCastingScreen → Presentation；PaymentGatewayAdapter → External Integration
实体（entity）**跟着服务走**	实体属于哪个服务（看 §7 领域模型（domain model）），就放进**那个服务**的 Data Access	Coupon 属营销服务 → Marketing Manager Data Access
manifest ≠ deploy	manifest = 盒子里装什么（打包（packaging））；deploy = 盒子放到哪台机器（部署（deployment））	通知控制器放手机：错在 **deploy**，不是打包
整盒搬	deploy 线只能画在子系统上；盒子放到哪，里面的实体就跟到哪	数据库节点（node）放错区：节点上所有子系统一起搬
区域（zone）**规则**	DMZ 只放对外的反向代理（reverse proxy）和 Web 层；**数据库永远在内部区**（internal zone）；对接外部系统的 boundary 放 External Integration 层，部署在后端（backend）	Database server 在 DMZ → 移到 Internal Zone
服务 ≠ 服务器	服务 service 是业务划分，服务器 server / 节点 node 是机器	写"营销服务"，不写"营销服务器"

找错清单：8 项逐条扫

#	检查项	判断依据	典型纠正
①	实体（entity）打包（packaging）对不对	对照 §7：实体属于哪个服务	改打包到所属服务的 Data Access
②	组件放对层没	boundary / control / entity 各归各层	移到对应层的子系统（subsystem）
③	子系统混了多个服务	一个子系统只属于一个服务	按服务拆开
④	**前端**（front end）**部署**（deployment）**了后端**（backend）**职责**	手机只放用户交互（interaction）、设备必须的本地逻辑和本地存储（local storage）	删掉发通知、调外部系统、共享数据到手机的 deploy
⑤	**前端缺必须的部署**	用例要求离线（offline）、本地存储、本地凭证	补上 Data Access 到手机的 deploy
⑥	外部集成没经过集成层	§6 规定 External / Enterprise Integration 层对接外部系统	新建 **X Manager External Integration** 子系统，部署在后端
⑦	位置 / 区域（zone）不合理	存储在内部区；对外入口经 DMZ 反向代理（reverse proxy）	把数据库节点（node）移到 Internal Zone；补 DMZ
⑧	还没打包的元素	题目要求 **complete the packaging**	按"服务名 + 层名"新建子系统并画 manifest

PAST3 那种**逻辑详细部署图**（只有节点和子系统）另加四个维度：**分层是否完整**（有 Presentation 没 Control）、**应用与平台是否分清**（只给一个应用用的不该在 Platform 节点）、**外部集成是否走集成层、有没有 DMZ。**

✕ 常见误区 | 这样答会丢分 **先读图注，再找错**（error finding）

- 图注写 **deliberately omitted / incomplete**（如"刻意省略后端部署"）→ **没画的部分不算错。**
- **每道题以附录原图为准**，不要把前面小题自己提出的改造当成前提（assumption）。
- 先把需求（§2 用例）和领域模型（domain model）（§7）看完再找错；没看完需求，最容易漏掉 OrderNotif 这类实体的归属。
- 图里画着外部系统（支付网关（gateway）、Google Fit）时先问：这是**我们要部署的**，还是**外部已有的**？外部系统不是纠正（correction）对象。

下笔骨架：三段式（每段 1 分）

段	写什么	最少写到
Undesirable decision（识别（identify））	点名**哪个子系统**（subsystem）**/ 实体**（entity）**/ 哪条线**	用完整子系统名
Correction（纠正）	删哪条线、加哪条线、改打包（packaging）到哪	动作 + 对象 + 去向
Justification（论证）	只用**题目给的依据**：§7 归属、§2 用例要求、§6 分层、§5 DMZ	不加题目没给的假设（assumption）

github.com/bibi45c · 免费分享，禁止售卖

“ 可用句式 | ___ 处填案例内容 **T4 句子库（换名字即可）**

Undesirable decision 1: Entity _____ is packaged into the _____ Data Access subsystem.
不理想的决策 1：实体 _____ 被打包进了 _____ Data Access 子系统。

Correction: Package it into the _____ Data Access subsystem instead.
纠正：改为打包进 _____ Data Access 子系统。

Justification: In the domain model (Section 7), _____ belongs to the _____ service. An entity should be packaged with the service that owns it, so the two services stay loosely coupled.
论证：在领域模型（第 7 节）中，_____ 属于 _____ 服务。实体应该和拥有它的服务打包在一起，这样两个服务保持低耦合。

The _____ subsystem is deployed to the mobile device. This deployment should be removed, and the subsystem stays on the backend, because _____ (e.g. notifications are sent by the platform, not by the phone).
_____ 子系统被部署到了手机。应删除这条部署，让它留在后端，因为 _____（如通知由平台发送，不是由手机发出）。

The _____ Data Access subsystem is not deployed to the mobile device. A deploy link to the Mobile Device with Local Storage should be added, because the use case _____ requires _____ to be stored on the device.
_____ Data Access 子系统没有部署到手机。应加一条到 Mobile Device with Local Storage 的部署，因为用例 _____ 要求 _____ 保存在设备本地。

The Database server is placed in the DMZ. It should be moved to the Internal Zone, because the DMZ only holds the reverse proxy and the web tier; databases must not be exposed to the Internet.
Database server 被放在了 DMZ。应移到 Internal Zone，因为 DMZ 只放反向代理和 Web 层，数据库不能暴露给互联网。

_____Adapter talks to the external _____, but it is packaged into the _____ Control subsystem. Create a _____ External Integration subsystem for it and deploy it on the backend. Then changes in the external system only affect the integration layer.
_____Adapter 对接外部 _____，却被打包进 _____ Control 子系统。为它新建 _____ External Integration 子系统并部署在后端。这样外部系统的变化只影响集成层。

Only the changed part of the diagram is drawn below: removed links are crossed out, and new links are marked "new".
下面只画了图中改动的部分：删除的连线打叉，新加的连线标 "new"。

“ 可用句式 | ___ 处填案例内容 **T4 补充：先说"检查了什么"，支付相关的常见错**

I checked each entity against the domain model (Section 7): _____ belongs to the _____ Manager and is correctly packaged in _____ Data Access, so it is not an error.
我按领域模型（第 7 节）逐个核对了实体：_____ 属于 _____ Manager，打包在 _____ Data Access 是对的，不算错误。

Undesirable decision: _____ is packaged into the Payment Manager Data Access subsystem. **Correction:** package it into the _____ Data Access subsystem. **Justification:** in the domain model it belongs to the _____ Manager; an entity must stay in the Data Access layer of the service that owns it, never in a Presentation subsystem.
不理想的决策：_____ 被打包进 Payment Manager Data Access 子系统。纠正：改打包进 _____ Data Access 子系统。论证：领域模型中它属于 _____ Manager；实体必须留在所属服务的 Data Access 层，绝不能放进 Presentation 子系统。

Undesirable decision: the Payment Manager Control subsystem is deployed to the mobile device. **Correction:** remove this deployment; it stays on the backend. **Justification:** payments are captured by the backend through the external Payment Gateway, and PCI-DSS requires payment handling in as few subsystems as possible.
不理想的决策：Payment Manager Control 子系统部署到了手机。纠正：删除这条部署，留在后端。论证：支付由后端通过外部 Payment Gateway 完成，PCI-DSS 要求支付处理只在尽量少的子系统里进行。

Undesirable decision: _____Adapter, the boundary to the external Payment Gateway, is packaged into the _____ Control subsystem. **Correction:** create a Payment Manager External Integration subsystem for it and deploy it on the backend in the internal zone. **Justification:** boundaries to external systems belong to the External Integration layer (Section 6), and payment handling should stay in the payment service (PCI-DSS).
不理想的决策：对接外部 Payment Gateway 的边界 _____ Adapter 被打包进 _____ Control 子系统。纠正：为它新建 Payment Manager External Integration 子系统，部署在后端内部区。论证：对接外部系统的边界属于 External Integration 层（第 6 节），支付处理应留在支付服务内（PCI-DSS）。

真题：PAST Q3(c) 官方答案的三个错

#	识别	纠正	论证	清单项
1	Notification Manager Control **部署**（deployment）**到了手机**	删除这条 deploy，留在后端（backend）	通知由平台发出，手机只负责投票界面和本地存储（local storage）	④
2	Voting Manager Data Access **没部署到手机**	补上到 Mobile Device 的 deploy	用例要求投票细节（CastVote）存在选民设备本地作为凭证	⑤
3	实体（entity） VotingCompletionNotif 被打包（packaging）进 Voting Manager Data Access	改打包进 Notification Manager Data Access	领域模型（domain model）中它属于 NotificationManager 服务	①

三个里选两个写，范文（model answer）见 p.**47**。PAST2 Q3(a)（DesirableSpeed 打包错 + 补全 Analytics / Optimisation 打包）见 p.**52**；PAST3 Q2(a)（缺 Control、应用平台放反、没有 DMZ）见 p.**56**。

github.com/bibi45c · 免费分享，禁止售卖

× 常见误区｜这样答会丢分 **常见错误（别犯）**

1. 把"部署错"写成"打包错"：通知控制器放手机，纠正（correction）是**删除部署。**
2. 凭空新建"购物车子系统（subsystem）"：Cart 属于 Order Manager，正确是把 **Order Manager Data Access 整盒**部署到手机。
3. 论证（justification）里加了题目没给的假设（assumption）（"订单用户常看，所以放手机"）。
4. 子系统名写不全：要写 **Notification Manager Data Access**，不能写"新建 Notification Manager"。
5. 把支付网关（gateway）当成要"迁移（migration）"的对象；把 External Integration 子系统放进 DMZ（应在后端内部区（internal zone））。
6. 区域（zone）论证写成"无需连接外部服务"：规则是 **DMZ 只放代理和 Web 层，数据库在内部区。**

github.com/bibi45c · 免费分享，禁止售卖

PART 1　题型卡 T1–T17（怎么认 • 怎么写）

T1　领域模型诊断　•　T2　画类图与共享服务

✎ 考试怎么考 | 题型•分值•套路　怎么认 • 分值

T1：附了 §7 领域模型（domain model）图（`<<service>>` 框 + 实体（entity）），问 **identify the most likely cause / propose a countermeasure by changing the domain model / the main drawback (if any)**。PAST Q1(b)、PAST2 Q1(b)，**两小问共 9 分**（5 + 4）。
T2：没有领域模型，给用例流程，要你 **use DDD to identify domain objects and draw a UML class diagram**，再 **identify shared services**。PAST3 Q1(b) 共 9 分。**"领域模型"这道大题，要么是 T1，要么是 T2。**

T1 判断总纲：五类症状

这些东西是不是总一起用、必须一起成立? 总一起用 / 必须同时成立 → 放一起；允许稍后对齐、负载差别大、通用与特有混在一起 → 拆开。

症状（题干关键词）	原因	对策（改领域模型）	缺点
慢、不及时 promptly / quickly / latency / timely	一起用的实体（entity）分在**不同服务**，每次都要**跨进程通信**（inter-process communication），比进程内调用（in-process call）慢得多；或一起变化的实体被拆成**多个小聚合**（small aggregate），一次计算要几次事务（transaction）	把相关实体**迁到同一个服务**（或合成一个聚合（aggregate））；**或**在使用方保留**只读副本**（read-only copy），用领域事件（domain event）同步（synchronous）	服务变大、事务范围（transaction scope）变大（可在服务内分多个聚合缓解（mitigate））；副本（replica）：数据重复、最终一致（eventually consistent）
难以复用 reuse / other platforms / drastic adaptation	通用服务（generic service）里**混入了业务特有实体**	特有实体**迁回各自业务服务**（点名去哪），通用服务只留通用部分	**通常没有大缺点**（drawback）；各服务还能独立扩展（scale independently）
内存占用高 memory / footprint	**聚合太大**：加载一个就载入整个对象图	按"设计小聚合"拆开，**只用 ID 引用父聚合**，按需加载（有强规则的留在聚合内）	读关联（Association）数据要多次查询；跨聚合更新变成最终一致
无法独立扩展 scale independently	负载差别很大的功能放在同一个服务	**只把高负载的**实体和功能拆成独立服务	服务间通信增多，一致性更难保证
数据不一致 consistency / invariant / rule violated	必须同时更新的实体分在**不同聚合或服务**，没人守得住规则	**合并成一个聚合**，由聚合根（aggregate root）检查规则	聚合变大；热门时并发（concurrency）冲突增加，部分请求要等待或重试（retry）

"慢"类选对策（countermeasure）：搬到一起，还是留副本?

问自己	偏向"搬到一起"	偏向"留只读副本"
晚几秒更新会出事吗?	会：**库存、余额、座位**	不会：商品描述、统计数、位置、推荐
有"必须同时成立"的规则吗?	有：库存不能超卖	没有
数据只读吗?	要被改：别用副本	只读：适合副本
搬过来服务会不会杂得离谱?	不会，才适合搬	会，就用副本

库存不能用副本：副本晚更新会导致超卖。**选副本的前提**（assumption）**是这条规则允许短暂延迟**（latency），并在答案里写一句 **Assume _____.**

✕ 常见误区 | 这样答会丢分　T1 易错（老师明确说过）

1. 答"加缓存（cache）、加负载均衡（Load Balancing）、加机器、按年月分表"：**偏题**，题目要求通过**修改领域模型**（domain model）解决。
2. 原因（cause）写笼统（"设计不合理"）：必须点名**哪个实体在哪个服务**，所以导致什么。
3. 对策不点名**搬去哪个服务**。
4. 缺点不写：没有也要写 **No major drawback** + 原因。
5. 用自己的猜测当原因（"处方太多"）：只用题目给的事实。
6. 拆聚合拆过头：有"必须同时成立"规则的实体不能拆（网银 Beneficiary"不超过 5 个"要留在 Account 里）。
7. 内存题引用的是**父聚合的 ID**（courseId），不是子实体自己的。
8. 两小问的对策互相矛盾（PAST2：(i) 合并进 BusNetworkManager，(ii) 又说它太大）。

T1 下笔骨架与句子

段	PAST 5 分题	PAST 4 分题
Cause	2	1.5
Countermeasure	1.5 + 1.5（两半：特有分离 + 通用内聚（cohesion））	1.5
Drawback	0（但要写）	1

github.com/bibi45c · 免费分享，禁止售卖

“ 可用句式 | ___ 处填案例内容 **原因 Cause**

Cause: _____ is in the _____ service, but _____ is in the _____ service. To _____ (use case), the two services must work together, so every request needs inter-process communication, which is much slower than an in-process call.

> 原因：_____ 在 _____ 服务里，而 _____ 在 _____ 服务里。为了 _____（用例），两个服务必须协作，所以每个请求都要跨进程通信，比进程内调用慢得多。

Cause: The _____ service contains business-specific entities such as _____ and _____. Other platforms do not need them, so the service cannot be reused without big changes.

> 原因：_____ 服务里有 _____、_____ 这类业务专属实体。别的平台用不到它们，所以不大改就无法复用这个服务。

Cause: _____ and _____ are in one aggregate. Loading one _____ loads all of its _____ objects into memory, so each instance uses a lot of memory.

> 原因：_____ 和 _____ 在同一个聚合里。加载一个 _____ 就会把它所有的 _____ 对象载入内存，所以每个实例都很占内存。

Cause: _____ and _____ must change together (the rule "_____"), but they are in different aggregates / services, so no single aggregate can protect the rule.

> 原因：_____ 和 _____ 必须一起变化（规则"_____"），但它们分在不同的聚合 / 服务里，没有一个聚合能守住这条规则。

“ 可用句式 | ___ 处填案例内容 **T1 补充：原因先点名"哪条跨框连线"**

In Figure _____, the association between _____ (in the _____ Manager) and _____ (in the _____ Manager) crosses the service boundary, and _____ is derived from _____.

> 在图 _____ 中，_____（在 _____ Manager）和 _____（在 _____ Manager）之间的关联跨越了服务边界，_____ 是由 _____ 推导出来的。

Every change of _____ (_____ updates per _____) must therefore be sent from the _____ Manager to the _____ Manager by inter-process communication before _____ can see it, which makes it hard to meet the _____-second requirement.

> 因此每次 _____ 变化（每 _____ 有 _____ 次）都要先从 _____ Manager 跨进程传到 _____ Manager，_____ 才能看到，很难满足 _____ 秒的要求。

Countermeasure: move _____ from the _____ Manager into the _____ Manager, next to _____, so the update and the read happen in one service.

> 对策：把 _____ 从 _____ Manager 移到 _____ Manager，和 _____ 放在一起，让更新和读取在同一个服务里完成。

Drawback: the _____ Manager now receives all _____ updates and becomes larger; the _____ Manager has to read _____ through an API or events, which is eventually consistent.

> 缺点：_____ Manager 要接收全部 _____ 更新，变得更大；_____ Manager 要通过 API 或事件读取 _____，只能最终一致。

“ 可用句式 | ___ 处填案例内容 **对策 Countermeasure**

Countermeasure: Move _____, _____ and _____ from the _____ service into the _____ service, so the entities that are always used together are in one service. (Only the changed service boxes are drawn.)

> 对策：把 _____、_____、_____ 从 _____ 服务移到 _____ 服务，让总是一起用的实体在同一个服务里。（只画改动的服务框。）

Countermeasure: Assume _____ only reads _____. Keep a read-only copy of _____ in _____, updated by a _____ Updated domain event from _____.

> 对策：假设 _____ 只读取 _____。在 _____ 里保留 _____ 的只读副本，由 _____ 发出的 _____ Updated 领域事件更新。

Countermeasure: (1) Move each specific entity back to the service that owns it: _____ to _____, _____ to _____. (2) Keep only the generic _____ in the _____ service, so it becomes a generic, cohesive service that other platforms can reuse.

> 对策：（1）把每个专属实体移回拥有它的服务：_____ 到 _____，_____ 到 _____。（2）_____ 服务只保留通用的 _____，成为通用、内聚、别的平台能复用的服务。

Countermeasure: Split the large aggregate into small aggregates _____ and _____. _____ refers to _____ only by its ID (_____Id), so only the needed objects are loaded.

> 对策：把大聚合拆成小聚合 _____ 和 _____。_____ 只通过 ID（_____Id）引用 _____，只加载需要的对象。

Countermeasure: Move _____ into the _____ service and put it in the same aggregate as _____. The aggregate root _____ checks the rule "_____" in one transaction.

> 对策：把 _____ 移进 _____ 服务，和 _____ 放在同一个聚合里。由聚合根 _____ 在一个事务里检查规则"_____"。

Countermeasure: Move only the high-load _____ into a new _____ service, so it can scale independently; _____ stays in the original service.

> 对策：只把高负载的 _____ 拆成新的 _____ 服务，让它独立扩展；_____ 留在原服务。

github.com/bibi45c · 免费分享，禁止售卖

❝ 可用句式 | ___ 处填案例内容 **缺点 Drawback**

Drawback: The _____ service becomes larger and less cohesive, and its transaction scope grows. This can be reduced by using several aggregates inside the service.
缺点：_____ 服务变大、内聚下降，事务范围变大。可以在服务内划分多个聚合来缓解。

Drawback: The data is duplicated and the copy is only eventually consistent, so the displayed _____ may be a few seconds old. This is acceptable because _____ rarely changes.
缺点：数据重复，副本只是最终一致，显示的 _____ 可能晚几秒。因为 _____ 很少变，这可以接受。

Drawback: Reading related data needs more queries, and updates across aggregates are only eventually consistent.
缺点：读取关联数据要多查几次，跨聚合的更新只能最终一致。

Drawback: No major drawback. Each kind of _____ now scales independently with its own service.
缺点：没有大的缺点。每类 _____ 现在随自己的服务独立扩展。

Drawback: The aggregate becomes larger. At busy times more users update the same aggregate, so some requests must wait or retry.
缺点：聚合变大。高峰时更多用户同时修改同一个聚合，部分请求需要等待或重试。

T2 画类图与共享服务

五步做法

步	做什么	注意
1	逐条读用例流程，圈**名词（候选实体**（entity）**）**和**动词（候选关系）**	"检查有没有空笼位" → 笼位是实体
2	删掉**只是属性的名词**（日期、数量、押金金额）和**外部系统**（NHS、Payment Gateway）	订单、评价是我们自己的实体，不能删
3	画实体和关系，标**多重性**（multiplicity）；"是一种"用继承（inheritance），"由……组成"用组合（composition）	可选关系写 **0..1**（器材订单不需要处方）
4	按功能内聚（cohesion）分组，每组画一个 `<<service>>` 框；每个实体写 **2–3 个属性**	不能只画实体不分服务
5	共享服务（shared service）：**两个以上应用都要用**的功能 + §2/§3 点名的扩展方向	每个写"给谁用、做什么"

类图（class diagram）符号

符号	含义	例
实线 + 两端数字	关联（Association），多重性 `1`、`0..1`、`*`、`1..*`	`Patient 1 — 0..* Order`
空心菱形 ◇	聚合（aggregate）：部分可独立存在	Poll ◇— Candidate
实心菱形 ◆	组合：部分随整体创建和销毁	Order ◆— OrderItem（1 — 1..*）
空心三角 ▷	继承 / 泛化，箭头指向父类	Medicine —▷ Product

多重性怎么读： `Order * — 1 Pet` = 一只宠物可以有很多张订单，一张订单只对应一只宠物。

共享服务判断问句： 如果现在去做外卖平台，这个服务能原样拿去用吗？**用户与身份、支付、通知、目录、订单、配送跟踪、报表**能；带笼位、宠物的不能。通用域（generic subdomain）（各行业都要、与业务无关）最适合共享，判断标准**不是"简单"**。

❝ 可用句式 | ___ 处填案例内容 **T2 句子库**

Nouns in the use cases become candidate entities; attributes such as dates and quantities, and external systems such as _____, are not modelled as entities.
用例里的名词是候选实体；日期、数量这类属性，以及 _____ 这类外部系统，不建模为实体。

_____ places _____ **(1 to 0..);** _____ **contains** _____ **(composition, 1 to 1..);** _____ and _____ are kinds of _____ (inheritance).
_____ 下 _____（1 对 0..）；_____ **包含** _____ **（组合，1 对 1..）**；_____ 和 _____ 是 _____ 的子类（继承）。

The entities are grouped into services by functional cohesion: _____Manager holds _____ and _____, because they are always used together.
实体按功能内聚分到服务里：_____Manager 放 _____ 和 _____，因为它们总是一起使用。

_____ Service should be shared, because _____ and _____ both need it to _____; building it once speeds up future applications such as _____.
_____ 服务应该共享，因为 _____ 和 _____ 都要用它来 _____；只建一次能加快以后 _____ 这类应用的开发。

例子：宠物寄养 App（参考分法，不唯一）

服务	实体	依据
UserManager（共享）	Owner	用户与身份，通用域
BookingManager	BoardingOrder、Pet、Photo	围绕一次寄养，总一起用
ShopManager	Shop、Kennel、Staff、Review	围绕店的运营和口碑
PaymentManager（共享）	Payment	支付，通用域；订单里只留押金金额属性
NotificationManager（共享）	不画实体	通知交给它

关系：Owner 1— **Pet；BoardingOrder** —1 Pet；BoardingOrder **—1 Shop；BoardingOrder 1—** Photo；BoardingOrder 1— **Payment；Shop 1—** Kennel；Shop 1—1.. **Staff；Shop 1—** Review。**宠物和店的联系是通过订单产生的，不能直接连 1—1。**

github.com/bibi45c · 免费分享，禁止售卖

真题位置

样卷	题	症状 / 任务	范文
PAST	Q1(b)(i) 5 分	NotificationManager 难以复用（reuse）	p.**44**
PAST	Q1(b)(ii) 4 分	Observer 看结果不及时	p.**44**
PAST2	Q1(b)(i) 5 分	到站时间算得慢（三个小聚合（small aggregate） + 跨服务（cross-service））	p.**48**
PAST2	Q1(b)(ii) 4 分	BusNetworkManager 内存高（BusRoute+BusTrip 大聚合（aggregate））	p.**48**
PAST3	Q1(b) 9 分	从用例画类图（class diagram） + 共享服务（shared service）	p.**54**

github.com/bibi45c · 免费分享，禁止售卖

PART 1 题型卡 T1–T17（怎么认 · 怎么写）

T3 API 设计：RMM · 版本 · 性能重构 · Level 3

✎ 考试怎么考 | 题型·分值·套路 **怎么认 · 分值 · 三种问法**

题干（question stem）出现 **API / URI / HTTP verb / RMM Level 2 or 3 / versioning / refactor the API / outside-in / illustrate with a concrete example**。每卷必考，约 **6 分**。 ① 给一张**有错的 API 表**：是否 Level 2、更好的版本（version）策略、重画表（PAST Q1(c)，三问各 2 分 = 解释 + 改进；各 1 分是推断） ② **大批量并发**（concurrency）**更新**为什么延迟（latency），重构 API 并举例（PAST2 Q1(c)，原因（cause）约 2 + 重构约 2 + 例子约 2） ③ 选一个共享服务（shared service），**设计 RMM Level 3 API**：URI、动词、输入输出、用途（PAST3 Q1(c)(ii) 约 4 分）

必备表 1：HTTP 方法与状态码

方法	用途	安全	幂等
GET	读取，**不能改变服务器状态**	是	是
POST	创建资源（Resource） / 追加事件 / 触发会改状态的动作（邀请、开始投票）	否	否
PUT	**整体替换**已有资源	否	是
PATCH	**部分更新**（只传要改的字段）	否	否（可设计成幂等（idempotent））
DELETE	删除	否	是

码	什么时候用	码	什么时候用
200 OK	GET 成功；PUT / PATCH 更新成功	400	请求格式或参数错（校验失败仍同步（synchronous）返回）
201 Created	POST **当场**建好了资源	401 / 403	未认证（Authentication） / 无权限
202 Accepted	**收下了，后台异步**（asynchronous）**处理**（异步重构的标志）	404 / **409**	不存在 / **状态冲突**（已发货还改地址）
204 No Content	DELETE 成功	429 / 503	超过限流（rate limiting） / 暂不可用

201 还是 202？问"事情做完了吗"：做完 = 201，只是收下 = 202。

必备表 2：RMM 四级（从 Level 0 编号）

等级	判断标准	例子
Level 0	只有**一个 URI、一种方法**	`POST /voteService` + `{"action":"startVote"}`
Level 1	**多个 URI**，但**所有操作同一种方法**	`POST /orders/get` 、`POST /orders/cancel`
Level 2	资源（Resource）名词 + **正确的动词** + **正确的状态码**（status code）	`GET /polls/42` → 200；`POST /polls/42/votes` → 201
Level 3	Level 2 + **响应里带超媒体链接**（hypermedia link）（rel + href），链接**随状态变化**	`"links":[{"rel":"cast-vote","href":"/polls/42/votes"}]`

三问判断： 只有一个地址？→ L0。所有操作同一种方法？→ L1。响应里有 links 吗？没有 → L2，有 → L3。写 "Level 1"，**不要写"第二级"**。

必备表 3：版本

段	什么时候加	例
MAJOR	**不兼容**：删字段、新增**必填**参数	1.1.0 → 2.0.0
MINOR	向后兼容（backward compatible）的新功能：新增可选字段、新增接口	1.1.0 → **1.2.0**（先看当前版本（version）！）
PATCH	向后兼容的修复	1.0.0 → 1.0.1

方法	写法	评价
URI 版本	`/v1/votes`	**版本废弃时 URI 变，客户端代码出错**
自定义请求头（request header） **（推荐）**	`Accept-version: 1.1`	**URI 保持稳定**，客户端只换请求头里的版本号
Accept 媒体类型	`Accept: application/vnd.app.v1+json`	标准内容协商（Content Negotiation），略复杂

问法 ①：改错表（PAST Q1(c)）

三个改动 = 评分点： ① **GET 改 POST**（会改状态的操作）② **去掉 URI 里的版本**（version）**号**，改请求头（request header） ③ **删掉多余的输入输出**（按用例逐个核对）。

✕常见误区 | 这样答会丢分 **易错**

只改了动词，忘了删多余输入输出、忘了去掉 `/v1.1`；不看用例就假设（assumption） invite 要传 voter-id；版本号和 RMM 等级混为一谈。

github.com/bibi45c · 免费分享，禁止售卖

问法 ②：API 性能题的判断链（PAST2 Q1(c)）

#	问自己	如果是	措施	答案里的句子
1	调用方要不要**在这次响应里拿到计算结果**?	不要，只要知道收到了	**异步**（asynchronous）**接收**	**The API validates the request and returns 202 Accepted immediately; the processing is done in the background.**
2	每次调用是"新发生一件事"还是"替换整个东西"?	新发生一件事	**PUT 改 POST**，在自己的资源（Resource）下追加事件	**Each _____ sends a POST under its own resource to append an event.**
3	请求是不是太碎?	是	**批量接口**（Batch Endpoint）或事件流（event streaming）摄取（**先看案例是否禁止批量**：PAST2 乘客事件规定逐条上报）	**Provide a batch endpoint so one request carries many updates.**
4	请求里有没有**服务器本来就知道**的信息?	有	**删掉冗余**（redundancy）**输入**	**The _____ is derived from the _____, so it is removed from the input.**
5	同一对象的**事件顺序重要吗**?	重要	事件流**按该对象的 ID 分区**（partition）	**The event stream is partitioned by _____ ID, so the events of one _____ stay in order.**
6	后台处理得过来吗?	处理不过来	每个分区一个 worker **并行**（Parallel）**消费**	**The consumers process the partitions in parallel and in order.**

- **不能改异步的**：调用方必须立刻拿到结果才能继续（支付是否成功、登录校验）。
- **分区键**（partition key）**选"顺序必须一致的那个对象"**（包裹 ID、班次 ID），不选扫码枪 ID；分区数固定，很多键共用一个分区，不会"太细"。
- **URI 标识资源**，不放分区 ID、枪 ID：`POST /parcels/{parcel-id}/scan-events`，枪 ID 放请求体（request body）。
- 限流（rate limiting）（429）可作最后一句补充；"降低设备上报频率"慎用，可能违反案例要求。

❝ 可用句式 | ___ 处填案例内容 **性能**（performance）**题句子库**

Reason 1: The API is synchronous. Before it replies, the service recomputes _____, so thousands of concurrent requests wait in a queue.
> 原因 1：API 是同步的。回复之前服务要重算 _____，所以成千上万的并发请求在排队。

Reason 2: Every call uses PUT on one shared resource (_____), so concurrent updates overwrite and block each other, although each update is really a new event.
> 原因 2：每次调用都对同一个共享资源（_____）用 PUT，并发更新互相覆盖、互相阻塞，而每次更新其实是一个新事件。

Reason 3: The input contains _____, which the server can already find from _____, so the request body is larger than needed.
> 原因 3：输入里带了 _____，服务器本来就能从 _____ 知道，所以请求体比需要的大。

Refactoring: Make the API resource-oriented and asynchronous. Each _____ sends POST /_____/{id}/_____. The API checks the request, publishes it to an event stream (e.g. Amazon Kinesis) partitioned by _____ ID, and returns 202 Accepted at once. _____ consumes the partitions in parallel and in order.
> 重构：把 API 改成面向资源、异步。每个 _____ 发送 POST /_____/{id}/_____。API 校验请求，发布到按 _____ ID 分区的事件流（如 Amazon Kinesis），立即返回 202 Accepted。_____ 并行且按顺序消费各分区。

Example: POST /bus-trips/175-0832/positions with body {"position": "1.2966,103.7764", "speed": 32, "timestamp": "08:15:30"} returns 202 Accepted in a few milliseconds; the arrival times are updated within 5 seconds.
> 例子：POST /bus-trips/175-0832/positions，请求体 {...}，几毫秒内返回 202 Accepted；到站时间在 5 秒内更新。

❝ 可用句式 | ___ 处填案例内容 **T3 补充：例子要具体，只更新受影响的部分**

The refactored API only accepts what changes: POST /connectors/{connectorId}/status-events with {"status": ..., "timestamp": ...}. The _____ and _____ are removed, because the server already knows them (they belong to _____).
> 重构后的 API 只接收会变的内容：POST /connectors/{connectorId}/status-events，请求体 {"status": ..., "timestamp": ...}。删掉 _____ 和 _____，因为服务器本来就知道（它们属于 _____）。

Each event updates only the availability of the affected connector, not the map of the whole city, so the change is visible to drivers within _____ seconds.
> 每个事件只更新受影响那个充电口的可用状态，不重算全城地图，所以司机在 _____ 秒内就能看到变化。

Example: POST /connectors/C-1042/status-events with {"status": "Occupied", "timestamp": "18:02:10"} returns **202 Accepted** within milliseconds.
> 例子：POST /connectors/C-1042/status-events，请求体 {"status": "Occupied", "timestamp": "18:02:10"}，几毫秒内返回 202 Accepted。（例子必须有真实的 ID 和数值）

Meter values go to a separate resource: POST /sessions/S-88123/meter-readings with {"kWh": 12.4, "timestamp": "18:02:30"} → 202 Accepted.
> 电表读数走另一个资源：POST /sessions/S-88123/meter-readings，请求体 {"kWh": 12.4, "timestamp": "18:02:30"} → 202 Accepted。

问法 ③：设计 Level 3 API（PAST3 Q1(c)(ii)）

至少 4 个操作：列表、详情、更新、追加事件；每个写 **URI + HTTP + 输入 + 输出（含 links）+ 用途。**

github.com/bibi45c · 免费分享，禁止售卖

URI	HTTP	输入	输出（含链接）	用途
`/delivery-orders?supplierId=..&status=..`	GET	查询参数（Query Parameter）	200；列表；links: self, next, 每张单	查某供应商的配送单
`/delivery-orders/{no}`	GET	—	200；状态、预计时间、地址；links: self, update-status, order	查看单张配送单
`/delivery-orders/{no}/status`	PUT	status, timestamp	200；links: self, delivery-order	更新配送状态
`/delivery-orders/{no}/events`	POST	event, reason, 新预计时间	**201 Created**；links: self, delivery-order	记录一个配送事件

`PUT /…/status` 是整体替换 status 这个子资源（Resource），所以用 PUT（幂等（idempotent））；只改订单里的某个字段才用 PATCH。

links 三步法： ① 每个响应加 `self`；② 表格里其他操作逐个问"**当前状态允许吗**"，允许才加；③ 加相关资源（事件指回 `order`，列表加 `next`）。rel 名写"这个操作干什么"（`cancel-order`、`update-address`），不写空洞的 `update-timestamp`。状态不允许的请求返回 **409 Conflict**。

```
GET /orders/5  → 200 OK        （已付款未发货）
{ "id": 5, "status": "paid",
  "links": [ {"rel": "self",           "href": "/orders/5"},
             {"rel": "update-address", "href": "/orders/5"},
             {"rel": "cancel-order",   "href": "/orders/5"},
             {"rel": "ship",           "href": "/orders/5/events"} ] }
```

已发货后：`links` 只剩 `self`、`orders`（可加 `tracking`）

❝ 可用句式 | ___ 处填案例内容 **通用句子**

`GET /orders/5/cancel` is wrong, because cancelling changes the state, but GET must be safe and read-only. Use `DELETE /orders/5`, or `PATCH /orders/5` with {"status": "cancelled"}.

> `GET /orders/5/cancel` 是错的，因为取消会改变状态，而 GET 必须安全、只读。改用 `DELETE /orders/5`，或 `PATCH /orders/5`，体 {"status": "cancelled"}。

The API is not at RMM Level 2: _____ and _____ change the state on the server, but they use GET. They should use POST.

> 这个 API 不符合 RMM Level 2：_____ 和 _____ 会改变服务器状态，却用了 GET。应该用 POST。

I would put the version in a custom request header such as `Accept-version: 1.1`. The URI then stays stable; with URI versioning, the URI changes when a version is retired and client code breaks.

> 我会把版本放在自定义请求头里，如 `Accept-version: 1.1`。这样 URI 保持稳定；URI 版本在废弃某个版本时地址会变，客户端代码就会出错。

The input _____ is removed, because _____; the output only returns _____, which the caller needs.

> 删掉输入 _____，因为 _____；输出只返回调用方需要的 _____。

The API uses plural nouns, the correct HTTP verbs and status codes, and every response has hypermedia links to the next possible actions, so it is at RMM Level 3. It is versioned with the Accept-version header and protected with OAuth 2.0.

> 这个 API 用复数名词、正确的 HTTP 动词和状态码，每个响应都带指向下一步操作的超媒体链接，所以达到 RMM Level 3。用 Accept-version 请求头做版本管理，用 OAuth 2.0 保护。

✎ 考试怎么考 | 题型·分值·套路 **T3 检查清单**

- [ ] URI 里只有名词、复数、`/` 开头；动作交给 HTTP 方法
- [ ] 没有用 GET 做会改状态的操作；改一个字段用 PATCH 不用 PUT
- [ ] URI 里没有版本（version）号；版本放请求头（request header）
- [ ] 路径里已有的信息（{voter-id}）请求体（request body）里不重复；电话等敏感信息不放 URL
- [ ] 每个例子都有 **方法 + URI + 请求体 + 返回码**
- [ ] Level 3：每个响应有 links（rel + href），且随状态变化
- [ ] 题干（question stem）没说清的，用一句 **Assume** _____ 写出前提（assumption）

真题范文（model answer）：PAST Q1(c) p.**44** · PAST2 Q1(c) p.**49** · PAST3 Q1(c)(ii) p.**55**。

github.com/bibi45c · 免费分享，禁止售卖

PART 1　题型卡 T1–T17（怎么认・怎么写）

T5　云原生参考架构 + K8s 部署计划

✎ 考试怎么考 | 题型·分值·套路　**怎么认・分值**

题干（question stem）出现 **reference architecture / cloud-native / hybrid / monolith MVP / microservices, scheduled jobs, event-driven, serverless / internal communication / stateful service / Kubernetes deployment plan / added, removed, changed components。**PAST Q2(a) **10 分**，PAST2 Q2(a) **7 分。** PAST 评分：**参考架构**（Reference Architecture） **4**（服务、注册中心、状态机制、存储）+ **通信组件 2**（事件服务、通知服务）+ **缓存**（cache） **1** + **K8s 部署 3**（Pod、Deployment、Service）。

第 1 步：按负载给每块功能选风格（混合架构）

#	这块功能是	选	一句理由（英文）
1	**核心、持续使用**的交互（interaction），要稳定低延迟（latency）、独立扩缩	**微服务**（microservices）（容器（Container） + HPA）	**It is used all the time and needs low latency, so it runs as a microservice that scales out.**
2	**突发、短小**，由事件触发，平时没人用	**Serverless**（Lambda）	**It is short and bursty, so a serverless function scales with spikes and costs nothing when idle.**
3	**周期性**批处理（每晚、结束后）	**定时任务**（scheduled job）（CronJob）	**It runs on a schedule, so it is a scheduled job.**
4	**实时汇总**（aggregation），事件要有序、可重放、多个消费者（consumer）	**事件流**（event streaming）（Kafka / MSK / Kinesis）	**Events must stay in order and be read by several consumers, so we use an event stream.**
5	只是**任务分发、削峰**（peak shaving），一件事一人处理	**消息队列**（message queue）（SQS）	**Each task is done once by one worker, so a message queue is enough.**

答案几乎从来不是"选一种"，而是混合。 理由（reason）要写**负载特点**（§4 数字、§2 用例），不是"可复用（reusable）""独立部署（deployment）"。六边形架构（Hexagonal Architecture）是代码内部结构，**不是**部署风格，混合题里不选它。

第 2 步：照模板画组件（图 8-1，AWS 为例）

组件	AWS 产品	为什么需要
客户端	浏览器、手机 App（本地存储（local storage））、IoT 设备	
CDN + WAF	**CloudFront** + AWS WAF	静态内容就近分发；挡恶意流量
API 网关（gateway）	**Amazon API Gateway**	统一入口：路由、聚合（aggregate）、认证（Authentication）、限流（rate limiting）、监控（monitoring）
身份认证	**Cognito**	注册登录
容器（Container）平台	**EKS** + Istio 服务网格（service mesh）	微服务（microservices）独立部署（deployment），HPA 自动扩缩（auto-scaling）
服务注册 / 发现	K8s DNS（CoreDNS）	服务互相找到对方
Serverless	**Lambda**（+ SNS 发通知）	事件触发、突发、按调用付费
定时任务（scheduled job）	K8s **CronJob** / EventBridge Scheduler	报表、汇总（aggregation）
事件流（event streaming）	**Kafka（MSK）/ Kinesis**	有序、持久、可重放、多消费者（consumer）
消息队列（message queue）	**SQS**	任务分发、削峰（peak shaving）
内存缓存（In-Memory Cache）	**ElastiCache（Redis）**	实时计数、会话、热点数据
每服务一个库	**RDS（PostgreSQL）/ DynamoDB**	服务之间数据解耦
对象存储（object storage）	**S3**	报告、静态资源（Resource）
可观测性（Observability）	Prometheus + Grafana、ELK/EFK、Jaeger、**CloudWatch**	运行起来能看见问题
交付	CI/CD + GitOps（ArgoCD）、镜像仓库（Container Registry） **ECR**、IaC	自动、安全（security）地发布

多云（Multi-Cloud）**时额外加：** Kubernetes（可移植）、IaC（Terraform）、全局负载均衡（Load Balancing） / DNS 做故障转移（failover）、统一身份、统一监控、镜像仓库复制。

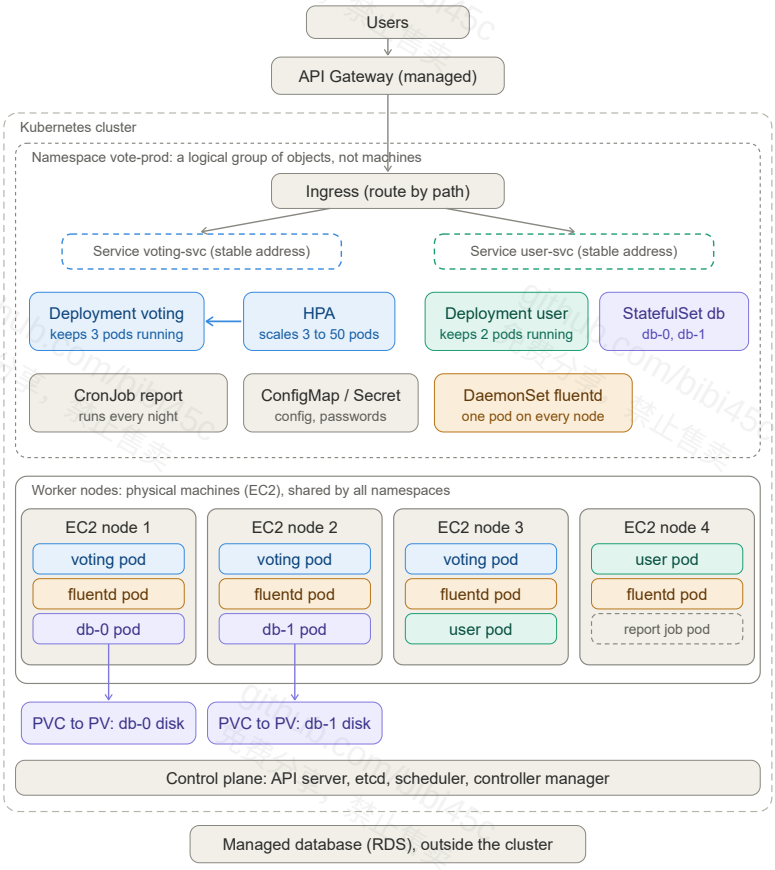
第 3 步：四个小问的答法

小问	写什么
(i) 组件与交互（interaction）	照上表画，**每个框写托管服务**（managed service）**名**，画出交互连线；多云（Multi-Cloud）写额外组件

github.com/bibi45c · 免费分享，禁止售卖

小问	写什么
(ii) 内部通信	需要立即拿结果的查询 → **同步**（synchronous） **REST/gRPC**（经服务发现（service discovery）、服务网格（service mesh）） ；解耦、削峰（peak shaving）、多消费者（consumer） → **异步**（asynchronous）： 要有序可重放 → 事件流（event streaming），一次性任务 → 消息队列（message queue）。**在图上标出**这些组件
(iii) 有状态（stateful）的服务	**点名一个服务** + 机制：默认**外部状态存储**（external state store） （ElastiCache Redis / RDS），任意 Pod 可替换；只有题目要求集群（Cluster）内自己跑才写 StatefulSet + PV/PVC。**不用粘性会话**（sticky session） （实例挂了会话就丢、负载不均、妨碍缩容（scale in））
(iv) K8s 部署（deployment）计划	**Pod、Deployment、Service 三个词必须出现**（3 分），见下

第 4 步：K8s 部署计划（3 分）



#	写什么
①	一个 **Namespace**（如 `vote-prod`）隔离（isolation）生产环境；**节点组**（Node Group）**跨多个可用区**（availability zone）
②	每个微服务（microservices）打包（packaging）成镜像（container image），以 **Deployment** 部署，**Pod 至少 2–3 个副本**（replica）；负载大的（voting、proctoring）配 **HPA**（如 5–50 个）；节点（node）不够时 **Cluster Autoscaler** 加节点
③	每个 Deployment 前面一个 **ClusterIP Service**：稳定地址 + 负载均衡（Load Balancing）；服务间经 CoreDNS 发现、Istio sidecar 通信
④	**Ingress** 按路径把外部请求路由到各 Service（前面接 API Gateway）
⑤	有状态（stateful）组件（Kafka、Redis、数据库）用**托管服务**（managed service），或 **StatefulSet + PVC**
⑥	报表用 **CronJob**；配置（configuration）和密码用 **ConfigMap / Secret**；日志（logs）代理用 **DaemonSet**
⑦	发布用**滚动更新**（rolling update）或**金丝雀**（canary）

副本数要给依据并写假设（assumption）： 最小看可用性（availability）（2–3 个，跨可用区）；最大 = 峰值（peak） ÷ 单个 Pod 承载量。**Assume one pod handles about 2,000 concurrent voters, so 100,000 voters need up to 50 pods. HPA 只给负载大的服务配。**

对象	一句话	对象	一句话
Pod	最小部署单元（Deployment Unit），不跨节点	Service	一组 Pod 的稳定地址 + 负载均衡（集群（Cluster）内）
Deployment	管无状态（stateless） Pod，保持副本数，滚动更新	Ingress	集群外的 HTTP 请求按路径分到 Service
StatefulSet	有固定名字和自己盘的有状态 Pod	HPA	改 Deployment 的副本数（加 Pod）
DaemonSet	每个节点一个 Pod（日志收集）	Cluster Autoscaler	加机器（节点）
CronJob	定时任务（scheduled job）	Namespace	逻辑分组，**不是一组机器**

github.com/bibi45c · 免费分享，禁止售卖

改造本地部署架构（PAST2 Q2(a) 问法）

旧组件	去向	标签
硬件负载均衡（Load Balancing）器	托管负载均衡（ELB）/ Ingress	changed
Web 服务器	**S3 + CloudFront**	changed
应用服务器	**容器**（Container）**化微服务**（microservices）**+ 自动扩缩**（auto-scaling）（EKS）	changed
单库加备库	**每服务一个托管库、多可用区**（Multi-AZ）	changed
固定的 Web、应用服务器、硬件负载均衡	—	removed
API Gateway、事件流（event streaming）、Redis、Lambda + SNS、Cognito、监控（monitoring）、CI/CD、数据接入（IoT Core / Kinesis）	新增	added

图上用 added / removed / changed 标签；只画改动部分，新增连线要画出。

❝ 可用句式 | ___ 处填案例内容 **T5 句子库**

The backend is hybrid. The core features (_____, _____) run as microservices on Amazon EKS and scale with an HPA; _____ is aggregated by event-stream consumers; _____ runs as a scheduled job; notifications are Lambda functions. Each workload uses the most suitable and cost-effective style.

> 后端用混合架构。核心功能（_____、_____）以微服务运行在 Amazon EKS 上，用 HPA 扩缩；_____ 由事件流消费者汇总；_____ 用定时任务；通知用 Lambda 函数。每种负载都用最合适、最省钱的风格。

Requests that need an answer at once use synchronous REST through the service mesh. High-volume updates use an event stream: each _____ publishes a _____ event, and _____ consumes it. This decouples the services and absorbs the peak.

> 需要立刻得到结果的请求走同步 REST，经服务网格。大量更新用事件流：每个 _____ 发布一个 _____ 事件，由 _____ 消费。这样服务解耦，也能吸收峰值。

The _____ service is stateful: it keeps _____. The state is stored outside the pods in ElastiCache (Redis) and saved to Amazon RDS, so any pod can be replaced or scaled without losing data.

> _____ 服务是有状态的：它保存 _____。状态放在 Pod 外的 ElastiCache（Redis）里，并保存到 Amazon RDS，所以任何 Pod 都能被替换或扩容而不丢数据。

Each microservice is deployed as a Deployment with at least three Pods across three availability zones, behind a ClusterIP Service. An Ingress routes / _____ to _____ -svc. An HPA scales _____ from 5 to 50 Pods on CPU; the Cluster Autoscaler adds nodes when Pods no longer fit.

> 每个微服务以 Deployment 部署，至少 3 个 Pod，分布在 3 个可用区，前面是 ClusterIP Service。Ingress 把 / _____ 路由到 _____ -svc。HPA 按 CPU 把 _____ 从 5 个扩到 50 个 Pod；Pod 放不下时 Cluster Autoscaler 加节点。

The report runs as a CronJob; configuration and passwords are kept in ConfigMaps and Secrets; Kafka, Redis and the databases use managed services.

> 报表用 CronJob 运行；配置和密码放在 ConfigMap 和 Secret 里；Kafka、Redis 和数据库用托管服务。

The hardware load balancer is replaced by Elastic Load Balancing (changed); the web servers by S3 and CloudFront (changed); the application servers by containerised microservices with auto-scaling (changed); the single database by managed Multi-AZ databases per service (changed).

> 硬件负载均衡器换成 Elastic Load Balancing（changed）；Web 服务器换成 S3 + CloudFront（changed）；应用服务器换成可自动扩缩的容器化微服务（changed）；单一数据库换成每个服务一个托管的多可用区数据库（changed）。

github.com/bibi45c · 免费分享，禁止售卖

❝ 可用句式｜___处填案例内容 **T5 小问补充句式：图怎么用文字说、有状态（stateful）服务怎么写**

(i) The diagram below shows the main components from left to right: clients (mobile app, web portal, _____ devices) → CloudFront and Amazon API Gateway → an Ingress in front of the Amazon EKS cluster → the microservices _____, _____ and _____ → their own databases.

(i) 下图从左到右画出主要组件：客户端（手机 App、网页门户、_____ 设备）→ CloudFront 和 Amazon API Gateway → EKS 集群前面的 Ingress → 微服务 _____、_____、_____ → 各自的数据库。

Device data takes a separate path: the _____ units send messages to AWS IoT Core, which writes them to Amazon Kinesis Data Streams; the _____ Manager and _____ Manager consume the stream.

设备数据走另一条路：_____ 设备把消息发到 AWS IoT Core，再写入 Amazon Kinesis Data Streams；_____ Manager 和 _____ Manager 消费这个事件流。

Supporting parts are also shown: a service registry (CoreDNS), Redis (ElastiCache) for the latest values, a CronJob for _____ (the batch use case), Lambda + SNS for notifications, and CloudWatch for monitoring.

图上还画了支撑组件：服务注册（CoreDNS）、存最新值的 Redis（ElastiCache）、做 _____（批处理用例）的 CronJob、发通知的 Lambda + SNS、负责监控的 CloudWatch。

(ii) Internal communication: calls that need an answer at once use synchronous REST through service discovery (e.g. _____ calls _____ to check _____). High-volume updates use asynchronous publish/subscribe: _____ publishes a _____ event to the event stream, and _____ consumes it. These are marked "sync" and "async" on the diagram.

(ii) 内部通信：需要立刻拿结果的调用走同步 REST，经服务发现（如 _____ 调 _____ 查 _____）。大量更新走异步发布 / 订阅：_____ 向事件流发布 _____ 事件，_____ 消费。图上分别标 sync 和 async。

(iii) Stateful service: the _____ Manager keeps the state of each _____ (e.g. pending → captured → refunded). The state is stored in Amazon RDS, outside the pods, so the pods stay stateless and can be replaced or scaled without losing data.

(iii) 有状态服务：_____ Manager 保存每笔 _____ 的状态（如 待处理 → 已扣款 → 已退款）。状态存在 Pod 外的 Amazon RDS，所以 Pod 保持无状态，替换或扩容都不丢数据。

A refund that spans several services is handled as a Saga: each step is a local transaction with a compensating action, so a failed step can be undone.

跨几个服务的退款用 Saga 处理：每一步是一个本地事务，并有对应的补偿操作，某一步失败可以撤回。

✕常见误区｜这样答会丢分 **T5 易错**

1. 只堆服务名，不画交互（interaction）、不标通信组件 → 丢"通信组件 2 分"。
2. K8s 计划里没写 **Pod、Deployment、Service** → 3 分直接丢；写"Pod"当部署对象（应写 Deployment）。
3. 所有服务都配 HPA；副本（replica）数不给依据。
4. 有状态服务不点名；用粘性会话（sticky session）。
5. 只写"NoSQL""缓存（cache）"，不写具体产品。
6. 混淆 Service（集群内稳定地址）和 Ingress（外部请求按路径分发）；混淆 HPA（加 Pod）和 Cluster Autoscaler（加节点（node））。
7. 把 Namespace 当成一组机器；把可用区（availability zone）当异地备份。

真题范文（model answer）：PAST Q2(a) p.**46** · PAST2 Q2(a) p.**51**。

github.com/bibi45c · 免费分享，禁止售卖

PART 1　题型卡 T1–T17（怎么认 · 怎么写）

T6　推荐云服务 · AWS 物理部署图

✎ 考试怎么考 | 题型·分值·套路　**怎么认 · 分值 · 三种问法**

题干（question stem）出现 **propose cloud storage / database services / physical deployment diagram / use some of the AWS services listed / fully managed ML service / alternatives。** ① 推荐两个云存储服务，各对上一个质量属性（quality attribute）（PAST2 Q1(e) 4 分：每个服务 2 分） ② 用题目给的 AWS 服务画**物理部署图**（physical deployment diagram） **+ 至少三个架构决策**（architecture decision）（PAST3 Q2(b) 9 分：图约 3–4 + 决策约 5–6） ③ 托管 ML 架构图 + 每个组件的替代品（PAST2 Q2(c) 3 分） **老师说过：必须写具体厂商的具体服务名，熟悉一家即可（本手册以 AWS 为主）。**

按需求选存储（对号入座）

需求特征	存储类型	AWS 产品	满足属性的理由（写进答案）
海量高频写入（遥测、位置、选票、运动轨迹）	键值 / 宽列（Wide-Column） / 时序	**DynamoDB**、Keyspaces、Timestream	按分区键（partition key）自动分片（sharding），写吞吐（write throughput）水平扩展（horizontal scaling）；毫秒级读写
毫秒级读、实时计数、会话、排行榜	内存缓存（In-Memory Cache）	**ElastiCache for Redis**	数据在内存，亚毫秒延迟（latency）；可加节点（node）、分片
强一致（strongly consistent）**事务**（transaction）**（订单、支付、库存、账户）**	关系库（relational database）	**RDS / Aurora（Multi-AZ）**	ACID 事务；多可用区（Multi-AZ）自动故障转移（failover）；读副本（read replica）扩读
结构多变的文档、商品目录	文档库	DocumentDB	schema 灵活，可按任意字段建索引
全文搜索、日志（logs）**分析**	搜索引擎（Search Engine）	OpenSearch Service	倒排索引、聚合（aggregate）查询
历史分析、报表	数据仓库（Data Warehouse） / 数据湖（data lake）	Redshift、S3 + Athena、EMR	列式存储，适合大批量分析
图片、报告、备份、路线图	对象存储（object storage）	**S3**	几乎无限容量、11 个 9 耐久、便宜
好友推荐、关系遍历	图数据库（Graph Database）	Neptune	擅长遍历关系
指标（metric）**（随时间变化的数字）**	时序库（time-series database）	Timestream / CloudWatch	按时间范围查询、聚合

一句话：读多加副本（replica），**写多要分片，热点数据加缓存**（cache）。 库存、余额、支付**别用**最终一致（eventually consistent）的存储；缓存不能当主存储（可能丢数据）。

逻辑节点 → AWS（画物理部署图用）

逻辑位置 / 节点	AWS
用户、静态前端（front end）	Route 53 + **CloudFront** + **S3**
DMZ（对外入口）	**API Gateway** + WAF：区域（zone）级托管服务（managed service），在 VPC 之外，起反向代理（reverse proxy） / DMZ 作用，经 VPC Link 连到私有子网（private subnet）（若用 ALB，ALB 放公有子网（public subnet））
Application / Platform 节点（node）	**私有子网**的 **EKS**（或 Elastic Beanstalk） + **Lambda**
Data Store 节点	**RDS**、**DynamoDB**、**ElastiCache（Redis）**、**S3**（私有子网）
Integration 节点	集成子网，经 **NAT 网关**（gateway）访问外部系统
认证（Authentication）	**Cognito**
分析	**EMR** / Redshift

画法： 一个 VPC，**两个可用区**（availability zone），公有子网（ALB、NAT 网关） / 私有子网（EKS、Lambda、数据库）；API Gateway、CloudFront、Cognito 画在 VPC 外；数据库**不放公有子网或 DMZ。**题目给的服务清单**不必全用**，但要用上一部分。

▶ 案例演练 | 演示怎么套用　**物理部署图**（physical deployment diagram）**文字骨架（PAST3 Q2(b)）**

```
Users → Route 53 → CloudFront → S3 (web front end)
      → API Gateway (+WAF, Cognito)            ← edge, outside the VPC (DMZ role) → VPC Link
VPC, 2 AZs, private subnets:
  EKS cluster (or Elastic Beanstalk): app control + shared platform services, auto-scaling
  Lambda: event-driven services (gamification, social sharing, health sync)
  RDS Multi-AZ (users, challenges) | DynamoDB (workout tracks) | ElastiCache Redis (leaderboards) | S3 (media)
  Integration subnet → NAT gateway → Google Fit / social media
EMR: historical analytics
```

github.com/bibi45c · 免费分享，禁止售卖

写"架构决策"的格式：选了什么 + 为什么 + 满足哪个需求

❝ 可用句式 | ___处填案例内容 **T6 句子库**

Decision 1 – API Gateway and Cognito at the edge: all requests go through Amazon API Gateway in front of the private subnets, and Amazon Cognito handles sign-up and sign-in. This follows the DMZ / reverse-proxy rule in Section _____ and scales to millions of users.

> 决策 1：边缘用 API Gateway 和 Cognito。所有请求先经过私有子网前面的 Amazon API Gateway，Amazon Cognito 负责注册登录。这符合第 _____ 节的 DMZ / 反向代理要求，并能扩展到数百万用户。

Decision 2 – Containers for always-on services, Lambda for event-driven ones: _____ run all the time, so they run as containers on Amazon EKS across two availability zones with auto-scaling. _____ is triggered when _____, so it is an AWS Lambda function that scales automatically and costs nothing when idle.

> 决策 2：常驻服务用容器，事件驱动的用 Lambda。_____ 一直在运行，所以以容器跑在 Amazon EKS 上，跨两个可用区自动扩缩。_____ 在 _____ 时触发，所以做成 AWS Lambda 函数，自动扩展、空闲不花钱。

Decision 3 – Polyglot persistence: high-volume _____ go to Amazon DynamoDB for fast writes at any scale; _____ go to Amazon RDS (Multi-AZ) for transactions; _____ are cached in ElastiCache (Redis); media goes to Amazon S3. Data is partitioned by _____ for growth and isolation.

> 决策 3：多语言持久化。大量的 _____ 存 Amazon DynamoDB，任何规模都能快速写入；_____ 存 Amazon RDS（多可用区）以支持事务；_____ 缓存在 ElastiCache（Redis）；媒体存 Amazon S3。数据按 _____ 分区，便于增长和隔离。

The two most important quality attributes for the data store are scalability (write throughput) and performance (low latency), because _____ devices send _____ updates per minute and users must see changes within _____ seconds.

> 存储最重要的两个质量属性是可扩展性（写吞吐）和性能（低延迟），因为 _____ 台设备每分钟发 _____ 次更新，用户必须在 _____ 秒内看到变化。

I recommend **Amazon DynamoDB**, a fully managed key-value database. It partitions data by _____ ID, so write throughput scales out (scalability), and each read or write takes milliseconds (performance).

> 我推荐 Amazon DynamoDB，全托管的键值数据库。它按 _____ ID 分区，写吞吐能水平扩展（可扩展性），每次读写都是毫秒级（性能）。

I also recommend **Amazon ElastiCache for Redis**. The latest _____ are kept in memory, so _____ users get answers well within _____ seconds (performance), and nodes can be added as users grow (scalability).

> 我还推荐 Amazon ElastiCache for Redis。最新的 _____ 放在内存里，_____ 个用户的查询远低于 _____ 秒（性能）；用户增长时可以加节点（可扩展性）。

托管 ML 架构（PAST2 Q2(c) 问法）

数据流：**接入 → 预处理 → 存储（历史 + 额外变量）→ 训练 → 在线推理 → 写结果 → 对外提供**；每个框写产品名，旁边写替代品。

步	AWS	替代品
接入	Kinesis Data Streams	Amazon MSK、Azure Event Hubs、Google Pub/Sub
预处理	Lambda	容器（Container）（EKS）
历史存储	S3（数据湖）	Azure Data Lake Storage、Google Cloud Storage（分析用 BigQuery）
训练 + 推理	**SageMaker**（训练 + 实时端点（Endpoint））	Azure Machine Learning、Google Vertex AI
结果	DynamoDB → API Gateway	Cosmos DB / Firestore

三厂商对照（最常用的一行）

组件	AWS	Azure	Google Cloud
托管 K8s	EKS	AKS	GKE
函数（function）	Lambda	Functions	Cloud Run functions
API 网关	API Gateway	API Management	Apigee / API Gateway
对象存储（object storage）	S3	Blob Storage	Cloud Storage
关系库（relational database）	RDS / Aurora	Azure SQL / PostgreSQL	Cloud SQL / Spanner
键值 / 文档	DynamoDB	Cosmos DB	Firestore / Bigtable
缓存	ElastiCache	Azure Managed Redis	Memorystore
事件流（event streaming）	Kinesis / MSK	Event Hubs	Pub/Sub
消息队列（message queue）/ 通知	SQS / SNS	Service Bus / Notification Hubs	Pub/Sub / FCM
机器学习（Amazon SageMaker）	SageMaker	Azure ML	Vertex AI
身份	Cognito	Entra External ID	Identity Platform
监控（monitoring）	CloudWatch	Azure Monitor	Cloud Monitoring

完整对照表见学习笔记 11（PART 4）。

github.com/bibi45c · 免费分享，禁止售卖

× 常见误区｜这样答会丢分 **T6 易错**

1. 只写"用 NoSQL / 缓存"，不写产品名。
2. 只写产品名，不写**满足哪个属性、靠什么机制。**
3. 物理图漏了**两个可用区**（availability zone）**、公有 / 私有子网**（private subnet）；数据库放进公有子网（public subnet）或 DMZ。
4. 只画图不写决策，或决策只写"用了 Lambda"不写为什么。
5. 库存、支付用最终一致（eventually consistent）的存储。

真题范文（model answer）：PAST2 Q1(d)(e) p.**50** · PAST2 Q2(c) p.**51** · PAST3 Q2(b) p.**57**。

github.com/bibi45c · 免费分享，禁止售卖

PART 1　题型卡 T1–T17（怎么认 • 怎么写）

T7　质量属性　•　T9　平台思维

✎ 考试怎么考 | 题型•分值•套路　**T7 怎么认 • 分值**

题干（question stem）出现 **the most important / crucial non-functional requirement(s)、quality attributes、Do you agree that usability...?、as part of the architecture guiding principles**。PAST Q1(d)、PAST2 Q1(d)，**2 分 = 每个有说服力的理由**（reason）**1 分**。

T7 套路：选属性 → 引数字 → 说后果 → 说为什么比别的更关键

案例里看到	选的属性	理由要引用
10 万并发（concurrency）、**90% 请求 3 秒内、投票时间窗固定**	**性能 / 可扩展性** Performance / Scalability	并发数（Concurrency）+ 百分位（percentile）+ 时限
每人只能投一票、个人信息、支付	**安全（完整性、保密性**（confidentiality）**）** Security (integrity)	业务规则（business rule）+ 后果"结果不可信"
1 万台设备持续上报、数据不断增长	**可扩展性（写吞吐**（write throughput）**）**	设备数、每分钟更新数
5 秒内展示、2 秒内回传	**性能（低延迟**（latency）**）**	时限
不能停、选举当天必须可用	**可用性** Availability	业务时间窗
多租户（multitenancy）、**按机构隔离**（isolation）	**安全（隔离）+ 可扩展性（分片**（sharding）**）**	§3 多租户原文

题目问"最重要的两个"就只写两个，不要列五六个。官方 PAST 立场：**不同意易用性**（usability）**最重要**（老师说其他立场论证（justification）合理也给分，但和官方冲突风险（risk）大）。易用性和安全是此消彼长（trade-off）。

❝ 可用句式 | ___ 处填案例内容　**T7 句子库**

I disagree. Usability is important, but it is not the most important non-functional requirement. The two most critical ones are performance/scalability and security.

> 我不同意。易用性重要，但不是最重要的非功能需求。最关键的两个是性能/可扩展性和安全。

Performance/scalability: Section 4 requires the system to support 100,000 concurrent voters, with 90% of requests finished within 3 seconds, and the voting period is fixed. If the system is slow or crashes at peak time, voters cannot vote and the election fails.

> 性能/可扩展性：第 4 节要求系统支持 10 万并发选民，90% 的请求 3 秒内完成，而且投票时间是固定的。高峰时系统变慢或崩溃，选民就投不了票，选举就失败了。

Security: The system must make sure each voter can vote only once. If someone can vote twice, the result is not trustworthy and the system fails its main purpose.

> 安全：系统必须保证每位选民只能投一次。如果有人能投两次，结果就不可信，系统就失去了主要目的。

Poor usability only makes the task less convenient, but problems in performance or security make the system fail. Also, security and usability have an inverse relationship, so here security has higher priority.

> 易用性差只是让操作不方便，而性能或安全出问题会让系统失败。另外安全和易用性此消彼长，所以这里安全优先级更高。

The two most important quality attributes for the data store are scalability of writes and low latency, because _____ devices keep sending updates and users must see changes within _____ seconds. They matter more than rich queries, because the data is simple and read by key.

> 存储最重要的两个质量属性是写入的可扩展性和低延迟，因为 _____ 台设备一直在发更新，用户必须在 _____ 秒内看到变化。它们比复杂查询更重要，因为数据简单、按键读取。

❝ 可用句式 | ___ 处填案例内容　**T7 补充：质量属性**（quality attribute）**别叫错名字**

Integrity: the _____ values must be accurate and complete, because _____ (e.g. billing and settlement) is calculated from them.

> 完整性（Integrity）：_____ 数据必须准确、完整，因为 _____（如计费和结算）是根据它们算出来的。

Availability: Multi-AZ deployment with automatic failover keeps the _____ store running when one data centre fails.

> 可用性（Availability）：多可用区部署加自动故障转移，让 _____ 存储在一个机房出问题时继续运行。

Security (confidentiality): data is encrypted at rest with KMS and in transit with TLS, and access is controlled with IAM roles.

> 安全（保密性）：静态数据用 KMS 加密、传输中用 TLS 加密，访问用 IAM 角色控制。

Scalability (write throughput): _____ units send _____ messages every _____ seconds, about _____ writes per second at peak, so the store must scale out writes.

> 可扩展性（写吞吐）：_____ 台设备每 _____ 秒发 _____ 条消息，高峰时约每秒 _____ 次写入，所以存储必须能水平扩展写入。

github.com/bibi45c · 免费分享，禁止售卖

✕ 常见误区｜这样答会丢分 **常见中式英语（改法）**

原写法	问题	改为
everyone only have a ticket	ticket 是门票	each voter can cast only one **vote**
the vote has his own time window	vote 是一张票	the **voting period** is fixed
100K people to use for one moment	不准确	100,000 **concurrent** voters
the vote result will be not reliable	语序错	the result will **not be trustworthy**
性能是要求并发不出错	"不出错"是可靠性（reliability）	meet the response time **under** 100,000 concurrent users

质量属性场景（Quality Attribute Scenario）（六部分，最后一项必须有数字）

部分	例（Vote Anywhere）
Source 刺激源	100,000 voters
Stimulus 刺激	cast votes within the 5-minute voting window
Artifact 制品（系统的哪部分）	Voting service and its database
Environment 环境	peak time on election day, normal operation
Response 响应	accept and confirm every vote, lose none
Response Measure 度量	90% of requests within 3 s, maximum < 5 s, error rate < 0.1%

T9 平台思维

✎ 考试怎么考｜题型·分值·套路 **T9 怎么认 • 分值**

题干（question stem）出现 **seed / interaction / producer / consumer / the most crucial architectural aspect(s) for the success of the platform / expose as API to**。PAST2 Q1(a)、PAST3 Q1(a) 各 2 分；PAST3 Q1(c)(i) "开放哪两个 API、给谁" 约 2 分。

概念	意思	YouTube	Vote Anywhere
Seed 种子	用户来平台想要的核心价值	视频	投票（Poll）及结果
Producer 生产者	提供种子的角色	上传者	Election Admin、Voter
Consumer 消费者	使用种子的角色	观众	Public User、Observer
Interaction 交互	围绕种子的动作（**两侧都写**）	上传、观看、评论	Conduct Vote、View Results
Magnet / Toolbox / Matchmaker	吸引双方 / 让交互发生的工具 / 用数据配对	内容 / 播放器、API / 推荐算法	

交互对应的工程要素（六个）： 服务设计、架构、对外暴露服务（API）、业务流程、**可扩展性**（scalability）、**性能**（performance）。T9 只要写**一两个最关键的**（crucial / critical），每个绑一个 §4 数字。种子要选**一个具体的**（菜品、行程订单），不要三个并列。

套路： 点名种子、生产者、消费者、交互 → 选一个工程方面（可扩展性 / 实时性能 / 多租户（multitenancy） + 共享 API / 数据分析）→ 用 §4 数字说明为什么关键 → 说做不到会怎样（用户流失）。

❝ 可用句式｜__ 处填案例内容 **T9 句子库**

The seed of the platform is _____, produced by _____ and consumed by _____ through _____ and _____.
平台的种子是 _____，由 _____ 产生，_____ 通过 _____ 和 _____ 来使用。

The most crucial architectural aspect is the scalability of the _____ services: _____ users view _____ at the same time, so the services must be stateless and scale out automatically, keeping 90% of requests within _____ seconds.
最关键的架构方面是 _____ 服务的可扩展性：_____ 个用户同时查看 _____，所以服务必须无状态、能自动扩容，保持 90% 的请求在 _____ 秒内响应。

The second aspect is the performance of the real-time update pipeline: _____ updates per minute must reach users within _____ seconds. An event stream partitioned by _____ lets the services process updates in parallel and in order. Old data would drive users away.
第二个方面是实时更新管道的性能：每分钟 _____ 次更新必须在 _____ 秒内到达用户。按 _____ 分区的事件流让服务并行且按顺序处理更新。数据过时会把用户赶走。

A crucial aspect is multitenancy with shared, exposed services: many _____ must join as tenants with their data isolated, while sharing common services such as _____ through APIs. New producers can then join easily, and the ecosystem grows.
关键方面是多租户加共享、开放的服务：许多 _____ 要作为租户加入、数据隔离，同时通过 API 共享 _____ 等通用服务。新的生产者就能轻松加入，生态得以扩大。

I would expose the _____ Service API to _____ (producers), because today _____ is done by hand; with the API their own systems can _____ directly, which brings more seeds onto the platform.
我会把 _____ 服务的 API 开放给 _____（生产者），因为现在 _____ 是手工完成的；有了 API，他们自己的系统就能直接 _____，把更多种子带进平台。

✎ 考试怎么考｜题型·分值·套路 **T9 三句话拿满 2 分（只写一个方面）**

种子 + 双方 + 交互写一句 → 写**一个**方面并带上 §3 / §4 的数字 → 加一句"做不到会怎样"。题目问 one 就只写一个，**不用写第二个方面**；也不要套别的卷子的句子（模拟卷 2 Q1(a) 的常见失分点）。

github.com/bibi45c · 免费分享，禁止售卖

❝ 可用句式 | ___ 处填案例内容 **T9 补充：开售洪峰型平台（抢票、限量发售）**

The virtual queue absorbs the peak and admits users at a rate the _____ service can handle, and each _____ must be _____ by only one user, so the system stays fair. If the _____ fails or _____ twice, users and _____ lose trust and leave the platform.

> 虚拟队列吸收洪峰，按 _____ 服务能承受的速度放人；每个 _____ 只能被一个用户 _____，保证公平。如果 _____ 崩溃或 _____ 两次，用户和 _____ 会失去信任、离开平台。

"一次只建一种交互"为什么： 先用小范围验证价值（MVP）；控制风险（risk）和成本；降低用户学习成本；更容易找到变现点（LinkedIn 最后才变现）。

真题范文（model answer）：PAST Q1(d) p.**45** · PAST2 Q1(a)(d) p.**48** · PAST3 Q1(a)(c)(i) p.**54**。

github.com/bibi45c · 免费分享，禁止售卖

PART 1 题型卡 T1–T17（怎么认・怎么写）

T8 指标设计（评价型 / 设计型）

✎ 考试怎么考｜题型·分值·套路 **怎么认・分值・两种问法**

题干（question stem）出现 **metric / application metric / measure and track / captured directly / calculated from / frequency / retention / location。** ① **评价型**（PAST Q1(e) 4 分）：**Do you agree that _____ can be used to evaluate scalability and usability?** → 表态 + 指出（identify）它只是量 + **两个更好的指标**（metric），**每个带理由**（reason）**（各 2 分）** ② **设计型**（PAST2 Q3(b) 6 分）：**Propose a metric … If captured directly … If calculated …** → 定义与计算约 2 + 组件 / 频率 / 保存 / 位置约 2 + **两个依赖指标**（dependent metric）**同样说明约 2**

第 1 步：指标要测"结果"，不是"用了多少"

想评价	选什么类型的指标	例
可扩展性（scalability）	高峰期的**处理时间、排队时间**（Queueing Time）、**成功率**	每位选民平均排队时间；高峰期下单成功率
性能（performance）	**响应时间**（response time）（看 P90 / P99，不只看平均）	请求响应时间 P90
易用性（usability）	**完成任务的时间、放弃率、成功率**	选好餐厅到下单成功的耗时
可用性（availability）	**Yield = 成功请求 / 总请求**、Uptime	支付成功率
准确度	预测值与实际值的**误差**	MAE

坏指标（metric）**一眼认出：** 每天 / 每月的总数、访问量、处理人数 → 只是使用量，受注册人数影响，人数高甚至可能是排长队的结果。

第 2 步：直接采集，还是计算得出?

指标是	类型	写法
一个事件的**计数**，或已有的测量值（CPU）	**直接采集**（capture）	谁产生、谁上报
两个时间戳之差的平均、比率、两个值的比较	**计算得出**	公式 + **依赖的两个基础指标**（metric）（各写：谁在什么时刻记录、保存多久、存哪）

依赖指标（dependent metric）**= 公式里用到的两个原始数据。** "平均接单等待时间"依赖 ①下单时间戳 ②接单时间戳。

第 3 步：七个字段逐项填（不会漏分）

#	字段	✗ 不行	✓ 可以（默认值）
①	名称与定义	每日订单数	选好餐厅到下单成功的 **P90 耗时（秒）**，按城市和小时
②	理由（reason）（点名质量属性（quality attribute））	空着 / 抄题	反映**易用性**（usability）：耗时越短、放弃越少，流程越顺
③	直接 / 计算	不写	计算型：依赖"开始下单"和"下单成功"两个时间戳
④	负责组件	**日志**（logs）**组件**、"系统"	**案例领域模型**（domain model）**里的服务名**发事件（OrderManager）+ **流处理任务**（Stream-Processing Job）**（Kinesis + Lambda）**或定时任务（scheduled job）计算
⑤	频率	**按需**	**每个事件实时采集**（capture），**每 1 分钟汇总**（aggregation）；分析型"每天一次"
⑥	保存时长（retention period）	**按需**、"长期"	原始事件 **30–90 天**；分钟汇总 **90 天**；日汇总 **3 年**（§5 有 GDPR 等保留要求要遵守）
⑦	存储位置	RDS、ElastiCache、"数据库"	**时序库**（time-series database）**Timestream / CloudWatch**；原始数据 **S3**；分析 **Redshift**

可观测性（Observability）**三条流（图 6-2）：** 指标 Metrics → **Prometheus** 拉取 → **时序库 TSDB** → Grafana；日志 Logs → **Fluentd / Logstash** → **Elasticsearch** → Kibana；追踪（tracing）Traces → **OpenTelemetry** → **Jaeger / Zipkin**。指标回答"有多慢、有多少"，日志回答"发生了什么"，追踪回答"慢在哪个服务"。**日志组件只负责记录，不是指标采集组件。**

❝ 可用句式｜___处填案例内容 **评价型（PAST Q1(e)）**

I disagree. The number of _____ per day only measures usage. It depends on how many _____ are registered, not on how well the system works. It does not show how the system handles the peak load (scalability) or how smoothly users finish the main task (usability). A high number may even mean that users waited a long time.

> 我不同意。每天的 _____ 数只衡量使用量，取决于注册了多少 _____，而不是系统表现得好不好。它既不能说明系统怎样应对高峰负载（可扩展性），也不能说明用户完成主要任务是否顺畅（易用性）。数字高甚至可能说明用户等了很久。

Better metric 1 – average identity-verification time: the time from the voter being admitted into the polling room to the proctor finishing the verification (queueing time not included). Shorter verification means the platform serves more voters at peak (scalability) and supports proctors well (usability).

> 更好的指标 1：平均身份核验时长，从选民被请进投票室到监考员完成核验的时间（不含排队）。核验越快，平台高峰时能服务越多选民（可扩展性），也说明对监考员支持得好（易用性）。

Better metric 2 – average queueing time per voter: it shows the voter's experience and whether the platform keeps up during the voting window.

> 更好的指标 2：每位选民的平均排队时间，反映选民体验，以及平台在投票时间窗内是否跟得上。

github.com/bibi45c · 免费分享，禁止售卖

❝ 可用句式 | ___ 处填案例内容 **设计型（PAST2 Q3(b) / 自练）**

Metric: _____ = _____ (in seconds), per _____ and per hour. It reflects _____, because _____.
| 指标：_____ = _____（秒），按 _____ 和小时统计。它反映 _____，因为 _____。

It is a calculated metric that depends on two base metrics: _____ and _____.
| 这是计算型指标，依赖两个基础指标：_____ 和 _____。

Calculation: a stream-processing job (Kinesis + Lambda) joins the two by _____ ID and calculates the value every 1 minute. Per-minute values are kept for 90 days and daily values for 3 years in Amazon Timestream.
| 计算：流处理任务（Kinesis + Lambda）按 _____ ID 把两者关联，每 1 分钟计算一次。分钟值在 Amazon Timestream 保存 90 天，日值保存 3 年。

Dependent metric 1 – _____ timestamp: emitted by the _____ Manager when _____; kept for 90 days in Amazon S3.
| 依赖指标 1：_____ 时间戳。由 _____ Manager 在 _____ 时发出；在 Amazon S3 保存 90 天。

Dependent metric 2 – _____ timestamp: emitted by the _____ when _____; kept for 90 days in the same S3 data lake, so both can be joined by _____ ID.
| 依赖指标 2：_____ 时间戳。由 _____ 在 _____ 时发出；在同一个 S3 数据湖保存 90 天，两者按 _____ ID 关联。

Metric – MAE of predicted arrival times: MAE = average of |predicted − actual arrival time| over all predictions for a bus service in a period. A falling MAE shows that the prediction is improving.
| 指标：预测到站时间的 MAE。MAE = 一段时间内某线路所有预测的 |预测 − 实际到站时间| 的平均值。MAE 下降说明预测在变准。

❝ 可用句式 | ___ 处填案例内容 **T8 补充：指标要测题目点名的那个对象**

Better metric 1 – connector availability ratio = time a connector is not Faulted ÷ total time, per station per day. It reflects the **availability of the stations themselves**, not of the payment system.
| 更好的指标 1：充电口可用时间比例 = 非故障时间 ÷ 总时间，按站、按天统计。它反映的是充电站本身的可用性，而不是支付系统的可用性。

(Or) **Start success rate** = successful session starts ÷ start attempts. A failed start means the driver could not charge, whatever the reason.
| （或）启动成功率 = 成功启动的会话数 ÷ 尝试启动的次数。启动失败就意味着司机没充上电，不管原因是什么。

Better metric 2 – P90 time from scanning the QR code to charging started, and the share of drivers who give up. It reflects the **experience** of the drivers.
| 更好的指标 2：从扫码到开始充电的 P90 耗时，以及中途放弃的司机比例。它反映司机的体验。

One metric measures one thing: do not mix a success rate and a duration in the same metric.
| 一个指标只测一件事：不要把成功率和耗时混在同一个指标里。

自练标准答案：打车"平均接单等待时间"

Definition: average of (driver acceptance time − order creation time) per city and per 5-minute window. **Reason:** reflects performance and scalability at peak hours. **Dependent metric 1 – order creation timestamp:** emitted by the OrderManager when the order is created, kept 90 days in S3. **Dependent metric 2 – acceptance timestamp:** emitted by the DispatchManager when a driver accepts, kept 90 days in S3. **Calculation:** a stream-processing job (Kinesis + Lambda) joins both by order ID every minute. **Retention and location:** per-minute averages kept 90 days and daily averages 3 years in Amazon Timestream.

✕ 常见误区 | 这样答会丢分 **T8 易错**

1. 把总数当可扩展性（scalability）/ 易用性指标。
2. 只给名字，不写**理由**（反映哪个质量属性）。
3. 频率写"按需"，保存时长不写数字。
4. 计算型漏了**两个依赖指标**（dependent metric），或依赖指标没写全（采集、保存、位置）。
5. 把日志组件当采集组件；存储写 RDS / ElastiCache（指标首选时序库）。
6. 只看平均值：要看百分位（percentile）（P90 / P99）。

真题范文（model answer）：PAST Q1(e) p.**45** · PAST2 Q3(b) p.**52**。

github.com/bibi45c · 免费分享，禁止售卖

PART 1　题型卡 T1–T17（怎么认 • 怎么写）

T10　二选一决策　•　T12　服务共置

✎ 考试怎么考｜题型•分值•套路　**T10 怎么认 • 分值**

题干（question stem）给两个选项让你选：**A or B? / Would you choose … instead of …? / argue for …。**PAST Q1(a)(ii) 2 分、Q2(b) 3 分，PAST2 Q2(b) 3 分。**每个有说服力的理由**（reason）**1 分**；3 分题常把第 3 分给在"对后端（backend）的影响"或"扩展决策"上。 **套路：先点出案例的要求（§4 数字、用例原话）→ 选 → 两个理由，每个绑一个案例事实 → 题目还问影响就写影响。**

常考的四组二选一

二选一	判断问句	选 A 的信号	选 B 的信号
消息队列（message queue） **vs 事件流**（event streaming）	一件事一人处理一次？还是要有序、多方读、可重放？	任务分发、削峰（peak shaving）、处理完就不留（发邮件）→ **队列（SQS）**	结果要 **deterministic and repeatable**、两个结果都要读同一批事件、要回溯 → **事件流（Kafka / Kinesis）**
原生 App vs PWA	要不要深度访问硬件？	IoT 传感器、执行器、蓝牙、GPS 深度使用 → **原生**	简单应用、要快、一套代码多平台 → **PWA**
虚拟机（Virtual Machine (VM)） **vs 容器**（Container）	负载变化快吗？要独立扩缩吗？	遗留应用、需要不同 OS、强隔离（isolation） → **VM**	秒级启动跟上突发、每个服务独立扩、密度高成本低 → **容器（EKS）**
托管服务（managed service） **vs 自建**（self-hosted）	流量稳不稳？有没有运维团队？	突发 / 不可预测、小团队 → **托管**（按用量付费、厂商运维）	高而稳定 7×24、强监管、要完全控制 → **自建**
微服务（microservices） **vs Serverless**	这件事是持续的还是偶发的？	持续使用、低延迟（latency） → **微服务**	偶发、短小、事件触发 → **Serverless**（有冷启动（cold start））
编排（orchestration） **vs 协同**（choreography）	流程要不要显式集中？	流程显式、一目了然 → **编排**（中心服务耦合（coupling）高）	松耦合、加功能只加订阅（subscribe）者 → **协同**（要追踪（tracing）才看得清）
同步（synchronous） **vs 异步**（asynchronous）	调用方要不要立刻拿结果？	要立刻拿结果的查询 → **同步**	解耦、削峰、多消费者（consumer） → **异步**

对比	消息队列	事件流
消息去向	投递后**删除**	写入持久化日志（logs），保留期内**不删**
顺序	标准队列多消费者竞争，不保证顺序（FIFO 队列可按组有序，但吞吐（throughput）受限，读完仍删除、不能重放）	**同一分区**（partition）**内严格有序**
重放 / 多个消费者	不能 / 一条只给一个	**能 / 多个消费者组**（Consumer Group）**各读一遍**
产品	SQS、RabbitMQ	Kafka（MSK）、Kinesis、Event Hubs

❝ 可用句式｜___ 处填案例内容　**T10 句子库**

Reason 1 – Ordering: to give deterministic and repeatable results, the _____ events must be processed in the order they happened. An event stream keeps events in order within a partition (e.g. partitioned by _____), while competing consumers on a queue may process them out of order.

> 理由一（顺序）：要得到确定、可重复的结果，_____ 事件必须按发生顺序处理。事件流在分区内保持顺序（如按 _____ 分区），而队列上互相竞争的消费者可能乱序处理。

Reason 2 – Retention: both _____ and _____ must read the same events. An event stream keeps them for a set period, so two consumer groups can read them independently and replay them, but a message queue deletes a message once it is delivered.

> 理由二（保留）：_____ 和 _____ 都要读同一批事件。事件流把它们保留一段时间，两个消费者组可以各自读取、还能重放；而消息队列在消息投递后就删除。

Both a native app and a PWA can keep client-side state (local storage, session storage), so storage alone does not decide. The future _____ need deep access to device hardware such as IoT sensors, so I choose a **native app**. With client-side orchestration, the backend services do not need to change.

> 原生应用和 PWA 都能保存客户端状态（local storage、session storage），所以光看存储决定不了。未来的 _____ 需要深度访问设备硬件（如 IoT 传感器），所以我选原生应用。采用客户端编排，后端服务不需要修改。

I choose **containers** on Kubernetes (Amazon EKS). (1) They start in seconds, so they follow the peaks of _____ users, while VMs take minutes. (2) Each service scales on its own driver: view services on requests, _____ on the update rate or queue length. (3) Containers are lighter, so the cost is lower.

> 我选跑在 Kubernetes（Amazon EKS）上的容器。（1）容器秒级启动，能跟上 _____ 个用户的高峰，虚拟机要几分钟。（2）每个服务按自己的指标扩展：查看服务按请求数，_____ 按更新速率或队列长度。（3）容器更轻，成本更低。

I choose a managed service, because the traffic is bursty and the team has no operations staff: we pay per use and the vendor handles upgrades, scaling and the SLA.

> 我选托管服务，因为流量是突发的，团队没有运维人员：按用量付费，升级、扩缩和 SLA 由厂商负责。

github.com/bibi45c · 免费分享，禁止售卖

❝ 可用句式 | ___ 处填案例内容 **T10 补充：事件流怎么分区（PAST2 型）· 什么时候用队列 · 同步的代价**（trade-off）

Each onboard unit publishes its position to an event stream (Amazon Kinesis) partitioned by busTripId. ArrivalTimeComputer instances consume the partitions in parallel and write arrival times to DynamoDB, which the passenger app reads through the API.

> 每台车载单元把位置发布到按 busTripId 分区的事件流（Amazon Kinesis）。ArrivalTimeComputer 的多个实例并行消费各分区，把到站时间写入 DynamoDB，乘客 App 通过 API 读取。

DesirableSpeedComputer reads the same events as a second consumer group and returns the desirable speed to the bus within 2 seconds. A message queue cannot do this, because a message is deleted once one consumer has taken it.

> DesirableSpeedComputer 作为第二个消费者组读同一批事件，2 秒内把理想车速回传给车辆。消息队列做不到，因为一条消息被一个消费者取走就删除了。

For _____ (e.g. sending confirmation e-mails), each task only needs to be done once by one worker and order does not matter, so a message queue (Amazon SQS) is enough; it absorbs peaks and the workers scale with the queue length.

> 对于 _____（如发确认邮件），每个任务只需要一个 worker 做一次，顺序也不重要，所以消息队列（Amazon SQS）就够了；它能削峰，worker 按队列长度扩缩。

Synchronous calls couple the caller to the callee: if _____ is slow, _____ is slow too. Moving _____ to an asynchronous event removes it from the user's request path.

> 同步调用把调用方和被调方绑在一起：_____ 慢，_____ 也跟着慢。把 _____ 改成异步事件，就把它移出了用户请求的路径。

T12 服务共置：哪些服务放进同一个容器

✎ 考试怎么考 | 题型·分值·套路 **T12 怎么认 · 分值**

题干（question stem）出现 **collocate / the same container instance / licence constraint / communication affinity**。PAST2 Q1(f)、PAST3 Q1(d)，各 **2 分**。

原则：通信亲和度（communication affinity）（一段时间内两个服务交换的**消息数量和大小**）越高，越该放一起。回到 §7 领域模型（domain model）或用例，找**跨框连线最多、调用最频繁**的一对；最好**扩展规律和发布节奏也相近**。

共置的好处	代价
网络调用变**进程内调用**（in-process call），延迟（latency）大降	失去独立扩展（scale independently）、独立部署（deployment）
少了网络流量和序列化开销	一个故障可能拖垮另一个
少一个网络故障点	

样卷	推荐共置	理由
PAST2 Q1(f)	**ArrivalTimeComputer + DesirableSpeedComputer**	每次位置更新 → 算到站时间 → 算理想车速（2 秒内回传），每分钟 1 万次都要交换；负载模式相同
PAST3 Q1(d)	**Workout Tracking + Gamification**	每次运动都把详细数据发给游戏化更新仪表盘（dashboard）、发徽章，消息最多最大。**支付不和订单放一起**（PCI-DSS 要求支付只在尽量少的子系统（subsystem）里）
PAST（举例）	PollManager + ObservationManager	查看结果每次都要两者协作

❝ 可用句式 | ___ 处填案例内容 **T12 句子库**

I propose to collocate _____ and _____. Every _____ triggers a call from _____ to _____, so these two exchange the largest number and size of messages (highest communication affinity).

> 我建议把 _____ 和 _____ 放进同一个容器。每次 _____ 都会触发 _____ 调用 _____，所以这两个服务交换的消息数量和大小最大（通信亲和度最高）。

Collocating them turns these network calls into in-process calls, which cuts latency and network traffic; they also have the same load pattern, so they can be scaled together.

> 共置后这些网络调用变成进程内调用，降低延迟和网络流量；它们的负载模式也相同，可以一起扩缩容。

I would not collocate _____ with _____, because _____ (e.g. PCI-DSS requires payment handling to stay in as few subsystems as possible).

> 我不会把 _____ 和 _____ 放在一起，因为 _____（如 PCI-DSS 要求支付处理只在尽量少的子系统里）。

github.com/bibi45c · 免费分享，禁止售卖

“ 可用句式 | ___ 处填案例内容 **T12 补充：从领域模型和用例找"哪一对聊得最多"**

In Figure _____, _____ in the _____ Manager is derived from _____ in the _____ Manager. This association crosses the service boundary.

| 在图 _____ 中，_____ Manager 里的 _____ 由 _____ Manager 里的 _____ 推导而来。这条关联跨越了服务边界。

Every _____ (e.g. a status change from any of the _____ charger units) triggers a call between these two services, so they exchange the largest number of messages.

| 每一次 _____ （如 _____ 台充电桩中任何一台的状态变化）都会触发这两个服务之间的调用，所以它们交换的消息最多。

By contrast, _____ runs only once a month as a batch job, so _____ and _____ exchange few messages and are not the best pair.

| 相比之下，_____ 每月只作为批处理运行一次，所以 _____ 和 _____ 交换的消息很少，不是最合适的一对。

I would not collocate the PaymentManager with other services, because PCI-DSS requires payment data to be handled by as few subsystems as possible.

| 我不会把 PaymentManager 和其他服务放在一起，因为 PCI-DSS 要求支付数据只由尽量少的子系统处理。

真题范文（model answer）： PAST Q1(a)(ii) p.**43** · PAST Q2(b) p.**46** · PAST2 Q1(f) p.**50** · PAST2 Q2(b) p.**51** · PAST3 Q1(d) p.**56**。

github.com/bibi45c · 免费分享，禁止售卖

PART 1 题型卡 T1–T17（怎么认 • 怎么写）

T11 性能 • 负载测试 • 分片

✎ 考试怎么考 | 题型•分值•套路 **怎么认 • 分值**

题干（question stem）出现 **performance test / load test results vary / not repeatable / Postman / throughput / bottleneck / write throughput / read replica / shard / scale writes / side effects to monitor**。PAST Q1(a)(i) 2 分（原因（cause） 1 + 办法 1）、Q3(d) **6 分（分片**（sharding）**方式 2 + 受影响组件及理由**（reason）**4）**。 老师说排队（queueing）论数学**不考计算**，公式只用来写论证（justification）。

先判断在问哪一类

症状	原因的机制	措施
几个测试员结果波动大、不可信	没有统一的**工作负载模型**（workload model）；Postman 是单机功能测试工具	把工作负载模型写进测试标准；**JMeter** 从**多台施压机**（load generator）执行；按 **P90** 判定
整体吞吐量（throughput）**上不去**	串行（Serial）链路吞吐（throughput） = **min(各组件)**，被最慢的瓶颈（bottleneck）限制	每个组件都要 ≥ 目标；低于目标的**都要改**：并行（Parallel）多实例（吞吐相加）、分片（sharding）（写）、读副本（read replica）或缓存（cache）（读）
高峰期响应变慢	利用率（utilisation）过高 → **排队时间**（Queueing Time）**急剧增长**（R = S / (1 − U)）	阈值 **60–70%** 提前扩容（scale out）；缓存、预计算（precomputation）；限流（rate limiting）、排优先级
数据库写入跟不上	单主库（Primary），所有写都进一个库；**读副本只扩读**	**分片**（哈希、点名分片键（shard key））；前面加事件流（event streaming）缓冲写入峰值（peak）、批量写
读很多、响应慢	重复读相同数据	缓存（看命中率（hit ratio））、读副本、预计算
自动扩缩（auto-scaling）**来回抖动**	**乒乓效应**（Ping-Pong Effect）	扩容激进、缩容（scale in）保守，设冷却时间（Cooldown Period）

负载测试：工作负载模型（PAST Q1(a)(i)）

要素	写什么（要写数字）
负载曲线	恒定 → 负载测试（load test）；阶梯递增 → 压力测试（Stress Testing）；**突增 → 突发测试**（Spike Testing）（抢票）；长时间稳定 → 耐久测试（Soak / Endurance Testing）
虚拟用户与到达率（arrival rate）	到达率 = 总人数 ÷ 时间：100 万 ÷ 300 秒 ≈ 每秒 3,300
场景组合（Workload Mix）	如 70% 浏览、20% 下单、10% 支付
思考时间（Think Time）、持续时间、测试数据	模拟真实用户
通过标准（pass criteria）	**P90 < X 秒**（X 用案例数字）、最大 < Y 秒、错误率（Error Rate） < 0.1%

❝ 可用句式 | ___ 处填案例内容 **负载测试句子**

Cause: the testers did not use a common workload model. The criteria give the number of users, the 30-second window and the maximum response time, but not how the load is applied over time, the request mix or the think time. Each tester made different assumptions, so the results differ.

> 原因：测试员没有用统一的工作负载模型。测试标准给了用户数、30 秒时间窗和最大响应时间，却没说负载如何随时间施加、请求组合和思考时间。每个测试员假设不同，所以结果不同。

More reliable way: write the workload model into the test criteria (e.g. 100,000 Verify Identity calls ramped up within 30 seconds, with a defined request mix), and run it with a load-testing tool such as JMeter from several load generators. Pass if P90 < 3 s and the maximum < 5 s; repeat the run to confirm.

> 更可靠的办法：把工作负载模型写进测试标准（如 30 秒内逐步增加到 10 万次 Verify Identity 调用，规定请求组合），用 JMeter 这类负载测试工具从多台施压机执行。P90 < 3 秒、最大 < 5 秒为通过；重复运行确认。

Postman is a functional testing tool that sends requests from one machine, so it cannot create 100,000 concurrent users.

> Postman 是功能测试工具，从一台机器发请求，模拟不了 10 万并发用户。

写吞吐瓶颈 → 分片（PAST Q3(d)，6 分）

部分	要点
为什么加副本（replica）**没用**	读副本（read replica）只分担读，**所有写仍进一个主库**（Primary）
分片（sharding）**方案（2 分）**	按**唯一标识**（Voter ID / NRIC / 护照号）**哈希分片**（Hash），每片有自己的主库（可再配读副本）；可再加**事件流**（event streaming）**缓冲**写入峰值（peak）、批量写
受影响组件及理由（reason）**（4 分）**	逐个点名：**写入的控制器**要知道分片规则写对片；**存数据的服务**记录到正确分片；**用户服务**（用户分散在不同片）；**汇总**（aggregation）**服务**要跨片汇总。基础设施（infrastructure）：更多数据库实例、分片路由层、按片监控（monitoring）
要监控的副作用（side effect）	① 热点 / 不均匀的分片 ② 跨片汇总变慢（可能影响"1 分钟内更新"）③ "每人一票"等规则跨片更难保证 ④ 重新分片（Re-sharding）复杂、运维成本上升 ⑤ 每片的复制延迟（Replication Lag）

github.com/bibi45c · 免费分享，禁止售卖

分片策略	特点
哈希	分布均匀、少热点（`hash(voterId) mod N`）→ **默认选它**
范围	便于范围查询，但**易热点**（按注册日期分，新用户全挤在最新一片）
查找表	灵活，多一次查询

❝ 可用句式 | ___ 处填案例内容 **分片句子**

A read replica only scales reads; all writes still go to the single primary, so the write bottleneck stays. The writes must be spread by sharding.
读副本只扩展读；所有写仍进同一个主库，所以写瓶颈还在。必须用分片把写入分散开。

Shard the database by the voter's unique ID with a hash function, so the votes are spread evenly. Each shard has its own primary and replicas. Votes can also be buffered in an event stream and written in batches to absorb the 5-minute peak.
按选民唯一 ID 用哈希函数对数据库分片，让选票均匀分布。每个分片有自己的主库和副本。也可以先把选票缓冲在事件流里再批量写入，吸收 5 分钟的峰值。

Affected components: _____ Controller must know the sharding rule to write each record to the right shard; _____ Manager stores its data in the right shard; _____ Manager must aggregate results across all shards.
受影响的组件：_____ Controller 要知道分片规则，把每条记录写到正确的分片；_____ Manager 把数据存到正确分片；_____ Manager 要跨所有分片汇总结果。

Side effects to monitor: hot or uneven shards; slower cross-shard aggregation, which may delay the _____-minute results; harder checks for rules such as "one vote per voter"; and more complex re-sharding and operations.
要监控的副作用：热点或不均匀的分片；跨分片汇总变慢，可能拖慢 _____ 分钟内更新的结果；"每人一票"这类规则更难检查；重新分片和运维更复杂。

❝ 可用句式 | ___ 处填案例内容 **T11 补充：扩展立方体 • 读写分离（第 9 章）**

I would scale along the three axes of the scale cube. Y-axis: split _____ out of the monolith into its own service, so only this busy service gets more instances while _____ and _____ stay unchanged. X-axis: run several stateless replicas of it behind a load balancer with auto-scaling (HPA). Z-axis: shard the _____ data by _____ ID so writes are spread across shards.
我会沿扩展立方体三个方向扩展。Y 轴：把 _____ 从单体拆成独立服务，只给这个忙的服务加实例，_____ 和 _____ 不动。X 轴：在负载均衡器后面运行它的多个无状态副本，配自动扩缩（HPA）。Z 轴：按 _____ ID 对 _____ 数据分片，把写入分散到各分片。

I would apply CQRS: _____ events update a pre-computed read model (e.g. results per _____) in a cache, and _____ query this read model directly. Reads and writes then scale independently and no longer compete for the same database.
我会用 CQRS：_____ 事件更新缓存里一个预先算好的读模型（如每个 _____ 的结果），_____ 直接查这个读模型。读和写各自扩展，不再争同一个数据库。

With event sourcing, every change is stored as an event, so we have a full audit trail and can replay the events to rebuild the state, for example to recompute _____ after the algorithm is improved.
用事件溯源，每次变化都存成一个事件，所以有完整的审计记录，还能重放事件重建状态，比如算法改进后重新计算 _____。

性能论证用的数字和公式

想说明	怎么写
串行（Serial） vs 并行（Parallel）	串行：响应时间（response time）**相加**、吞吐（throughput） = **min**；并行：响应时间取最长、吞吐**相加**
为什么阈值 60–70%	R = S / (1 − U)：50% 时 2 倍，70% 时超过 3 倍，95% 时约 20 倍；新实例启动要时间，要**提前**扩
Little 定律	n = λ × R：每秒 1,000 个 × 0.2 秒 = 系统里同时 200 个
瓶颈（bottleneck）	总服务需求（Service Demand） D = 访问次数 × 服务时间（Service Time），**D 最大的是瓶颈**，别处加资源（Resource）没用
缓存（cache）	命中率（hit ratio）低时几乎无效，**未命中比不加缓存还慢**
扩缩准则	主动（算启动时间）；**扩容**（scale out）**激进、缩容**（scale in）**保守**；避免乒乓；可按时间表预先扩容（投票日前）
容量规划（capacity planning）	存储看**累积**（复合增长）；计算看**峰值**（peak）；加机器**不线性**（Amdahl）

❝ 可用句式 | ___ 处填案例内容 **性能**（performance）**论证**（justification）**句子**

In a serial chain the throughput is limited by the slowest component, so every component must handle the target rate. The bottleneck is _____ (_____ requests per second); it is removed by _____. Assume writes are the bottleneck.
串行链路的吞吐量受最慢的组件限制，所以每个组件都要能达到目标速率。瓶颈是 _____（每秒 _____ 个请求），用 _____ 解决。假设写入是瓶颈。

Above about 70% utilisation the queueing time grows sharply, and new instances need time to start, so auto-scaling starts at 60 to 70% CPU, before the knee of the curve.
利用率超过约 70% 后排队时间急剧增长，而且新实例启动需要时间，所以自动扩容在 CPU 60–70% 时开始，在曲线拐点之前。

Scale out aggressively and scale in conservatively, with a cool-down period, to avoid the ping-pong effect.
扩容要激进、缩容要保守，并设冷却时间，避免乒乓效应。

真题范文（model answer）：PAST Q1(a)(i) p.**43** · PAST Q3(d) p.**47**。

github.com/bibi45c · 免费分享，禁止售卖

PART 1 题型卡 T1–T17（怎么认 · 怎么写）

T13–T17 可用性 · 容量 · ISO 25010 · 安全 · 韧性迁移

× 常见误区 | 这样答会丢分 **这五类三套样卷**（sample paper）**都没考**

课程教过、老师没说不考。没有官方评分，骨架按 T1–T12 的通用给分逻辑（观点 + 案例数字 + 理由（reason））推断。**时间不够就先跳过这一章**；真遇到再翻。安全（security）一般不是重点，但论证（justification）合理也给分，可以在部署（deployment）/ 架构题里作补充论据。

T13 可用性计算与高可用设计

触发词： availability, nines, downtime, MTBF, MTTR, single point of failure, redundancy。

量	公式
可用性（availability）	A = MTBF / (MTBF + MTTR)；年停机分钟 = (1 − A) × 365 × 24 × 60
串联（in series）	A = A1 × A2 × A3（**低于任何一个组件**；三个 99% 串联约 97%）
并联（in parallel）	A = 1 − (1 − A1)(1 − A2)（**高于任何一条支路**；两个 99% 并联 99.99%）
几个 9	99% ≈ 3.65 天 / 年；99.9% ≈ 8.8 小时；99.99% ≈ 53 分钟；99.999% ≈ 5 分钟
Yield / Harvest	Yield = 成功请求 / 总请求（比 uptime 更重要）；Harvest = 可用数据 / 全部数据

算题六步： ① 画请求路径，标每个组件的 A ② 冗余（redundancy）的一组先按并联算成一个值 ③ 整条路径串联相乘 ④ 和目标比，**最弱的组件就是最大单点约束**（constraint） ⑤ 给改造方案再算一遍 ⑥ 写**独立性假设**（assumption）（共用电源、存储、同一个软件缺陷会让实际值更低）。

课程作业范本（数字照抄）： 服务器各 99%、防火墙（firewall）各 99.9%、Mainframe 98%。现状 $0.999^2 \times 0.99^4 \times 0.98 = \mathbf{93.95\%} < 98\%$；每层改成一对：防火墙对 99.9999%、服务器对 99.99%、Mainframe 对 99.96% → $0.999999^2 \times 0.9999^4 \times 0.9996 = \mathbf{99.92\%}$，达标。

提高可用性的战术： 检测（Ping/Echo、心跳（heartbeat）、健康检查（health check）/ 存活探针）→ 恢复（主动 / 被动冗余（Passive Redundancy）、备用、重试（retry）、降级（degradation）、回滚（rollback））→ 重新引入（影子运行、状态重同步（synchronous）、逐级重启）→ 预防（移出服务、事务（transaction））；云上多可用区（Multi-AZ）、K8s ReplicaSet 自愈（Self-Healing）；用混沌工程（Chaos Engineering）验证。老师说：**2% 的不可用多来自软件更新，不是硬件** → 部署（deployment）策略（金丝雀（canary）、蓝绿（blue/green））也是可用性战术。

“ 可用句式 | ___ 处填案例内容 **T13 句子**

The components are in series, so the overall availability is the product: _____ × _____ = _____%, which is below the target of _____%. In series, the total is always lower than any single component.

组件是串联的，所以整体可用性是乘积：_____ × _____ = _____%，低于 _____% 的目标。串联时整体总是低于任何一个组件。

The weakest component is _____ (_____%). Making it redundant gives 1 − (1 − _____)² = _____%, and the end-to-end availability rises to _____%.

最弱的组件是 _____（_____%）。给它做冗余后为 1 − (1 − _____)² = _____%，端到端可用性提高到 _____%。

This assumes the redundant nodes fail independently; shared power, storage or the same software bug would lower it, so failover must be tested.

这假设冗余节点各自独立失效；共用电源、存储或同一个软件缺陷都会让它变低，所以必须演练故障切换。

“ 可用句式 | ___ 处填案例内容 **T13 补充：先写出数字，外部依赖怎么改**

The components are in series, so the availability is _____ × _____ × _____ × _____ ≈ _____%, which is below the target of _____%.

这些组件是串联的，所以可用性 = _____ × _____ × _____ × _____ ≈ _____%，低于 _____% 的目标。（一定要写出算出来的数）

The weakest component is the external _____ (_____%). We cannot make an external system redundant inside our platform, and multi-AZ does not help it.

最弱的是外部的 _____（_____%）。外部系统没法在我们平台内部做冗余，多可用区对它也没用。

Option 1 – Second provider: add a second _____ and fail over to it. Two in parallel give 1 − (1 − _____)² ≈ _____%, so the end-to-end availability rises to about _____%.

方案 1（第二家供应商）：再接一家 _____，故障时切换过去。两家并联 = 1 − (1 − _____)² ≈ _____%，端到端可用性提高到约 _____%。

Option 2 – Take it off the synchronous path: the request finishes after _____ is recorded, and the call to _____ is made asynchronously with retries and an idempotency key. The synchronous path is then _____ × _____ × _____ ≈ _____%. Assume the business accepts a delayed _____.

方案 2（移出同步路径）：请求在记录 _____ 后就结束，对 _____ 的调用改为异步，带重试和幂等键。这样同步路径只剩 _____ × _____ × _____ ≈ _____%。假设业务接受稍后再 _____。

github.com/bibi45c · 免费分享，禁止售卖

“可用句式｜___处填案例内容 **T13 补充：冗余要配检测和切换 · 用词 · 备份（第 13 章）**

I would run _____ in two Availability Zones (e.g. Amazon RDS Multi-AZ). Health checks detect a failed instance and traffic fails over to the standby automatically, so there is no single point of failure.

| 我会把 _____ 部署在两个可用区（如 Amazon RDS Multi-AZ）。健康检查发现实例故障后，流量自动切换到备用实例，因此没有单点故障。

Availability is whether _____ can be used when it is needed (e.g. 99.9% of the time); reliability is whether it keeps working correctly over a period without failure. _____ is mainly an availability requirement.

| 可用性是 _____ 在需要时能不能用（如 99.9% 的时间）；可靠性是它在一段时间内能否持续正确工作、不出故障。_____ 主要是可用性要求。

We take a full backup every Sunday and incremental backups on the other days. Daily backups are fast and small, but restoring after a failure on Friday needs Sunday's full backup plus every incremental backup from Monday to Thursday, applied in order.

| 每周日做完全备份，其他日子做增量备份。每天的备份快、占空间小，但周五出故障时，恢复需要周日的完全备份加上周一到周四的每一份增量，按顺序恢复。

T14 容量规划与估算

触发词： capacity, sizing, storage for N years, growth, forecast, utilisation。**两条总原则：存储看累积（cumulative），服务器看峰值（peak）。**

步	做什么
1 目标	够用 + 可接受的服务水平
2 预测负载	线性回归 Y = a + bX、移动平均、指数平滑；区分随机 / 趋势 / 季节性
3 负载与资源（Resource）的关系	厂商基准、自己做基准测试（Benchmark）、历史监控（monitoring）
4 资源需求	加采购提前期（Procurement Lead Time）；副本（replica）和备份**单独**算，避免重复

范本（数字照抄）： 峰值 = 日量 × 60% ÷ 240 分钟（60% 集中在 4 个忙时）→ 324,000/天 → 平均 675/分钟、峰值 810/分钟；5 年磁盘 15.97 → 81.53 TB，按 ≥ 120 TB 配（30% 余量（headroom））。加机器不是线性：1 / 2 / 4 台 Max TPS 17.04 → 30.81 → 55.76（Amdahl：串行（Serial）部分决定上限）。

“可用句式｜___处填案例内容 **T14 句子**

Server capacity is sized for the peak, not the average: peak = _____ requests × _____% ÷ _____ minutes = _____ per minute.

| 服务器容量按峰值而不是平均值定：峰值 = _____ 个请求 × _____% ÷ _____ 分钟 = 每分钟 _____。

Storage is cumulative: _____ per record × _____ records, growing _____% per month over _____ years ≈ _____ TB, plus _____% headroom; replicas and backups are sized separately.

| 存储是累积的：每条 _____ × _____ 条，每月增长 _____%，_____ 年约 _____ TB，再加 _____% 余量；副本和备份单独计算。

T15 质量属性理论（ISO 25010 / 效用树 / ASR）

概念	要点
ISO 25010 八特性（2011 版；2023 版增 Safety 为九个）	Functional Suitability、**Performance Efficiency**、Compatibility、Usability、**Reliability**、**Security**、**Maintainability**、Portability；答题时用子特性更准（Performance Efficiency → time behaviour / capacity）
属性间关系	正相关 / 无关 / **负相关**（安全（security） ↔ 性能（performance）、安全 ↔ 易用、一致性 ↔ 可用性（availability））
效用树（Utility Tree）	根 Utility → 质量属性（quality attribute） → 细化 → 叶子场景（scenario）；叶子按（重要性, 实现难度 / 风险（risk））打分，**(H, H) 先做**，最能左右架构
架构上重要的需求（Architecturally Significant Requirement (ASR)） ASR	关联（Association）关键功能（80/20）、关键质量、关键约束（constraint）（如必须集成某外部系统）、带来技术风险
决策记录	Issue / Decision / Assumptions / Alternatives / Justification
可用性 vs 可靠性（reliability）	可用性 = 需要时能不能用；可靠性 = 一段时间内能不能正确工作

“可用句式｜___处填案例内容 **T15 句子**

Using ISO 25010, the most important characteristic is _____ (sub-characteristic: _____), because Section 4 requires _____.

| 按 ISO 25010，最重要的特性是 _____（子特性：_____），因为第 4 节要求 _____。

In the utility tree, the scenario "_____" is rated (H, H): it is very important to the stakeholders and hard to achieve, so it drives the architecture.

| 在效用树里，场景"_____"评为 (H, H)：对干系人很重要、实现难度大，所以它驱动架构。

_____ and _____ have an inverse relationship: improving _____ (e.g. encrypting every call) costs _____ (latency), so the trade-off is _____.

| _____ 和 _____ 此消彼长：提高 _____（如每次调用都加密）会牺牲 _____（延迟），所以取舍是 _____。

github.com/bibi45c · 免费分享，禁止售卖

T16 安全取舍

链条： CIA（保密性（confidentiality）、完整性（integrity）、可用性（availability）；另有真实性（Authenticity）、授权（Authorisation）、不可否认（Non-Repudiation））→ **风险**（risk）**= 威胁**（threat）**+ 漏洞**（vulnerability）（T + V = R）→ 资产（Asset）分类（皇冠明珠 crown jewels / 跳板 stepping stones）→ **STRIDE** → 风险 = 可能性 × 影响（DREAD）→ 处置：缓解（mitigate）/ 规避 / 转移 / 接受。

STRIDE	破坏的属性	STRIDE	破坏的属性
Spoofing 仿冒	真实性	Information disclosure 泄露	保密性
Tampering 篡改	完整性	Denial of service 拒绝服务	可用性
Repudiation 抵赖	不可否认	Elevation of privilege 提权	授权

原则： 纵深防御（defence in depth）（多层）、最小权限（least privilege）（最少权限、最短时间）、完全仲裁（Complete Mediation）（每次访问都检查，不缓存（cache）授权决定）、最少公共机制。TLS 只保护 TLS 客户端到服务器这一段，不一定端到端。云上：责任共担模型（Shared Responsibility Model）、IAM、传输中和静态加密（encryption）（谁持有密钥（key）很关键）、租户专属密钥（Tenant-Specific Key）、DMZ 网络分区（partition）。

“ 可用句式 | ___ 处填案例内容 **T16 句子**

_____ (threat) exploits _____ (vulnerability) to harm _____ (asset), resulting in _____ (consequence).
_____（威胁）利用 _____（漏洞）损害 _____（资产），造成 _____（后果）。

Controls: preventive – _____; detective – _____; recovery – tested offline backups. With several layers, if one layer is broken, the others still protect the system.
控制措施：预防类 _____；检测类 _____；恢复类：经过测试的离线备份。有多层保护，一层被攻破，其他层仍能保护系统。

Trade-off: _____ improves confidentiality but reduces _____ (performance / usability); this is acceptable because _____.
取舍：_____ 提高了保密性，但降低了 _____（性能 / 易用性）；这可以接受，因为 _____。

T17 韧性、发布与迁移

主题	要点
故障机理	连锁反应、**级联故障**（cascading failure）（下游慢沿调用链传播）、**重试风暴**（Retry Storm）、缓存踩踏（Dogpile / Cache Stampede）dogpile、吵闹邻居（Noisy Neighbour）、分片（sharding）不均
对策（countermeasure）	**超时**（timeout）**+ 指数退避重试**（Retry with Exponential Backoff）（50/100/200 ms）；**熔断器**（Circuit Breaker）**三态**：关闭 → 打开（直接返回错误 / 上次的值 / 通用答案）→ 半开（放少量请求试探）；**舱壁**（bulkhead）隔离（isolation）；减载（Load Shedding）、背压（Backpressure）、限流（rate limiting）；跨服务（cross-service）事务（transaction）用 **Saga**（本地事务 + 补偿事务，编排（orchestration）或协同（choreography））
部署（deployment）策略	**蓝绿**（blue/green）（两套同时在线，一键切换回滚（rollback），资源（Resource）翻倍）；**滚动**（逐台替换，新旧要兼容）；**金丝雀**（canary）（新版本（version）先接 5–10% 流量，适合试新功能、新 ML 模型）；**影子流量**（Shadowing）（同时发新旧，只用旧结果）
不兼容的表结构变更	**expand – migrate – contract**：建新结构 → 双写（新写新表、读不到再读旧表、删除打墓碑标记（Tombstone））+ 金丝雀 → 迁历史数据 → 只读新表 → 删旧；每步可回滚
上云迁移（migration）7R（课件；课堂讲六种，Replace 即 Repurchase）	Rehost（搬到 VM）、Replatform（搬到容器（Container）/ 托管库）、Rearchitect、Rebuild、Replace（买 SaaS）、Retain、Retire；**成本 / 风险**（risk）**优先 → rehost、replatform；敏捷 / 创新 → rearchitect**

“ 可用句式 | ___ 处填案例内容 **T17 句子**

To stop the failure of _____ from spreading, calls to it use a timeout and a circuit breaker. When the failure rate passes the threshold, the breaker opens and returns _____ (the last good value); after a cool-down it goes half-open to test recovery.
为防止 _____ 的故障扩散，调用它时加超时和熔断器。失败率超过阈值时熔断器打开，返回 _____（上次的正确值）；冷却后进入半开状态试探是否恢复。

To change the schema with almost no downtime and quick rollback, use expand–migrate–contract: _____ → _____ → _____, released as a canary (5–10% of traffic). The old table is kept until the new version is proven.
要几乎不停机、能快速回滚地修改表结构，用 expand–migrate–contract：_____ → _____ → _____，以金丝雀方式发布（5–10% 流量）。新版本被证明可靠之前保留旧表。

I recommend _____ (e.g. replatform) for _____, because the main driver is _____ (cost / risk / agility); _____ would give more agility but a higher cost of change.
我建议对 _____ 采用 _____（如 replatform），因为主要驱动是 _____（成本 / 风险 / 敏捷）；_____ 更敏捷，但变更成本更高。

github.com/bibi45c · 免费分享，禁止售卖

PART 2　样卷逐题范文（简单英文 + 中文）

PAST　Vote Anywhere 投票平台（有官方答案）

怎么用这一章

每题四块：**题目大意（中文）→ 题型**（question type）**与得分点**（marking points）**→ 简单英文范文**（model answer）**（一句英文一句中文）→ 易错。**范文用短句和常用词写成，比官方答案短，但得分点一个不少。原题按页码回看 `PM Sample Exam Questions\`。**PAST 是唯一有官方答案和评分的样卷**（sample paper）。

案例速读：Vote Anywhere

选民用手机在任何地方投票；监考员 Proctor 在线核验身份，把选民逐个请进"投票室"；观察员 Observer 实时看结果。云原生（Cloud-Native），服务用微服务（microservices）实现。

附录	要抄的事实
§4 并发（concurrency）	10 Election Admins、1,000 Proctors、1,000 Observers、**100,000 Voters**、5 Platform Admins
§4 响应	交互（interaction）请求（如 Verify Identity）**90% < 3 秒**；简要结果平均 **1 分钟**内更新，详细结果平均 **3 分钟**内
§4 其他	**不预期多租户**（multitenancy）；安全（security）：DMZ 反向代理（reverse proxy）、个人信息加密（encryption）、GDPR
§7 领域模型（domain model）	UserManager（PlatformAdmin、User）；PollManager（ElectionAdmin、Poll、PollReport、Candidate）；ProctoringManager（Proctor、PollingRoom、ProofOfIdentity）；ObservationManager（Observer、CurrentDetailedResults、CurrentSimplifiedResults）；VotingManager（Voter、CastVote）；NotificationManager（6 个专属 Notif + 通用 NotificationEmail）
§8 Figure 6	只画投票用例的手机端（mobile device）打包（packaging）部署（deployment）；**图注：不完整、可能有错、刻意省略**（deliberately omitted）**后端**（backend）**部署**

题干难词	意思	题干难词	意思
proctor	监考员	cast a vote	投票
polling room	投票室	deterministic and repeatable	结果确定、可重复
verify identity	核验身份	drastic adaptation	大幅改造
promptly	及时地	deprecated	被废弃的（版本（version））
crucial	最关键的（crucial / critical）	proposal	提议

Q1(a)(i) 性能测试结果不稳定 • Q1(a)(ii) 队列还是事件流

Q1(a)(i) 题目大意 • T11 • 2 分（原因（cause）**1 + 办法 1）**

Verify Identity 要求 10 万选民下 90% 请求 < 3 秒；测试参数有 30 秒时间窗、最大 < 5 秒。几位测试员用 Postman 测，结果各不相同。**给一个可能原因 + 一个更可靠的测试办法。**

✍ **参考答案｜可照着写，名字和数字换成新案例的**　**Q1(a)(i)**

Cause: The testers did not share one workload model. The criteria give the number of voters, the 30-second window and the 5-second maximum, but not how the load grows over time or the mix of requests. Each tester made different assumptions, so the results are different.

> 原因：测试员没有统一的工作负载模型。标准给了选民数、30 秒时间窗和 5 秒上限，却没说负载如何随时间增加、请求怎样组合。每个人假设不同，所以结果不同。

Better way: Write the workload model into the test criteria, for example 100,000 Verify Identity calls ramped up within 30 seconds with a fixed request mix. Run it with a load-testing tool such as JMeter from several machines. It passes if P90 < 3 s and the maximum < 5 s. Repeat the test to check that the result is stable.

> 更好的办法：把工作负载模型写进测试标准，例如 30 秒内逐步增加到 10 万次 Verify Identity 调用、请求组合固定。用 JMeter 这类负载测试工具从多台机器执行。P90 < 3 秒、最大 < 5 秒为通过。重复测试确认结果稳定。

Q1(a)(ii) 题目大意 • T10 • 2 分（每个理由（reason）**1 分）**

"更新简要 / 详细结果"用例：简要结果 1 分钟、详细结果 3 分钟内更新，结果要**确定且可重复**。有人想在每次投票结束时用**消息队列**（message queue）触发它，你主张用**事件流**（event streaming）。**给两个理由。**

github.com/bibi45c · 免费分享，禁止售卖

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(a)(ii)**

Reason 1 – Order: The results must be deterministic and repeatable, so the vote events must be processed in the order they happened. An event stream keeps the order inside a partition (partitioned by poll). With a message queue, several consumers compete and may process votes out of order.

理由一（顺序）：结果要确定、可重复，所以投票事件必须按发生顺序处理。事件流在分区内保持顺序（按 poll 分区）。消息队列上多个消费者互相竞争，可能乱序处理。

Reason 2 – Read twice: The use case updates both the simplified and the detailed results, so the same vote events must be read twice. An event stream keeps the events for a period, so two consumer groups can read them and replay them. A message queue deletes a message once it is delivered.

理由二（读两遍）：这个用例要同时更新简要结果和详细结果，同一批投票事件要读两遍。事件流会把事件保留一段时间，两个消费者组都能读、还能重放。消息队列在消息投递后就删除了。

易错： 只写"Postman 不好，换工具"（关键是测试设计统一）；写"事件流全局有序"（只在分区（partition）内有序，要说按什么分区）。

Q1(b) 领域模型诊断（9 分）

Q1(b)(i) 题目大意 • T1 • 5 分（原因（cause）**2 + 对策**（countermeasure）**1.5 + 1.5 + 缺点**（drawback）**0）**

NotificationManager 定义平台所有通知，而各类通知其实属于别的服务。你认为它**难以复用**（reuse）**到其他平台。原因 / 改领域模型**（domain model）**的对策 / 缺点。**

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(b)(i)**

Cause: All kinds of notifications are put in one service. The NotificationManager contains business-specific entities: ProctoringNotif, PollingNotif, VotingCompletionNotif, RejectionNotif, ObservationNotif and VerificationNotif. They belong to other services. Another platform does not need them, so the service cannot be reused without big changes.

原因：所有类型的通知都放在一个服务里。NotificationManager 包含业务专属实体：ProctoringNotif、PollingNotif、VotingCompletionNotif、RejectionNotif、ObservationNotif、VerificationNotif。它们属于别的服务。别的平台用不到，所以不大改就无法复用。

Countermeasure: (1) Move each specific notification to the service that owns it: VerificationNotif to UserManager, ProctoringNotif to ProctoringManager, ObservationNotif to PollManager, and PollingNotif, VotingCompletionNotif and RejectionNotif to VotingManager. (2) Keep only the generic NotificationEmail in the NotificationManager, so it becomes a generic service that other platforms can reuse. (Only the changed boxes are drawn.)

对策：（1）把每种专属通知移到它所属的服务：VerificationNotif 到 UserManager，ProctoringNotif 到 ProctoringManager，ObservationNotif 到 PollManager，PollingNotif、VotingCompletionNotif、RejectionNotif 到 VotingManager。（2）NotificationManager 只留通用的 NotificationEmail，成为别的平台能复用的通用服务。（只画改动的框。）

Drawback: No major drawback. The notifications are now separated by kind, so each owning service can also scale on its own.

缺点：没有大的缺点。通知按类别分开后，各所属服务还能独立扩展。

Q1(b)(ii) 题目大意 • T1 • 4 分（原因 1.5 + 对策 1.5 + 缺点 1）

你担心 Observer **无法及时**看到所监看 Poll 的最新 CurrentSimplifiedResults。**原因 / 对策 / 缺点。**

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(b)(ii)**

Cause: Poll is in the PollManager, but Observer and the results are in the ObservationManager. To view the results of a poll, the two services must work together, so every request needs inter-process communication. This is much slower than an in-process call.

原因：Poll 在 PollManager 里，而 Observer 和结果在 ObservationManager 里。查看某个投票的结果需要两个服务协作，每个请求都要跨进程通信，比进程内调用慢得多。

Countermeasure: Move the entities of the ObservationManager (Observer, CurrentSimplifiedResults, CurrentDetailedResults) into the PollManager. Then the results are read inside one service, and Observers see them promptly.

对策：把 ObservationManager 的实体（Observer、CurrentSimplifiedResults、CurrentDetailedResults）移进 PollManager。结果在一个服务内读取，观察员就能及时看到。

Drawback: The PollManager becomes a larger service with a larger transaction scope. This can be reduced by using several aggregates inside the service.

缺点：PollManager 变成更大的服务，事务范围也变大。可以在服务内划分多个聚合来缓解。

易错： 写"加缓存（cache）、加负载均衡（Load Balancing）"（要改领域模型）；对策不点名搬去哪；缺点不写。

Q1(c) API 表：RMM、版本、重构（6 分）

题目大意 • T3 • 三问各 2 分（解释 1 + 改进 1）

两个接口都属 ProctoringManager，自称 RMM Level 2：Invite Next Voter = `GET /v1.1/votes/{voter-id}/invite`（输入选民 ID，输出投票室里的选民列表）；Start Vote = `GET /v1.1/votes/{voter-id}/start`（输出候选人列表和选民列表）。① 是不是 Level 2？② 更好的版本（version）策略？③ 重画正确的表（有多余 / 缺失的输入输出、可能错的动词）。

github.com/bibi45c · 免费分享，禁止售卖

URI	Service	Inputs	Outputs	HTTP
`/votes/invite`	ProctoringManager	（无）	Voter-id of the invited voter	**POST**
`/votes/{voter-id}/start`	ProctoringManager	Voter identifier	List of candidates of the poll	**POST**

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(c)**

1. Not Level 2. Level 2 needs each operation to use the HTTP verb that matches what it does. Inviting a voter and starting a vote both change the state on the server, but they use GET. GET must be safe and have no side effects, so both should use POST.

> 不符合 Level 2。Level 2 要求每个操作用和它的作用相符的 HTTP 动词。邀请选民、开始投票都会改变服务器状态，却用了 GET。GET 必须安全、无副作用，所以两个都应该用 POST。

2. Versioning: The version is in the URI (/v1.1/...). When a version is retired, the URI changes and client code breaks. A better way is a custom request header such as `Accept-version: 1.1`. The URIs stay clean and stable, and only breaking changes raise the major version.

> 版本：版本号写在 URI 里（/v1.1/...）。某个版本废弃时 URI 会变，客户端代码就出错。更好的办法是用自定义请求头，如 `Accept-version: 1.1`。URI 保持干净稳定，只有不兼容的改动才升主版本号。

3. Corrected table: see above. Invite has no input, because the platform takes the next voter from the queue; it returns the invited voter-id. Start Vote only returns the candidates, so the list of voters is removed. Both use POST, and the version is removed from the URI.

> 改正的表见上。Invite 没有输入，因为平台从队列里取下一位选民；输出被邀请者的 voter-id。Start Vote 只返回候选人，所以删掉选民列表。两个都用 POST，URI 里去掉版本号。

易错： 只改动词，忘了删多余的输入输出、忘了去掉 `/v1.1`。

Q1(d) 易用性是不是最重要的非功能需求

题目大意 • T7 • 2 分（每个理由（reason） 1 分）

组员认为，考虑到选民很多，系统（尤其是帮监考员管理投票场次）的**易用性**（usability）是最关键的（crucial / critical）非功能需求（non-functional requirement）。**你同意吗？说明理由。**

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(d)**

I disagree. Usability for the proctors matters, but it is not the most crucial requirement.

> 我不同意。监考员的易用性有意义，但不是最关键的需求。

(1) **Performance and scalability are more critical.** The platform must support 100,000 concurrent voters, with 90% of requests within 3 seconds. If it cannot, voters cannot vote during the poll, however easy the screens are.

> （1）性能和可扩展性更关键。平台要支持 10 万并发选民，90% 的请求 3 秒内完成。做不到的话，界面再好用，选民也无法在投票期内投票。

(2) **Security is also crucial.** The platform must make sure each voter votes at most once. Otherwise the results cannot be trusted.

> （2）安全也很关键。平台必须保证每位选民最多投一次。否则结果不可信。

Q1(e) "每日处理选民数"能不能当指标

题目大意 • T8 • 4 分（每个指标（metric）连同理由（reason） 2 分）

组员提议用**每天处理的选民数**作为应用指标（Application Metric），认为数字高就说明可扩展性（scalability）和易用性（usability）好。**同意吗？说明理由。**

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(e)**

I disagree. The daily number of voters mainly shows how many voters registered and came, not how well the system works. A high number may even hide long queues. It says nothing about the peak load (scalability) or how smoothly voters and proctors finish their tasks (usability). Better metrics are:

> 我不同意。每日选民数主要反映注册和到场的人数，而不是系统表现好不好。数字高甚至可能掩盖了长队。它说明不了高峰负载下的表现（可扩展性），也说明不了选民和监考员完成任务是否顺畅（易用性）。更好的指标是：

Average identity-verification time – shorter verification lets the platform serve more voters at peak and shows that the system supports the proctors well.

> 平均身份核验时长：核验越快，平台高峰时能服务越多选民，也说明系统对监考员支持得好。

Average queueing time per voter – it shows the voter's experience and whether the platform keeps up during the voting window.

> 每位选民平均排队时间：反映选民体验，以及平台在投票时间窗内是否跟得上。

(Also acceptable: **average time to cast a vote** – it tracks the usability and performance of the main use case.)

> （也可以：平均投票用时，同时跟踪主用例的易用性和性能。）

github.com/bibi45c · 免费分享，禁止售卖

Q2(a) 单体 → 混合云原生参考架构 + K8s（10 分）

题目大意 • T5 • 10 分（参考架构（Reference Architecture） 4 + 通信组件 2 + 缓存（cache） 1 + K8s 部署（deployment） 3）

团队做了一个单体（monolith） MVP。把它改造成投票平台后端（backend）的**混合参考架构**（微服务（microservices）、定时任务（scheduled job）、事件 / 流驱动、Serverless 都可考虑）：(i) 画云视角主要组件和交互（interaction），多云（Multi-Cloud）还需要哪些组件；(ii) 除 API Gateway 外，内部服务怎么通信，在图上标出；(iii) 指出（identify）一个需要保存状态的服务和状态管理机制；(iv) 用 K8s 部署图或文字描述部署计划。

▶ 案例演练｜演示怎么套用 **画图清单（照着画框）**

手机 App / Web → **CloudFront** → **API Gateway** + 负载均衡（Load Balancing） / **Ingress** → EKS 集群（Cluster）：**Voting、Poll、Proctoring、Observation、User** 微服务（各自数据库 **PostgreSQL**）· **服务注册中心**（service registry）· **Istio 服务网格**（service mesh） → **Kafka 事件流**（event streaming）（VoteCast 事件）→ Observation 汇总（aggregation）· **Redis（ElastiCache）**存进行中的投票和计数 · **CronJob** 生成报表 · **Lambda + SNS** 发通知 · **GitOps CI/CD + 镜像仓库**（Container Registry） **ECR** · **监控**（monitoring） **（Prometheus / CloudWatch）**。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q2(a)**

(i) Components: Voters' apps and web pages reach the platform through CloudFront and Amazon API Gateway with a load balancer and an Ingress. The backend is hybrid. Voting, Poll, Proctoring, Observation and User run as microservices on Kubernetes (Amazon EKS), each with its own PostgreSQL database. Results are aggregated from an event stream (Kafka). Poll reports run as scheduled jobs. Notifications are serverless functions (Lambda + SNS). Other parts: a service registry, a container registry, GitOps CI/CD, a Redis cache and monitoring. **For multi-cloud** we also need Kubernetes for portability, Infrastructure as Code, a global load balancer and DNS for failover, one identity provider, unified monitoring and registry replication.

> (i) 组件：选民 App 和网页经 CloudFront、Amazon API Gateway（加负载均衡和 Ingress）进入平台。后端是混合架构。Voting、Poll、Proctoring、Observation、User 以微服务运行在 Kubernetes（Amazon EKS）上，各自有 PostgreSQL 数据库。结果由事件流（Kafka）汇总。投票报表用定时任务。通知用 Serverless 函数（Lambda + SNS）。其他：服务注册中心、镜像仓库、GitOps CI/CD、Redis 缓存、监控。多云还需要：Kubernetes 保证可移植、基础设施即代码、全局负载均衡和 DNS 做故障转移、统一身份、统一监控、镜像仓库复制。

(ii) Internal communication: Queries that need an answer at once use synchronous REST or gRPC through service discovery and the Istio service mesh (for example, Proctoring checks a voter's status). Votes use asynchronous publish/subscribe: each cast vote publishes a VoteCast event to Kafka, and the Observation service consumes it. This decouples the services, absorbs the peak, and keeps events if a consumer is down.

> (ii) 内部通信：需要立刻得到结果的查询用同步 REST 或 gRPC，经服务发现和 Istio 服务网格（如 Proctoring 查询选民状态）。投票用异步发布 / 订阅：每投一票就向 Kafka 发布 VoteCast 事件，由 Observation 服务消费。这样服务解耦、能吸收峰值，消费者宕机时事件也不丢。

(iii) Stateful service: The Voting service keeps the votes in progress and the running counts. The state is kept outside the pods in Redis (ElastiCache) and saved to PostgreSQL, so the pods stay stateless and can be scaled or replaced without losing votes.

> (iii) 有状态服务：Voting 服务保存进行中的投票和实时计数。状态放在 Pod 外的 Redis（ElastiCache）里，并保存到 PostgreSQL，所以 Pod 保持无状态，扩容或替换都不会丢票。

(iv) Kubernetes plan: One namespace for production, with nodes in three availability zones. Each microservice is a **Deployment** with at least three **Pods**, behind a ClusterIP **Service**. An Ingress routes /votes, /polls and /results to the Services behind the API Gateway. An HPA scales Voting and Proctoring from 5 to 50 Pods on CPU (assume one Pod serves about 2,000 voters). The report is a CronJob. Kafka, Redis and the databases use managed services. Configuration is in ConfigMaps and Secrets, and releases use rolling updates.

> (iv) Kubernetes 计划：生产环境一个 namespace，节点分布在三个可用区。每个微服务是一个 Deployment，至少 3 个 Pod，前面是 ClusterIP Service。Ingress 在 API Gateway 后面把 /votes、/polls、/results 路由到各 Service。HPA 按 CPU 把 Voting 和 Proctoring 从 5 个扩到 50 个 Pod（假设一个 Pod 服务约 2,000 名选民）。报表用 CronJob。Kafka、Redis 和数据库用托管服务。配置放 ConfigMap 和 Secret，发布用滚动更新。

易错： 只堆服务名不画交互、不标通信组件（丢 2 分）；K8s 计划没写 Pod / Deployment / Service（丢 3 分）。

Q2(b) 原生 App 还是 PWA

题目大意 • T10 • 3 分（每点 1 分）

选民前端（front end）要本地存储（local storage）备份；将来扩展到其他国家，设实体（entity）投票站，配 **IoT 传感器和执行器**。前端做原生还是 PWA？对后端（backend）有什么影响？

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q2(b)**

Storage: Both a native app and a PWA can keep client-side state (local storage, session storage), so local backup does not decide the choice.

> 存储：原生应用和 PWA 都能保存客户端状态（local storage、session storage），所以本地备份决定不了选哪个。

Decision: The future polling stations will use IoT sensors and actuators for voters with special needs. This needs deep access to the device hardware, so I choose a **native app**.

> 决定：未来的投票站要为有特殊需要的选民使用 IoT 传感器和执行器，需要深度访问设备硬件，所以我选原生应用。

Backend: With client-side orchestration, routing and shared state are handled in the client, which registers the APIs it needs. So the backend services do not need to change; the same APIs serve all clients.

> 后端：采用客户端编排，路由和共享状态在客户端处理，客户端注册它需要的 API。所以后端服务不需要修改，同一套 API 服务所有客户端。

github.com/bibi45c · 免费分享，禁止售卖

Q3(c) 打包部署模型找错（6 分）

题目大意 • T4 • 每个错：识别（identify）**1 + 纠正**（correction）**1 + 论证**（justification）**1，答两个**

对照 §7 领域模型（domain model）和 §8 Figure 6，找出**两个**不理想的（undesirable）架构决策（architecture decision），提出纠正，并解释。可以只画必要部分。（原卷印作 c，老师课上称 Q3(a)。）

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q3(c)（三个里任选两个）**

Decision 1: VotingCompletionNotif is packaged into the Voting Manager Data Access subsystem. **Correction:** package it into the Notification Manager Data Access subsystem. **Justification:** In the domain model it belongs to the NotificationManager service. Packaging it with VotingManager breaks the service boundary and couples the two services.

> 决策 1：VotingCompletionNotif 被打包进 Voting Manager Data Access 子系统。纠正：改打包进 Notification Manager Data Access 子系统。论证：领域模型中它属于 NotificationManager 服务。和 VotingManager 打包在一起破坏了服务边界，把两个服务耦合在一起。

Decision 2: The Voting Manager Data Access subsystem is not deployed to the mobile device. **Correction:** add a deploy link to the Mobile Device with Local Storage. **Justification:** The Conduct Vote use case stores the voting record on the voter's device, so the device needs this subsystem to save CastVote.

> 决策 2：Voting Manager Data Access 子系统没有部署到手机。纠正：加一条到 Mobile Device with Local Storage 的部署。论证：Conduct Vote 用例要把投票记录存在选民设备上，所以设备需要这个子系统来保存 CastVote。

Decision 3: The Notification Manager Control subsystem is deployed to the mobile device. **Correction:** remove this deployment; it stays on the backend. **Justification:** Notifications are sent by the platform, not by the phone. The phone only needs the voting screen and local storage.

> 决策 3：Notification Manager Control 子系统被部署到手机。纠正：删除这条部署，留在后端。论证：通知由平台发送，不是由手机发出。手机只需要投票界面和本地存储。

易错： 把"没画后端（backend）"当错误（图注说刻意省略（deliberately omitted））；把 Q1(b) 自己提出的改造当本题前提（assumption）。

Q3(d) 数据库写吞吐瓶颈（6 分）

题目大意 • T11 • 6 分（分片（sharding）**方式 2 + 受影响组件及理由**（reason）**4）**

10 万选民要在 **5 分钟**内投完票，数据库**写吞吐**（write throughput）不够；现在是一主库（Primary）加一个只读副本（read-only copy）。提出扩展写吞吐的完整方案（软件和基础设施（infrastructure）的改动和影响），并指出（identify）**至少两个**上线后要密切监控（monitoring）的副作用（side effect）。

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q3(d)**

Plan: A read replica only scales reads, so it does not help. I shard the database by the voter's unique ID (voter ID / NRIC / passport number) with a hash function, so the writes are spread evenly. Each shard has its own primary and replicas. Each voter shard can also be served by its own group of proctors. Votes can be buffered in an event stream and written in batches to absorb the 5-minute peak.

> 方案：只读副本只扩展读，帮不上忙。我按选民唯一 ID（选民 ID / NRIC / 护照号）用哈希函数对数据库分片，让写入均匀分散。每个分片有自己的主库和副本。每个选民分片还可以由自己的一组监考员负责。选票可以先缓冲在事件流里再批量写入，吸收 5 分钟的峰值。

Affected components: VoteController must know the sharding rule to write each vote to the right shard. **PollManager** must record poll data in the right shard. **UserManager** is affected because users are spread over shards. **ObservationManager** must aggregate the results across all shards. Infrastructure: more database instances, a shard routing layer and monitoring per shard.

> 受影响的组件：VoteController 要知道分片规则，把每张票写到正确的分片。PollManager 要把投票数据记录到正确的分片。UserManager 受影响，因为用户分散在不同分片。ObservationManager 要跨所有分片汇总结果。基础设施：更多数据库实例、分片路由层、按分片的监控。

Side effects to monitor: (1) hot or uneven shards; (2) slower cross-shard aggregation, which may delay the 1-minute simplified results; (3) harder checks for "one vote per voter"; (4) complex re-sharding and higher operation cost.

> 要监控的副作用：（1）热点或不均匀的分片；（2）跨分片汇总变慢，可能拖慢 1 分钟内更新的简要结果；（3）"每人一票"更难检查；（4）重新分片复杂、运维成本更高。

易错： 答"再加只读副本"；只写"按 ID 分片"不点名组件和理由（丢 4 分）。

github.com/bibi45c · 免费分享，禁止售卖

PART 2　样卷逐题范文（简单英文 + 中文）

PAST2　MLTTP 公交遥测平台

✕常见误区｜这样答会丢分　PAST2 没有官方答案

得分点（marking points）按 PAST 官方评分的颗粒度（Granularity）**推断**；范文（model answer）是可得分的一种写法，不是唯一答案。主线是"**海量实时更新**"：1 万台车载设备持续上报、5 秒内展示、2 秒内回传理想车速。

案例速读：MLTTP 公交遥测平台

陆路交通管理部门建设多模式遥测平台，先做公交，将来扩展到地铁、出租。第一阶段：平台 + Web 应用 + 乘客 App；第二阶段：历史数据分析、机器学习（Amazon SageMaker）预测、发车调整。

附录	要抄的事实
§4 并发（concurrency）	**100,000 Public Users**、**10,000 Registered Users**、5 System Admins、**10,000 Vehicle Onboard Units**；车队最多 3,000 辆
§4 响应	查看到站时间和载客量 **90% < 3 秒**；变化 **5 秒内**展示给用户；理想车速 **2 秒内**送到车载设备
§4 其他	不预期多租户（multitenancy）；§5 有租户专属密钥（Tenant-Specific Key）、PCI-DSS
§6 层名	Presentation / Control / **Platform Service** / Data Access / **Enterprise Integration**
§7 领域模型（domain model）	ArrivalTimeComputer（Position、Speed、ArrivalTime **各自一个聚合**（aggregate））；DesirableSpeedComputer（DesirableSpeed）；PassengerLoadComputer（PassengerEvent ← PassengerEntrance / PassengerExit、PassengerLoad）；BusNetworkManager（**BusTrip + BusRoute 同一聚合**，BusStop 单独）；UserManager
用例	乘客上下车事件**逐条上报、不批量**
试卷 Q1(a) 题干（question stem）	更新**每 30 秒同步**（synchronous）**一次**（只在该小题给出，附录里没有）

题干难词	意思	题干难词	意思
telemetry	遥测	onboard unit	车载设备
arrival time	到站时间	passenger load	载客量
desirable speed	理想车速	observed location	被关注的地点
runtime memory	运行内存	collocate	放在一起（共置（collocation））
staging environment	预发布环境	accuracy	准确度

Q1(a) 平台成功的两个关键架构方面

题目大意 • T9 • 2 分（每个方面 + 数字理由（reason）1 分）

两个种子（seed）：到站时间、载客量；第一阶段要支持**每分钟 1 万次更新**、90% 用户更新响应 < 3 秒；10 万公众用户、1 万注册用户且在增长；每 30 秒同步（synchronous）一次。**结合种子和交互**（interaction），**解释两个最关键的**（crucial / critical）**架构方面。**

✍ 参考答案｜可照着写，名字和数字换成新案例的　Q1(a)

The seeds are arrival times and passenger loads. They are produced by 10,000 onboard units and used by 100,000 public users and 10,000 registered users.

> 种子是到站时间和载客量，由 1 万台车载设备产生，由 10 万公众用户和 1 万注册用户使用。

(1) Scalability of the viewing services: many users view arrival times at the same time, especially at peak hours, and the number is growing. The view services must be stateless, scale out automatically and read the latest values from an in-memory cache, so that 90% of requests finish within 3 seconds.

> （1）查看服务的可扩展性：很多用户同时查看到站时间，高峰期尤其多，而且人数在增长。查看服务必须无状态、能自动扩容、从内存缓存读最新值，才能让 90% 的请求在 3 秒内完成。

(2) Performance of the real-time update pipeline: 10,000 updates per minute must be synchronised every 30 seconds (Q1(a)), and Section 4 requires changes to reach users within 5 seconds. An event stream partitioned by bus trip lets the services process updates in parallel and in order. If the seeds are old, users will leave the platform.

> （2）实时更新管道的性能：每分钟 1 万次更新要每 30 秒同步一次（Q1(a) 题干），第 4 节还要求变化 5 秒内送到用户。按班次分区的事件流让服务并行且按顺序处理更新。种子过时，用户就会离开平台。

Q1(b) 领域模型：算得慢 / 内存高（9 分）

Q1(b)(i) 题目大意 • T1 • 5 分（推断：原因（cause）2 + 对策（countermeasure）两部分各 1.5 + 缺点（drawback）要写）

ArrivalTimeComputer 根据位置和速度计算最新到站时间。你认为现有领域设计**不利于快速计算。原因 / 对策 / 缺点。**

github.com/bibi45c · 免费分享，禁止售卖

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(b)(i)**

Cause: Position, Speed and ArrivalTime are three separate aggregates, but they are always used together. Each calculation loads and saves three aggregates in three transactions. The calculation also needs BusTrip and BusStop, which are in the BusNetworkManager, so each of the 10,000 updates per minute needs inter-process communication.

> 原因：Position、Speed、ArrivalTime 是三个独立的聚合，但它们总是一起用。每次计算都要在三个事务里加载、保存三个聚合。计算还需要 BusNetworkManager 里的 BusTrip 和 BusStop，所以每分钟 1 万次更新的每一次都要跨进程通信。

Countermeasure: (1) Merge Position, Speed and ArrivalTime into one aggregate per bus trip (e.g. TripTracking), because they change together. (2) Keep a read-only copy of each route's bus stops in the ArrivalTimeComputer, updated by a BusRouteChanged event from the BusNetworkManager. The arrival time is then calculated in one service and one transaction.

> 对策：（1）把 Position、Speed、ArrivalTime 合并成每个班次一个聚合（如 TripTracking），因为它们一起变化。（2）在 ArrivalTimeComputer 里保留每条线路车站的只读副本，由 BusNetworkManager 发出的 BusRouteChanged 事件更新。到站时间就在一个服务、一个事务里算完。

Drawback: The bus-stop data is duplicated and only eventually consistent, and the merged aggregate is larger. This is acceptable because routes and stops rarely change.

> 缺点：车站数据重复，只能最终一致，合并后的聚合也更大。因为线路和车站很少变，可以接受。

Q1(b)(ii) 题目大意 • T1 • 4 分（推断：原因 1.5 + 对策 1.5 + 缺点 1）

BusNetworkManager 管理线路的站点和班次。你担心它的实例**运行内存占用很高。原因 / 对策 / 缺点。**

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(b)(ii)**

Cause: BusRoute and BusTrip are in one aggregate. A route has many trips a day, and the fleet grows to 3,000 buses, so loading one BusRoute loads all its BusTrip objects, including the often-changing passengerCount, into memory.

> 原因：BusRoute 和 BusTrip 在同一个聚合里。每条线路一天班次很多，车队最多 3,000 辆，加载一条 BusRoute 就把它所有的 BusTrip 对象（包括经常变化的 passengerCount）都载入内存。

Countermeasure: Follow "design small aggregates": split it into a BusRoute aggregate and a BusTrip aggregate. BusTrip refers to its route only by ID (busRouteId), so only the trips being updated are loaded. Optionally move passengerCount to PassengerLoad in the PassengerLoadComputer, where it is calculated.

> 对策：按"设计小聚合"拆成 BusRoute 聚合和 BusTrip 聚合。BusTrip 只通过 ID（busRouteId）引用线路，只加载正在更新的班次。可选：把 passengerCount 移到 PassengerLoadComputer 的 PassengerLoad 里，那里才计算它。

Drawback: Listing all trips of a route needs one more query, and changes to both a route and its trips become eventually consistent through domain events.

> 缺点：列出一条线路的所有班次要多查一次；同时改线路和班次的变更要通过领域事件做到最终一致。

易错： 两小问对策方向矛盾——(i) 如果写"把实体（entity）并入 BusNetworkManager"，(ii) 又说它太大，就自相矛盾。

Q1(c) 大批量并发更新时 API 延迟 20–30 秒（6 分）

题目大意 • T3 • 6 分（推断：原因（cause）约 2 + 重构约 2 + 例子约 2）

车载设备调用 `PUT /arrivaltime`（输入班次、线路、位置、速度，输出"已处理 / 未处理"）和 `PUT /passengerload`（输入班次、线路、一个乘客事件）。负载测试（load test）中大批量并发（concurrency）更新时延迟（latency）约 20–30 秒。**解释原因，说明怎样重构 API，并用具体例子展示**（提示：由外而内（outside-in）设计）。

URI	HTTP	Input	Output	说明
`/bus-trips/{trip-id}/positions`	POST	position, speed, timestamp	**202 Accepted**; eventId	上报位置和速度（异步（asynchronous）处理）
`/bus-trips/{trip-id}/passenger-events`	POST	eventType (entrance/exit), timestamp	**202 Accepted**; eventId	上报一次上车或下车
`/bus-trips/{trip-id}/passenger-count`	PUT	passengerCount, timestamp	200 OK	清零人数（真正的"替换"）

github.com/bibi45c · 免费分享，禁止售卖

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(c)**

Reason 1 – synchronous: Before replying "processed", the service recalculates the arrival times for all observed locations and the desirable speed. So thousands of concurrent requests wait in a queue.

> 原因 1（同步）：回复"已处理"之前，服务要重算所有被关注地点的到站时间和理想车速，所以成千上万的并发请求在排队。

Reason 2 – PUT on one shared resource: All devices use PUT on /arrivaltime and /passengerload, so concurrent updates overwrite and block each other, although each update is really a new event.

> 原因 2（PUT 同一个共享资源）：所有设备都对 /arrivaltime 和 /passengerload 用 PUT，并发更新互相覆盖、互相阻塞，而每次更新其实是一个新事件。

Reason 3 – not resource-oriented, extra input: The trip already tells the route, so sending Bus Route again makes every request larger.

> 原因 3（不面向资源、多余输入）：班次已经能确定线路，再传 Bus Route 让每个请求都更大。

Refactoring: Make the API resource-oriented and asynchronous. Each onboard unit POSTs an event under its own bus trip. The API checks it, publishes it to an event stream (Amazon Kinesis) partitioned by trip ID, and returns 202 Accepted at once. ArrivalTimeComputer and PassengerLoadComputer read the partitions in parallel and in order, and push the results to users. Passenger events are still sent one by one, as the case requires.

> 重构：把 API 改成面向资源、异步。每台车载设备在自己的班次下 POST 一个事件。API 校验后发布到按班次 ID 分区的事件流（Amazon Kinesis），立即返回 202 Accepted。ArrivalTimeComputer 和 PassengerLoadComputer 并行且按顺序读取各分区，再把结果推给用户。乘客事件仍按案例要求逐条上报。

Example: Trip 175-0832 sends `POST /bus-trips/175-0832/positions` with body {"position": "1.2966,103.7764", "speed": 32, "timestamp": "08:15:30"}. It gets 202 Accepted within a few milliseconds, and the arrival times at the observed stops are updated within 5 seconds.

> 例子：班次 175-0832 发送 `POST /bus-trips/175-0832/positions`，请求体 {...}。几毫秒内收到 202 Accepted，被关注车站的到站时间在 5 秒内更新。

易错： 只说"加服务器"；没给具体例子（题目要 **illustrate with a concrete example**），例子漏了方法 / URI / 请求体（request body）/ 返回码之一。

Q1(d) 存储最重要的两个质量属性 • Q1(e) 两个云数据库

题目大意 • Q1(d) T7 2 分 • Q1(e) T6 4 分（每个服务 2 分）

(d) 根据第一阶段需求，找出存储 / 数据库最重要的**两个质量属性**（quality attribute），说明为什么比其他重要。(e) 基于这两个属性，提出**两个云存储服务**，说明它们怎样满足。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(d)**

The two most important quality attributes are **scalability of writes** and **performance (low latency)**. Up to 10,000 onboard units keep sending positions, speeds and passenger events, and the data keeps growing, so the store must scale writes horizontally. Changes must reach users within 5 seconds and the desirable speed must reach the bus within 2 seconds, so reads and writes must take milliseconds. They matter more than rich queries, because the data is simple and read by key (trip, stop). If the store is slow, the core interactions fail.

> 最重要的两个质量属性是写入的可扩展性和性能（低延迟）。多达 1 万台车载设备持续发送位置、速度和乘客事件，数据不断增长，所以存储必须能水平扩展写入。变化要 5 秒内到达用户，理想车速要 2 秒内送到车上，所以读写都必须是毫秒级。它们比复杂查询更重要，因为数据简单、按键（班次、车站）读取。存储一慢，核心交互就失败。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(e)**

Amazon DynamoDB – a fully managed key-value database with millisecond performance at any scale. It partitions data by key (trip ID), so write throughput scales out to 10,000 updates per minute (scalability), and each read or write takes milliseconds (performance). It stores positions, speeds, passenger events and arrival times.

> Amazon DynamoDB：全托管的键值数据库，任何规模下都是毫秒级。它按键（班次 ID）分区，写吞吐能扩展到每分钟 1 万次更新（可扩展性），每次读写都是毫秒级（性能）。存位置、速度、乘客事件和到站时间。

Amazon ElastiCache for Redis – a fully managed in-memory store. The latest arrival times and loads for each bus stop are kept in Redis, so 100,000 public users get answers well within 3 seconds (performance), and nodes can be added as users grow (scalability).

> Amazon ElastiCache for Redis：全托管的内存存储。每个车站最新的到站时间和载客量放在 Redis 里，10 万公众用户的查询远低于 3 秒（性能）；用户增长时可以加节点（可扩展性）。

其他可得分：Amazon Timestream（时序数据）、Kinesis + S3（历史数据湖（data lake））、Aurora。

Q1(f) 哪些服务放进同一个容器

题目大意 • T12 • 2 分

服务用容器（Container）部署（deployment），但受许可限制，要把两个以上服务放进同一个容器实例。**提出最适合共置**（collocation）**的服务，并说明好处。**

github.com/bibi45c · 免费分享，禁止售卖

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q1(f)**

I propose to collocate the **ArrivalTimeComputer** and the **DesirableSpeedComputer**. Every position update triggers a new arrival time, and every new arrival time triggers the desirable-speed calculation, which must reach the bus within 2 seconds. So they exchange messages for each of the 10,000 updates per minute, and they have the same load pattern.

> 我建议把 ArrivalTimeComputer 和 DesirableSpeedComputer 放进同一个容器。每次位置更新都触发新的到站时间计算，每个新的到站时间又触发理想车速计算，而理想车速必须 2 秒内送到车上。所以每分钟 1 万次更新中的每一次它们都要交换消息，负载模式也相同。

Collocating them turns the network calls into in-process calls, which cuts latency and network traffic, and they can be scaled together.

> 共置后网络调用变成进程内调用，降低延迟和网络流量，还能一起扩缩容。

另一种可得分：ArrivalTimeComputer + BusNetworkManager（领域模型（domain model）中跨框连线最多）。

Q2(a) 本地部署 → 云原生架构（7 分）

题目大意 • T5 + T10 • 7 分（推断：架构图约 3 + 标出新增 / 删除 / 修改约 2 + 选混合并论证（justification）**约 2）**

乘客 App BusInfo4Me 有交互（interaction）地图、仪表盘（dashboard）、实时到站和载客量、多段行程规划、常用地点。题中给了传统本地部署（deployment） Web 应用架构（application architecture）（硬件负载均衡（Load Balancing）、Web 服务器、应用服务器、数据库）。**重画为适合 MLTTP 的云原生**（Cloud-Native）**架构，标出新增、删除、修改的组件；后端**（backend）**选纯微服务**（microservices）**还是混合，结合改造后的架构论证。**

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q2(a)**

Added: CloudFront + S3 for the web app and maps; Amazon API Gateway (REST, plus WebSocket to push live updates); AWS IoT Core / Kinesis Data Streams to take in data from 10,000 onboard units; an EKS cluster running BusNetworkManager, ArrivalTimeComputer, PassengerLoadComputer, DesirableSpeedComputer, JourneyPlanner and UserManager; an event stream between services; ElastiCache (Redis) for the latest values; DynamoDB and RDS; Lambda + SNS for notifications; Cognito; CloudWatch and CI/CD.

> 新增：CloudFront + S3 承载 Web 应用和地图；Amazon API Gateway（REST，另加 WebSocket 推送实时更新）；AWS IoT Core / Kinesis Data Streams 接入 1 万台车载设备的数据；EKS 集群运行 BusNetworkManager、ArrivalTimeComputer、PassengerLoadComputer、DesirableSpeedComputer、JourneyPlanner、UserManager；服务之间的事件流；ElastiCache（Redis）存最新值；DynamoDB 和 RDS；Lambda + SNS 发通知；Cognito；CloudWatch 和 CI/CD。

Removed: the hardware load balancers and the fixed web and application servers. **Changed:** load balancing becomes Elastic Load Balancing / Ingress; web servers become S3 + CloudFront; application servers become containerised microservices with auto-scaling; the single database with a standby becomes managed Multi-AZ databases per service.

> 删除：硬件负载均衡器和固定的 Web、应用服务器。修改：负载均衡改为 Elastic Load Balancing / Ingress；Web 服务器改为 S3 + CloudFront；应用服务器改为自动扩缩的容器化微服务；单库加备库改为每个服务一个托管的多可用区数据库。

Hybrid, because: interactive features (journey planning, viewing arrival times) run as microservices; telemetry is processed by event-stream consumers, in order per trip, in near real time; notifications are serverless functions that scale with spikes and cost nothing when idle; Phase 2 history analysis runs as scheduled jobs. Each workload uses the most suitable and cost-effective style.

> 选混合，因为：交互功能（行程规划、查看到站时间）用微服务；遥测数据由事件流消费者按班次有序、近实时处理；通知用 Serverless 函数，随突发量扩展、空闲不花钱；第二阶段历史分析用定时任务。每种负载都用最合适、最省钱的风格。

Q2(b) 虚拟机还是容器

题目大意 • T10 • 3 分

根据 §4 的扩展要求和上述功能，托管用**虚拟机**（Virtual Machine (VM)）**还是容器**（Container）？简要说明扩展设计决策。

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q2(b)**

I choose **containers** on Kubernetes (Amazon EKS). (1) The load changes fast at peak hours (100,000 public and 10,000 registered users). Containers start in seconds and the HPA adds Pods on CPU or request rate, while VMs take minutes. (2) Each microservice scales on its own driver: the view services on the number of users; ArrivalTimeComputer and PassengerLoadComputer on the update rate from 10,000 onboard units (stream lag). (3) Containers are lighter than VMs, so the cost is lower; the Cluster Autoscaler adds nodes across availability zones when needed.

> 我选跑在 Kubernetes（Amazon EKS）上的容器。（1）高峰期负载变化快（10 万公众用户、1 万注册用户）。容器秒级启动，HPA 按 CPU 或请求速率加 Pod，虚拟机要几分钟。（2）每个微服务按自己的指标扩展：查看服务按用户数；ArrivalTimeComputer 和 PassengerLoadComputer 按 1 万台车载设备的更新速率（事件流积压）。（3）容器比虚拟机轻，成本更低；需要时 Cluster Autoscaler 跨可用区加节点。

第 3 分常给在"扩展决策"上：说清每类服务按什么指标（metric）扩。

Q2(c) 托管 ML 预测到站时间

题目大意 • T6 • 3 分

用某云厂商的全托管服务（managed service）做高精度到站预测（时间序列 + 其他变量）。为 Update Arrival Times at Observed Location 用例**画架构图，标出推荐的云产品，并对每个组件给出替代选择。**

github.com/bibi45c · 免费分享，禁止售卖

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q2(c)**

Kinesis Data Streams takes in positions and speeds → Lambda cleans them and stores history in S3 with extra data (weather, traffic, time of day) → Amazon SageMaker trains a time-series model and hosts it as a real-time endpoint → for each update, Lambda calls the endpoint to predict arrival times and writes them to DynamoDB → API Gateway serves them to the apps.

> Kinesis Data Streams 接入位置和速度 → Lambda 清洗后连同额外数据（天气、交通、时段）把历史存进 S3 → Amazon SageMaker 训练时间序列模型并部署为实时推理端点 → 每次更新时 Lambda 调用端点预测到站时间，写入 DynamoDB → API Gateway 提供给应用。

Alternatives: ingestion – Amazon MSK, Azure Event Hubs, Google Pub/Sub; storage – Azure Data Lake Storage, Google Cloud Storage (BigQuery for analysis); ML – Azure Machine Learning, Google Vertex AI; serving – containers on EKS instead of Lambda.

> 替代：接入可用 Amazon MSK、Azure Event Hubs、Google Pub/Sub；存储可用 Azure Data Lake Storage、Google Cloud Storage（分析用 BigQuery）；机器学习可用 Azure Machine Learning、Google Vertex AI；推理可用 EKS 上的容器代替 Lambda。

Q3(a) 第二阶段逻辑架构：找一个错 + 补全打包（6 分）

题目大意 • T4 • 6 分（推断：错误决策 3 + 补全打包（packaging） 3）

参照 §2 第二阶段需求、§6 初始架构和 §8 Figure 6（新手画的不完整逻辑架构（Logical Architecture），没画部署（deployment）节点（node））：**找出一个不理想的**（undesirable）**决策并纠正**（correction）**、论证**（justification）**；再把尚未打包的功能元素打包进子系统**（subsystem）**。**

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q3(a)**

Undesirable decision: DesirableSpeed is packaged into the Arrival Time Data Access subsystem. **Correction:** remove this manifest and package DesirableSpeed into a new Desirable Speed Data Access subsystem. **Justification:** In the domain model, DesirableSpeed belongs to the DesirableSpeedComputer service, a different bounded context. Packaging it with arrival-time data couples the two services, so they cannot be deployed and scaled independently.

> 不理想的决策：DesirableSpeed 被打包进 Arrival Time Data Access 子系统。纠正：删掉这条 manifest，把 DesirableSpeed 打包进新的 Desirable Speed Data Access 子系统。论证：领域模型中 DesirableSpeed 属于 DesirableSpeedComputer 服务，是另一个限界上下文。和到站时间数据打包在一起把两个服务耦合，无法独立部署和扩展。

补全的子系统	打包的元素
Analytics Presentation	ConfigureAnalyticsScreen（boundary）
Analytics Control	AnalyticsConfigurationController、AnalyticsController
Analytics Data Access	AnalyticsConfiguration、AnalyticsReport
Bus Trip Optimisation Enterprise Integration	BusTripPlannerInterface（对接外部 Bus Trip Planner 的边界（Boundary）；§6 的层名是 Enterprise Integration）
Bus Trip Optimisation Control	BusTripOptimiser、TripEventComputer
Bus Trip Optimisation Data Access	BusTripChangeRequest、TripEvent
Passenger Load Data Access（补）	PassengerEntrance、PassengerExit

另一个可得分的决策：AnalyticsController 直接依赖别的服务的实体（entity） BusRoute（跨限界上下文（bounded context）），应改为经 BusNetworkManager 的 API 或事件读取。

Q3(b) 预测准确度指标（6 分）

题目大意 • T8 • 6 分（推断：定义与计算约 2 + 组件 / 频率 / 保存 / 位置约 2 + 两个依赖指标（dependent metric）约 2）

利益相关者（stakeholder）希望预测值逐步变准。在第二阶段历史分析中，**提出一个衡量并跟踪准确度的指标**（metric）：直接采集（capture）就写组件、频率、保存时长（retention period）、位置；要计算就写计算方法、组件、频率、保存、位置，并对**最多两个依赖指标**做同样说明。

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q3(b)**

Metric – MAE of predicted arrival times: MAE = the average of |predicted − actual arrival time| (in seconds) for all predictions of a bus service in a period. A falling MAE shows that the prediction is getting better.

> 指标：预测到站时间的 MAE。MAE = 一段时间内某条线路所有预测的 |预测 − 实际到站时间|（秒）的平均值。MAE 下降说明预测在变准。

Calculation: a scheduled analytics job (run by the AnalyticsController) matches each prediction with the actual arrival of the same trip at the same stop, and calculates MAE every day, per bus service and hour of day. **Retention and location:** daily MAE values are kept for 3 years in Amazon Timestream (or Redshift) to track long-term improvement.

> 计算：定时分析任务（由 AnalyticsController 执行）把每个预测和同一班次在同一车站的实际到站配对，每天按线路和时段计算 MAE。保存时长和位置：每日 MAE 在 Amazon Timestream（或 Redshift）保存 3 年，用来跟踪长期改进。

Dependent metric 1 – predicted arrival time: captured by the ArrivalTimeComputer every time it calculates a new arrival time (trip, stop, predicted time, timestamp); kept for 90 days in the data lake (Amazon S3).

> 依赖指标 1：预测到站时间。ArrivalTimeComputer 每次算出新的到站时间时记录（班次、车站、预测时间、时刻）；在数据湖（Amazon S3）保存 90 天。

Dependent metric 2 – actual arrival time: captured when the position sent by the onboard unit reaches the stop; kept for 90 days in the same data lake, so both can be joined by trip and stop.

> 依赖指标 2：实际到站时间。车载设备上报的位置到达车站时记录；在同一个数据湖保存 90 天，两者按班次和车站关联。

github.com/bibi45c · 免费分享，禁止售卖

◆ 要点 | 必背结论 **PAST2 比 PAST 多练到的**

1. T1 还要看**聚合**（aggregate）**（红圈）大小**：太碎 → 慢；太大 → 吃内存。
2. T3 要会诊断 API 延迟（latency）：同步（synchronous）、PUT 覆盖、冗余（redundancy）输入 → **异步**（asynchronous） **+ POST 事件 + 202。**
3. 质量属性（quality attribute）题和云服务题常连着出：先定两个属性，再让每个产品对上一个属性。
4. 指标题写全字段；计算型还要写**依赖指标。**
5. 两题之间不要自相矛盾。

github.com/bibi45c · 免费分享，禁止售卖

PART 2　样卷逐题范文（简单英文 + 中文）

PAST3　ONE Health 医疗健身平台

✕常见误区｜这样答会丢分　**PAST3 只有 Section A（34 分），没有官方答案**

得分点（marking points）为**推断。**附录**没有 §7 领域模型**（domain model），Q1(b) 要自己从 §5 用例流程建模。它最考"从零建模"：画类图（class diagram）、找共享服务（shared service）、设计 Level 3 API、改部署（deployment）图、映射到 AWS。

案例速读：ONE Health 医疗与健身平台

智慧城市项目：把大医院药房外包整合并扩展到小医院和诊所，让供应商和病人直接交互（interaction）；同时鼓励大众健身。四个应用：**PharmaMed**（订药）、**PharmaEquip**（订医疗器材）、**WeFitRun**（跑步）、**WeFitYoga**（瑜伽）。（附录开头说 two pilot applications，第 1 节写了四个，原文不一致，按四个答，必要时注明。）

附录	要抄的事实
§2 扩展性	要有公共服务和库；近期以"**药品 / 器材配送**"和"**新运动类型**"检验扩展性；任何电脑和手机都能访问
§3 性能（performance）	数据增长时保持响应；必要时按**医疗机构、供应商、服务类型分片**（sharding）；交互操作（如查配送状态）**90% < 1 秒**
§3 多租户（multitenancy）	**必须支持多租户**，租户（tenant）（医疗机构、供应商、子公司）之间数据和应用隔离（isolation）
§4 安全（security）	DMZ 反向代理（reverse proxy）；个人信息加密（encryption）、GDPR；**租户专属密钥**（Tenant-Specific Key）加密并自动轮换；**PCI-DSS：支付只在尽量少的子系统**（subsystem）**里处理**
§5 用例	Upload Prescriptions（医生开方进 **NHS**，医院管理员手工复制到平台；**NHS 不属于平台，没有集成计划**）；Order Medicines（按处方下单 → 付款 → 返回订单号和预计送达）；Order Equipment（**不需要处方**）；Track Delivery Orders；Make Payment（经 Payment Gateway）；Generate Non-Personal Analytics
WeFit	跟踪跑步 / 瑜伽、挑战、教练计划 / 指导课程、徽章、分享社交媒体、同步（synchronous） Google Fit

题干难词	意思	题干难词	意思
prescription	处方	supplier	供应商
healthcare provider	医疗机构	multitenancy / tenant	多租户 / 租户
reusable asset	可复用资产	expose	对外开放
communication affinity	通信亲和度	physical detailed deployment	物理详细部署

Q1(a) 平台成功的一个关键架构方面

题目大意 • T9 • 2 分

两个种子（seed）：处方药、器材；交互（interaction）：上传处方、订购药品 / 器材、跟踪订单。**结合种子和交互，解释一个最关键的**（crucial / critical）**架构方面。**

✍ 参考答案｜可照着写，名字和数字换成新案例的　**Q1(a)**

The seeds are prescriptions and equipment. They are produced by healthcare providers and suppliers, and used by patients who upload prescriptions, order and track their orders.

> 种子是处方和器材，由医疗机构和供应商产生，病人通过上传处方、下单和跟踪订单来使用。

The most crucial aspect is multitenancy with shared, open services. Many hospitals, clinics and suppliers must join as tenants, with their data and applications isolated (tenant-specific keys, data partitioned by tenant). At the same time they share common services such as prescription, catalogue, order, payment and delivery tracking through APIs. New producers can then join easily and the ecosystem grows. Without isolation, providers would not trust the platform with patient data.

> 最关键的方面是多租户加共享、开放的服务。许多医院、诊所和供应商要作为租户加入，数据和应用相互隔离（租户专属密钥、按租户分区数据）。同时它们通过 API 共享处方、目录、订单、支付、配送跟踪等通用服务。新的生产者就能轻松加入，生态得以扩大。没有隔离，医疗机构不会放心把病人数据交给平台。

也可以选可扩展性（scalability）（引用 90% < 1 秒和分片（sharding）要求），只要论证（justification）。

Q1(b) 用 DDD 画类图 + 推荐共享服务（9 分）

题目大意 • T2 • (i) 类图（class diagram）**约 5 分 • (ii) 共享服务**（shared service）**约 4 分**

(i) 分析 §5 的重要用例流程，用 DDD 方法找出领域对象及其关系，**画成 UML 类图。**(ii) 平台要有能跨应用共享、并开放给外部使用的服务，**找出推荐的共享服务。**

服务（限界上下文）	实体（主要属性）
UserManager	User（userId, name）← Patient（NRIC, address）、HealthcareProviderAdministrator、SupplierUser

github.com/bibi45c · 免费分享，禁止售卖

服务（限界上下文）	实体（主要属性）
ProviderManager	HealthcareProvider、PreferredSupplier（order of preference）
PrescriptionManager	Prescription（id, date）、PrescribedMedicine（dosage, quantity）
CatalogueManager	Product（id, name, price）← Medicine、Equipment；Supplier ← DrugSupplier、EquipmentSupplier
OrderManager	Order（id, date, status, deliveryAddress）、OrderItem（quantity, price）
PaymentManager	Payment（id, amount, option, status）— **PCI-DSS 只在这里**
DeliveryManager	DeliveryOrder（no, expectedDateTime, status）
AnalyticsManager	Counter（type, period, value）

关系与多重性（multiplicity）： Patient places Order（1 — 0..）；**Order ◆ OrderItem（组合**（composition）**，1 — 1..）**；OrderItem → Product（0.. **— 1）；药品 Order based on Prescription（0.. — 0..1**，器材订单不需要处方）；Prescription ◆ PrescribedMedicine；Prescription is for Patient；Order paid by Payment（1 — 1）；Order has DeliveryOrder（1 — 1..*）；Supplier supplies Product；Medicine / Equipment — ▷ Product（继承（inheritance））。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(b)(i) 文字说明（配类图）**

I read each use case and took the nouns as candidate entities and the verbs as relationships. Attributes such as dates and quantities, and external systems such as the NHS and the Payment Gateway, are not entities. Medicine and Equipment are kinds of Product (inheritance); an Order contains one or more OrderItems (composition). An equipment order needs no prescription, so Order to Prescription is 0..1. The entities are grouped into services by functional cohesion, as shown in the boxes.

> 我逐条读用例，把名词作为候选实体、动词作为关系。日期、数量这类属性，以及 NHS、Payment Gateway 这类外部系统，不是实体。Medicine 和 Equipment 是 Product 的子类（继承）；一个 Order 包含一个或多个 OrderItem（组合）。器材订单不需要处方，所以 Order 到 Prescription 是 0..1。实体按功能内聚分到各服务框里。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(b)(ii) 共享服务（每个写"给谁用、做什么"）**

(1) User & Identity – registration and login for all four apps. (2) Product Catalogue – medicines and equipment (PharmaMed, PharmaEquip). (3) Order. (4) Payment – the only PCI-DSS subsystem. (5) Delivery Tracking – used by both pharmacy apps and named in Section 2. (6) Notification. (7) Non-Personal Analytics – all apps. (8) Challenge Management – distance and period challenges (WeFitRun, WeFitYoga). (9) Workout Tracking – runs, yoga and future workout types. (10) Gamification – dashboards and badges. (11) Social Sharing. (12) Health Platform Sync – Google Fit. (13) Guided Programme – run coaching and yoga programmes.

> （1）用户与身份：四个应用的注册登录。（2）商品目录：药品和器材。（3）订单。（4）支付：唯一涉及 PCI-DSS 的子系统。（5）配送跟踪：两个药房应用都用，第 2 节点名。（6）通知。（7）非个人统计：所有应用。（8）挑战管理：距离挑战和周期挑战。（9）运动记录：跑步、瑜伽和未来的新运动。（10）游戏化：仪表盘和徽章。（11）社交分享。（12）健康平台同步：Google Fit。（13）指导课程：跑步教练计划和瑜伽课程。

They are shared because more than one application needs them, so building them once makes future services faster to build.

> 之所以共享，是因为不止一个应用需要它们，只建一次能让以后的服务更快开发出来。

易错： 把 NHS 画成实体（entity）；器材订单也必须关联（Association）处方；只画实体不分服务框。

Q1(c) 开放哪两个 API • 设计 Level 3 API（6 分）

题目大意 • (i) T9 约 2 分 • (ii) T3 约 4 分

(i) 从共享服务（shared service）里选**两个作为 API 开放给两类客户**（生产者（producer）/ 消费者（consumer）），点名具体客户并说明理由（reason）。(ii) 从中选一个，按 **RMM Level 3** 设计 API：服务 URI、HTTP 动作、输入输出定义，并解释每个操作的用途。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(c)(i)**

Prescription Service API – for healthcare providers (producers): today an administrator copies each prescription by hand. With an API, the hospitals' own systems can upload prescriptions directly. This cuts manual work and errors and brings more seeds onto the platform.

> 处方服务 API，给医疗机构（生产者）：现在由管理员手工复制每张处方。有了 API，医院自己的系统能直接上传处方，减少人工和差错，也把更多种子带进平台。

Delivery Tracking Service API – for drug and equipment suppliers: their logistics systems receive delivery orders and update the status automatically, so patients always see up-to-date status, status checks still return within 1 second for 90% of requests, and new suppliers can join easily.

> 配送跟踪服务 API，给药品和器材供应商：他们的物流系统自动接收配送单、更新状态，病人看到的状态始终是最新的，查询状态 90% 在 1 秒内返回，新供应商也容易接入。

理由**不要写"与 NHS 集成"**（附录说没有这个计划），写"医疗机构自己的系统"。

URI	HTTP	Input	Output（含 HATEOAS 链接）	Purpose
`/delivery-orders?supplierId={id}&status={s}`	GET	query parameters	200 OK; list; links: self, next, each order	List a supplier's delivery orders
`/delivery-orders/{no}`	GET	–	200 OK; status, expected time, address; links: self, update-status, order	View one delivery order
`/delivery-orders/{no}/status`	PUT	status, timestamp	200 OK; links: self, delivery-order	Update the delivery status

github.com/bibi45c · 免费分享，禁止售卖

URI	HTTP	Input	Output（含 HATEOAS 链接）	Purpose
`/delivery-orders/{no}/events`	POST	event (e.g. DELAYED), reason, new expected time	**201 Created**; links: self, delivery-order	Record a delivery event

`PUT /delivery-orders/{no}/status` 是整体替换 status 这个子资源（Resource），所以用 PUT（幂等（idempotent））。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(c)(ii) 收尾两句**

The API uses plural nouns, the correct HTTP verbs and status codes, and every response has hypermedia links to the next possible actions, so it is at RMM Level 3.

> 这个 API 用复数名词、正确的 HTTP 动词和状态码，每个响应都带指向下一步操作的超媒体链接，所以达到 RMM Level 3。

It is versioned with a custom header (Accept-version) and protected with OAuth 2.0 for each supplier tenant.

> 它用自定义请求头（Accept-version）做版本管理，并按供应商租户用 OAuth 2.0 保护。

Q1(d) 通信亲和度高的服务共置

题目大意 • T12 • 2 分

可复用服务（Reusable Service）用容器（Container）部署（deployment）；决策：通信亲和度（communication affinity）高（一段时间内交换的消息数量和大小大）的服务应放进同一个容器。**提出两个或以上最适合共享容器的服务并论证**（justification）。

✍ 参考答案｜可照着写，名字和数字换成新案例的 **Q1(d)**

I propose to collocate the **Workout Tracking Service** and the **Gamification Service**. Every run or yoga session sends detailed workout data (route, distance, time, laps) to Gamification to update dashboards and give badges. So these two exchange the largest number and size of messages.

> 我建议把运动记录服务和游戏化服务放进同一个容器。每次跑步或瑜伽都把详细运动数据（路线、距离、时长、分段）发给游戏化服务，更新仪表盘、发徽章。所以这两个服务交换的消息数量和大小最大。

Collocating them turns the calls into in-process calls, which cuts latency and network traffic. I would not put Payment with Order, because PCI-DSS requires payment handling to stay in as few subsystems as possible.

> 共置后调用变成进程内调用，降低延迟和网络流量。我不会把支付和订单放在一起，因为 PCI-DSS 要求支付处理只在尽量少的子系统里。

另一个可得分：Order + Delivery Tracking（每个订单都产生配送单，跟踪时都要查订单）。

Q2(a) 逻辑详细部署图：找三个错（6 分）

题目大意 • T4 • 6 分（每个：识别（identify）**+ 纠正**（correction）**+ 论证**（justification）**，共三个）**

分析 WeFitRun 和 WeFitYoga 的逻辑详细部署（deployment）图，**找出三个不理想的**（undesirable）**决策**，逐个纠正并论证。 图上实际：只有一个 Internal Zone 1，里面 Application、Platform、Data Store、Integration 四个节点（node）；没有 DMZ，也没有用户设备。Application 节点：WeFitRun Presentation、WeFitRun Control、WeFitYoga Presentation、Yoga Guided Programs Service。Platform 节点：Yoga Freestyles、Workout Tracking、Workout Gamification、Social Sharing、Health Tracking Sync。一个共用的 Data Subsystem；一个 Integration Subsystem。

扫四个维度： 分层完整？（WeFitYoga 没有 Control）· 应用与平台分清？（只给瑜伽用的放在 Platform，可共享的反而放在 Application）· 外部集成走集成层？（同步（synchronous） Google Fit 的放在 Platform）· 有没有 DMZ?

github.com/bibi45c · 免费分享，禁止售卖

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q2(a)**

Decision 1: There is no WeFitYoga Control subsystem. **Correction:** add one and deploy it to the Application node. **Justification:** The layering requires Presentation → Control. Without it, WeFitYoga Presentation must call platform services directly or reuse WeFitRun's control, which couples the two apps.

> 决策 1：没有 WeFitYoga Control 子系统。纠正：加上它，部署到 Application 节点。论证：分层要求 Presentation → Control。没有它，WeFitYoga Presentation 只能直接调平台服务或借用 WeFitRun 的控制层，把两个应用耦合在一起。

Decision 2: The yoga-only Yoga Freestyles Service is on the Platform node, while the shareable guided programmes are on the Application node. **Correction:** move freestyle guidance into WeFitYoga's own subsystems on the Application node, and make guided programmes (with WeFitRun's coaching plans) a shared Guided Programme Service on the Platform node; also add the missing shared Challenge Management Service there. **Justification:** The Platform node should only host services that several apps reuse, so that new workout types can be added quickly (Section 2).

> 决策 2：只给瑜伽用的 Yoga Freestyles Service 放在 Platform 节点，而可共享的指导课程却在 Application 节点。纠正：把自由练习指导移进 WeFitYoga 自己在 Application 节点上的子系统；把指导课程（连同 WeFitRun 的教练计划）做成 Platform 节点上共享的 Guided Programme Service；再在 Platform 上补上缺失的共享 Challenge Management Service。论证：Platform 节点只应放多个应用都复用的服务，这样新的运动类型能快速加入（第 2 节）。

Decision 3: Health Tracking Sync, which talks to the external Google Fit, runs on the Platform node, and everything is in one internal zone with no DMZ. **Correction:** add a DMZ location that only holds the reverse proxy, through which the apps come in; move Health Tracking Sync behind the Integration subsystem on the Integration node in the **internal zone**, which calls Google Fit through an outbound proxy / NAT. **Justification:** Section 4 requires the DMZ reverse-proxy practice. Outside traffic should not reach the internal zone directly, and an integration tier protects the platform from changes in external APIs.

> 决策 3：和外部 Google Fit 通信的 Health Tracking Sync 运行在 Platform 节点，而且所有东西都在一个内部区，没有 DMZ。纠正：新增 DMZ 位置，只放反向代理，应用经它进入；Health Tracking Sync 改为经 Integration 子系统，部署在**内部区**的 Integration 节点上，经出站代理 / NAT 访问 Google Fit。论证：第 4 节要求 DMZ 反向代理的做法。外部流量不应直接进内部区，集成层还能让平台不受外部 API 变化的影响。

其他可选：所有服务共用一个 Data Subsystem（不利于分片（sharding）和租户（tenant）隔离（isolation））；Presentation 部署在内部服务器而不是用户的手机 / 浏览器（WeFitRun 要用 GPS）。

Q2(b) AWS 物理部署图 + 三个架构决策（9 分）

题目大意 • T6 • 9 分（推断：图约 3–4 + 决策约 5–6）

基于 Q2(a) 改正后的图，**画物理详细部署**（deployment）**图**，用题目列出的部分 AWS 服务（API Gateway、ElastiCache、EMR、Lambda、RDS、Cognito、Elastic Beanstalk、Redis、DynamoDB、S3、CloudFront）说明云上部署，并解释**至少三个**导出这个物理架构（Physical Architecture）的重要决策。

▶ 案例演练 | 演示怎么套用 **图（画成框图）**

用户 → Route 53 → **CloudFront** → **S3**（Web 前端（front end））· **API Gateway + WAF + Cognito**（VPC 外的边缘入口，起 DMZ 作用）→ VPC，**两个可用区**（availability zone），私有子网（private subnet）：**EKS 或 Elastic Beanstalk**（应用控制 + 共享平台服务，自动扩缩（auto-scaling））· **Lambda**（游戏化、社交分享、健康同步（synchronous））· **RDS Multi-AZ**（用户、挑战）| **DynamoDB**（运动轨迹）| **ElastiCache Redis**（排行榜）| **S3**（媒体）· 集成子网 → NAT 网关（gateway） → Google Fit / 社交媒体 · **EMR** 历史分析。

✍ 参考答案 | 可照着写，名字和数字换成新案例的 **Q2(b)**

Decision 1 – API Gateway and Cognito at the edge: all app requests go through Amazon API Gateway in front of the private subnets, and Amazon Cognito handles sign-up and sign-in. This follows the DMZ / reverse-proxy rule in Section 4 and scales to millions of users.

> 决策 1：边缘用 API Gateway 和 Cognito。所有应用请求先经过私有子网前面的 Amazon API Gateway，Amazon Cognito 负责注册登录。这符合第 4 节的 DMZ / 反向代理要求，能扩展到数百万用户。

Decision 2 – Containers for always-on services, Lambda for event-driven ones: Workout Tracking, Challenge Management and the app control subsystems run all the time, so they run as containers (EKS or Elastic Beanstalk) in two availability zones with auto-scaling. Gamification, Social Sharing and Health Sync start when a workout ends, so they are Lambda functions that scale by themselves and cost nothing when idle.

> 决策 2：常驻服务用容器，事件驱动的用 Lambda。运动记录、挑战管理和应用控制子系统一直运行，所以以容器（EKS 或 Elastic Beanstalk）跑在两个可用区并自动扩缩。游戏化、社交分享、健康同步在一次运动结束时启动，所以做成 Lambda 函数，自动扩展、空闲不花钱。

Decision 3 – Polyglot persistence: large volumes of workout tracks go to DynamoDB for fast writes at any scale; users, challenges and programmes go to Amazon RDS (Multi-AZ) for transactions; leaderboards are cached in ElastiCache (Redis); maps and media go to S3. Data is partitioned by user and tenant for growth and isolation.

> 决策 3：多语言持久化。大量运动轨迹存 DynamoDB，任何规模都能快速写入；用户、挑战和课程存 Amazon RDS（多可用区）支持事务；排行榜缓存在 ElastiCache（Redis）；路线图和媒体存 S3。数据按用户和租户分区，便于增长和隔离。

Decision 4 – Delivery and analytics: the web front end is served from S3 through CloudFront for low latency, and EMR processes historical workout data for dashboards. **Decision 5 – Integration:** Google Fit and social-media sync run in an integration subnet and call out through a NAT gateway, which isolates the platform from external systems.

> 决策 4：内容分发与分析。Web 前端从 S3 经 CloudFront 分发，降低延迟；EMR 处理历史运动数据生成仪表盘。决策 5：集成。Google Fit 和社交媒体同步运行在集成子网，经 NAT 网关访问外部，把平台和外部系统隔离开。

github.com/bibi45c · 免费分享，禁止售卖

PART 3 英文写作工具箱

英文写作工具箱：规则 • 万能句型 • 中式英语改错

写英文答案的 10 条规则

#	规则	例
1	**一句只说一件事**，主语 + 动词 + 宾语	**The PollManager holds Poll.**
2	因果用 **because / so** 连，不要绕	**X is in A and Y is in B, so every request crosses two services.**
3	**用标签开头**，阅卷人一眼找到得分点（marking points）	**Cause: ... Countermeasure: ... Drawback: ...**
4	服务名、实体（entity）名、用例名、层名**照抄附录**，不翻译、不改大小写	Voting Manager Data Access、CastVote
5	**数字照抄**，带单位和百分位（percentile）	**90% of requests within 3 seconds**
6	一律用**一般现在时**；建议用 **I propose / should**	**The service sends ...; it should be moved ...**
7	否定放在 will / can / should **后面**、be **前面**：**will not be**，不写 **will be not**；单独的 be 动词，not 放后面：**is not**	**The result will not be trustworthy.**
8	不会的词**换简单说法**：**inter-process communication** 写不出就写 **calls between two services over the network**	—
9	题目没给的条件，用 **Assume** 一句写出前提（assumption），再答	**Assume the page only displays data.**
10	写不完整句子也比空白好：**关键词 + 箭头**也能拿部分分	**Cause: entities in 2 services → network calls → slow.**

答案骨架标签（直接抄）

标签	用在哪	标签	用在哪
Position: I agree / I disagree.	Do you agree 题	**Reason 1: / Reason 2:**	二选一（either-or decision）、质量属性（quality attribute）
Cause:	T1、T3、T11 原因（cause）	**Decision 1:**	T4 找错（error finding）、T6 决策
Countermeasure:	T1 对策（countermeasure）	**Correction:**	T4 纠正（correction）
Justification:	T4 论证（justification）、T6 理由（reason）	**Drawback: / Trade-off:**	缺点（drawback）、代价（trade-off）
Refactoring:	T3 重构	**Example:**	T3 举例
Assumption:	题干（question stem）不清时	**Side effects to monitor:**	T11 分片（sharding）
Metric: / Dependent metric 1:	T8	**Stateful service:**	T5 (iii)

万能句型（按功能查）

❝ 可用句式 | ___ 处填案例内容 **表态 • 引用案例**

I disagree with this proposal, for two reasons.
我不同意这个提议，理由有两个。

I agree, because _____.
我同意，因为 _____。

According to Section 4 of the case study, the platform must support _____ concurrent _____, with _____ % of requests completed within _____ seconds.
按案例第 4 节，平台必须支持 _____ 个并发 _____，_____ % 的请求在 _____ 秒内完成。

The use case " _____ " requires _____ to be stored on the device.
用例" _____ "要求 _____ 保存在设备上。

In the domain model (Figure _____), _____ is in the _____ service, while _____ is in the _____ service.
在领域模型（图 _____）中，_____ 在 _____ 服务里，而 _____ 在 _____ 服务里。

github.com/bibi45c · 免费分享，禁止售卖

❝ 可用句式 | ___ 处填案例内容 **原因 • 机制**

The most likely cause is that _____.
| 最可能的原因是 _____。

Because _____, every request has to _____, which is much slower than _____.
| 因为 _____，每个请求都要 _____，这比 _____ 慢得多。

As a result, requests queue up at peak time and the response time grows.
| 结果高峰时请求排队，响应时间变长。

This couples the two services, so a change in one forces a change in the other.
| 这把两个服务耦合在一起，一个改了另一个也要跟着改。

❝ 可用句式 | ___ 处填案例内容 **措施 • 理由** (reason)

I propose to move _____ from _____ to _____.
| 我建议把 _____ 从 _____ 移到 _____。

_____ should be split into _____ and _____.
| _____ 应该拆成 _____ 和 _____。

This works because _____; as a result, _____ can be done within _____ seconds.
| 这样有效是因为 _____；结果 _____ 能在 _____ 秒内完成。

This meets the requirement in Section 4 that _____.
| 这满足第 4 节"_____"的要求。

Then _____ can scale independently, without affecting _____.
| 这样 _____ 可以独立扩展，不影响 _____。

❝ 可用句式 | ___ 处填案例内容 **代价 • 假设** (assumption) **• 比较**

The trade-off is that _____. It can be reduced by _____.
| 代价是 _____。可以用 _____ 来减轻。

There is no major drawback; in fact, _____ can now scale independently.
| 没有大的缺点；实际上 _____ 现在还能独立扩展。

Assume _____. (If _____, then I would _____ instead.)
| 假设 _____。（如果 _____，那我会改为 _____。）

I would choose _____ instead of _____. Firstly, _____. Secondly, _____.
| 我会选 _____ 而不是 _____。第一，_____。第二，_____。

_____ is better for _____, while _____ is better for _____.
| _____ 更适合 _____，而 _____ 更适合 _____。

❝ 可用句式 | ___ 处填案例内容 **画图 • 举例 • 收尾**

Only the changed part of the diagram is drawn below. New elements are marked "new" and removed links are crossed out.
| 下面只画图中改动的部分。新元素标 "new"，删除的连线打叉。

For example, _____ sends _____ and receives _____.
| 例如，_____ 发送 _____，收到 _____。

The diagram shows _____; the numbers on the links show the order of the calls.
| 图中展示了 _____；连线上的数字表示调用顺序。

Therefore, both requirements in Section 4 (_____ and _____) are met.
| 所以第 4 节的两个要求（_____ 和 _____）都满足了。

中式英语改错表（常见错误）

✗ 别这样写	✓ 这样写	说明
everyone only have a ticket	each voter can cast only one **vote**	ticket 是门票；主谓一致
the vote has his own time window	the **voting period** is fixed	vote 是一张票，不用 his
100K people to use for one moment	**100,000 concurrent** voters	并发 (concurrency) = concurrent
the result will be not reliable	the result will **not be** trustworthy	not 放在 be 前
more faster / more easier	faster / easier	已经是比较级的词前不加 more（长词照常用 more scalable）
The reason is because ...	The reason is **that** ... / **Because** ...	二选一 (either-or decision)
It can make the system more good	It **improves** _____	用具体动词
depend the data	**depend on** the data	固定搭配
discuss about	**discuss**	不加 about
according to my opinion	**in my view** / I think	—

github.com/bibi45c · 免费分享，禁止售卖

✗ 别这样写	✓ 这样写	说明
the service is slowly	the service is **slow**	形容词作表语
deploy in the phone	deploy **to** the mobile device	deploy … to …
the information of the order	the **order details**	少用 of 堆砌
can not	**cannot**	一个词
use cache to solve	**add a cache** to … / **cache** _____ in Redis	加冠词或用动词
营销服务器 marketing server	marketing **service**	服务 ≠ 服务器

常用动词（答题够用的 30 个）

动词	意思	例
move _____ to _____	移到	move Observer to the PollManager
split _____ into _____	拆成	split the aggregate into BusRoute and BusTrip
merge _____ into _____	合并	merge the three aggregates into one
keep _____ in _____	保留在	keep a read-only copy in _____
package _____ into _____	打包（packaging）进	package it into the _____ Data Access subsystem
deploy _____ to _____	部署（deployment）到	deploy it to the mobile device
remove / add	删除 / 增加	remove this deployment; add a deploy link
store / save	存	store the state in Redis
scale out / scale in	扩容 / 缩容	scale out on CPU
partition / shard _____ by _____	按 _____ 分区（partition）/ 分片（sharding）	shard the database by voter ID
replicate	复制	replicate data to a read replica
cache	缓存	cache the latest results in Redis
publish / consume	发布 / 消费	publish a VoteCast event; consume it
expose _____ to _____	开放给	expose the API to suppliers
route _____ to _____	路由到	route /votes to voting-svc
isolate	隔离（isolation）	isolate tenant data
aggregate	汇总（aggregation）	aggregate results across shards
buffer / absorb the peak	缓冲 / 吸收峰值	buffer votes in a stream
recompute / calculate	重算 / 计算	recompute arrival times
validate	校验	validate the request
return	返回	return 202 Accepted
trigger	触发	trigger a notification
reuse	复用	reuse the service on other platforms
decouple / couple	解耦 / 耦合（coupling）	decouple the services
monitor	监控（monitoring）	monitor hot shards
replace	替换	replace the web servers with S3
reduce / cut	降低	cut latency
meet (a requirement)	满足	meet the 3-second requirement
support	支持	support 100,000 concurrent users
guarantee / make sure	保证	make sure each voter votes once

常用名词短语（也收进书末术语索引）

中文	English	什么时候用
跨进程通信	inter-process communication	T1 慢的原因
进程内调用	in-process call	共置、合并服务后的好处
只读副本	read-only copy	T1 留副本
领域事件	domain event	副本同步
最终一致	eventually consistent	副本、拆聚合的缺点
事务范围	transaction scope	服务变大的缺点
业务专属实体	business-specific entity	T1 复用题
通用服务	generic service	T1 复用题
高峰负载	peak load	T7、T11

github.com/bibi45c · 免费分享，禁止售卖

中文	English	什么时候用
并发用户	concurrent users	引用 §4
第 90 百分位	90th percentile (P90)	通过标准
工作负载模型	workload model	T11 测试
写吞吐	write throughput	T11 分片、T6
热点分片	hot shard	分片副作用
外部状态存储	external state store	T5 有状态服务
通信亲和度	communication affinity	T12
客户端编排	client-side orchestration	T10 PWA / 原生
超媒体链接	hypermedia link	T3 Level 3
不兼容的修改	breaking change	T3 版本
向后兼容	backward compatible	T3 版本
依赖指标	dependent metric	T8
保存时长	retention period	T8
多语言持久化	polyglot persistence	T6 决策
公有 / 私有子网	public / private subnet	T6 物理图
可用区	availability zone	T5、T6

连接词与数字写法

想表达	用
因为 / 所以	because _____ / so _____ / therefore _____ / as a result _____
但是 / 而	but _____ / however, _____ / while _____
而不是	instead of _____ / rather than _____
第一、第二	firstly, _____ / secondly, _____ （或 (1) (2)）
例如	for example, _____ / such as _____
否则	otherwise, _____
除非	unless _____

中文	写法
10 万并发（concurrency）选民	100,000 concurrent voters
90% 请求 3 秒内	90% of requests within 3 seconds / P90 < 3 s
每分钟 1 万次更新	10,000 updates per minute
5 秒内展示	shown to users within 5 seconds
平均 1 分钟内更新	updated within 1 minute on average
可用性 99.9%	99.9% availability (three nines)
保存 3 年	kept for 3 years
至少 3 个副本	at least three replicas

github.com/bibi45c · 免费分享，禁止售卖

PART 4 学习笔记精编（知识层）

学习笔记回忆卡：13 章要点 + 易错回顾（一行一题）

怎么用这一章

题型（question type）卡里没有的知识点 + "易错回顾"（题目 → 常见错答 → 正确要点）。

第 1 章 解决方案架构与质量属性

架构四要素： 部件、部件间关系、与环境的关系、指导设计和演进的原则。**原则词：** 抽象、封装（Encapsulation）、高内聚（High Cohesion）、低耦合（Low Coupling）。

题目信号	架构风格
一方请求、一方提供服务	客户端-服务器（Client-Server）
每层只调下一层、可分开部署（deployment）	**分层**
每个节点（node）既是客户端又是服务器	点对点 P2P
批量、定时、文件 / 数据库同步（synchronous）	数据流
调用后等返回结果	调用-返回（Call-Return）
一条消息只给一个人处理	事件 · 点对点
一条消息很多人都要知道	发布订阅（publish/subscribe）

点对点 vs 发布订阅： 问"这条消息需要几个接收者各自处理一遍"。Kafka 消费者组（Consumer Group）：组内一人处理（像点对点），组间各一份（像发布订阅）。

组件	一句话
反向代理（reverse proxy）	服务器的"前台"：转发、隐藏内部、负载均衡（Load Balancing）、按类型分发
防火墙（firewall）	区域（zone）间的"门卫"：按 IP、端口、协议放行 / 拦截；内容攻击靠 WAF
门户服务器（Portal Server）	面向人（B2C）的统一网页入口，整合多个系统界面
API 网关（gateway）	面向程序（B2B）的 API 总入口：路由、认证（Authentication）、限流（rate limiting）；理解 API 内容
ESB	内部异构系统的"翻译 + 调度"；集中 → 瓶颈（bottleneck）、单点故障（single point of failure）

ESB 去中心化 = "智能端点（Endpoint）、哑管道"：每个服务自带转换，故障只影响自己，代价（trade-off）是重复实现、难统一管理。

层次	管什么	类比
企业架构（Enterprise Architecture (EA)） EA	整个企业的业务流程和 IT 规则、标准	城市规划
解决方案架构（Solution Architecture (SA)） SA	一个系统的部件、部署、风格、质量属性（quality attribute）、关键决策	这栋楼的设计图
详细设计（Detailed Design）	一个部件内部：类、接口、表、输入输出	某个房间的施工图

上层约束（constraint）下层。**ISO 25010 八特性**、**常见权衡**（安全（security） ↔ 易用、性能（performance） ↔ 可维护、一致性 ↔ 可用性（availability））见 T15 卡；4+1 视图（4+1 View Model）：逻辑（类图（class diagram））、进程（时序）、开发（组件）、物理（部署）+ 场景（scenario）（用例）。

易错回顾

题目	常见错答	正确要点
5 个场景判断风格	4 题对；网银三层题答"C/S + 分层"	主答案写**分层**（"不能直连数据库""可分开部署"）
T7：易用性（usability）最重要吗	想写"同意，还有性能和安全"	官方**不同意**；性能理由（reason）写"10 万并发（concurrency）下 90% 3 秒"
常见错写	ticket / his own time window / will be not reliable	one **vote** / voting **period** / will **not be** trustworthy
EA / SA / 详细设计	把效用树（Utility Tree）放进详细设计；SA 里写"人员协同（choreography）"	效用树属 SA；SA 是**系统部件**的协同
六部分场景"应付大量用户"	Artifact 写成"业务请求"；没数字	Artifact = 系统哪部分；Measure **必须有数字**
"今晚开会"发给数据组	以为是点对点	每人都要收到 = 发布订阅；工单一人处理 = 点对点
防火墙能拦超大文件上传吗	以为能	不能；在网关 / 服务端校验请求体（request body）大小

第 2 章 架构方法：逻辑 → 物理

三项活动： 定义需求（上下文、用例、质量场景（scenario）、排序出架构上重要的需求（Architecturally Significant Requirement (ASR)））→ 逻辑架构（Logical Architecture）（与技术无关）→ 物理架构（Physical Architecture）（具体产品、数量、规格）。每步更新架构文档 SAD 并与干系人评审。

元素	概述	细化
功能元素	子系统（subsystem）、组件	接口、操作签名、前后置条件
部署（deployment）元素	位置、节点（node）	部署单元（Deployment Unit）、连接

架构决策（architecture decision）格式： Issue / Decision / Assumptions / Alternatives / Justification。例：规则引擎（约 30 条规则 → 自研简单引擎；备选：买商业引擎、写死；理由（reason）：规则少不划算，写死违反关注点分离（Separation of Concerns））。

逻辑 → 物理映射： 一对一；一对多（为可用性（availability）部署多个实例）；多对一（一个套装软件实现多个逻辑元素）。集成端点（Integration Endpoint）表写：源、目标、协议、格式、是否转换。

打包（packaging）三个作用： 决定盒子放哪台机器；方便找错（error finding）；改动影响范围小。

易错回顾

题目	常见错答	正确要点
优惠券属营销服务却在订单数据层	对；写成"营销服务器"	**服务**不是服务器
手机部署了通知控制子系统	纠正（correction）写成"改打包"	错在**部署**：删 deploy
PaymentRecord 打包进 Order DA	对	改进 Payment Manager Data Access
外卖下单找 3 错	新建"购物车子系统"；论证（justification）加了"订单常看"的假设（assumption）	整盒部署 Order Manager DA 到手机；**只用题目依据**
同上，OrderNotif	写"新建 Notification Manager"	写全：**Notification Manager Data Access**
数据库服务器在 DMZ	只搬 Payment；理由"无需连外部"	**整个节点**移内部区（internal zone）；理由是区域（zone）规则
PaymentGatewayAdapter 在 Control	想把支付网关迁到 DMZ；后来放 DMZ	新建 **Payment Manager External Integration**，放**内部区**

github.com/bibi45c · 免费分享，禁止售卖

第 3 章 平台思维

视角	平台是什么
产品	让外部各方在上面构建互补产品的基础
技术	应用构建并运行其上的一组服务
商业	促成生产者（producer）和消费者（consumer）交换的商业模式

平台 / 框架（Framework）**/ 库：** 你在平台上工作；框架调用你的代码；你调用库。**线性 vs 网络业务：** 关注流程 vs 关注交互（interaction）；Think roles, not markets（同一人可既是生产者又是消费者）。

平台要素	工程要素
种子（seed）	领域实体（Domain Entity）、可扩展性（scalability）、性能（performance）
交互	服务设计、架构、对外 API、业务流程、可扩展性、性能（六个）
磁铁（Magnet）	易用性（usability）设计、激励机制
工具箱（Toolbox）	Web、移动、API、集成
撮合者（Matchmaker）	数据收集与分析

易错回顾

题目	常见错答	正确要点
Grab 外卖的种子	写了"食品 / 商品 / 服务"三个并列	选**一个**：菜品（订单）；交互两侧都写
打车	种子"出行服务"	**行程订单**；补派单（撮合（matchmaking））和支付
高峰叫车的工程要素	性能写"不卡顿"；架构写"判断瓶颈（bottleneck）"	性能要**量化**；架构写**手段**（队列削峰（peak shaving）、水平扩展（horizontal scaling））
为什么一次只建一种交互	MVP、风险（risk）、学习成本、变现点	答得好，可直接用

第 4 章 DDD 与可复用资产

资产	白话	注意
可复用服务（Reusable Service）	独立运行，别人调接口（规模最大）	边界（Boundary）用 DDD 划
可复用（reusable）框架（Framework）	搭服务的模板（同技术栈）	
共享库（Shared Library）	复制进每个程序	**容易造成耦合**（coupling）

耦合：A 改了 B 要不要跟着改、跟着重测。**限界上下文**（bounded context）管三样：词的含义、自己的数据、负责的团队；一个 BC 通常是一个微服务（microservices）（一个 `<<service>>` 框）。划分三规则：按功能内聚（cohesion）；归类核心 / 子领域；拆了更好复用（reuse）就继续拆。**通用域**（generic subdomain）（通知、支付、身份）最适合共享，标准是"各行业都要"，不是"简单"。

构件	一句话
实体（entity）/ 值对象（Value Object）	有标识会变 / 无标识不可变（Address、Money）
聚合（aggregate）/ 聚合根（aggregate root）	必须整体一致的一组 / 唯一入口
仓储（Repository）	一个聚合一个
领域事件（domain event）	过去时命名，含命令全部属性，顺序重要

聚合 7 法则： 边界内保护不变量（invariant）；**设计小聚合**（small aggregate）；**只按 ID 引用**；用最终一致（eventually consistent）更新其他聚合；必须立即一起更新就合并；避免贫血模型（Anemic Domain Model）；一个聚合一个仓储。

上下文映射（Context Mapping）：防腐层（Anticorruption Layer）= 下游自己加翻译层；遵奉者（Conformist）= 下游照搬（**两者都在下游**）。集成用 RPC / REST / 消息，**避免共享数据库**（Shared Database）；消息要至少一次投递（At-Least-Once Delivery）+ 幂等接收者（Idempotent Receiver）。**读图四步：** 圈实体 → 找所属框 → 看跨框线 → 对照五类症状。

易错回顾（诊断题，一题一行）

场景	类别	正确要点 / 易错点
外卖通知服务	复用	判断：说"怎么发"留，说"发什么业务"搬走
教育 UserManager	复用	缺点（drawback）写"没有大缺点"（常见错写：写成改之前的问题）
旅游 BillingManager	复用	HotelDepositRule → HotelManager，FlightChangeFee → FlightManager
在线课程 Course 聚合	内存	引用父聚合 **courseId**（常错写成 studentId）
网银 Account	内存	Beneficiary "≤ 5 个"是不变量，**留下**；按年月分表不是改领域模型（domain model）
电商商品页	慢	Price、Stock 搬进 ProductManager；**库存不能用副本**（replica）**（超卖）**；拆图片是答非所问
在线考试	独立扩展（scale independently）	**只拆 Answer**
优惠券 100 次上限	不一致	Redemption 合并进 Coupon 聚合；缺点是并发（concurrency）冲突（不是数据出错）
医院病历	慢	Patient + Allergy 合并；Prescription 留只读副本（read-only copy）；**原因**（cause）**只用题目事实**
文件模板 / 合同条款	复用	要**点名搬去哪个服务**
社交主页	内存	拆粉丝、动态、收藏；依据是**有没有必须同时成立的规则**，不是体量
直播	独立扩展	只拆高负载的 Stream
OrderTracking 读 Location	慢	只读副本 + LocationUpdated 事件；若把 Location 搬走 → 写入方又慢
宠物寄养类图（class diagram）	T2	订单、评价不能删；漏了笼位、照片、支付；宠物和店通过订单连

第 5 章 API 设计与网关

REST 六约束（constraint）：客户端-服务器（Client-Server）；**无状态**（stateless）（每个请求带全部信息，可水平扩展（horizontal scaling））；可缓存（cache）；统一接口（URI、表述、自描述、HATEOAS）；分层系统；按需代码（可选）。

风格	适合
REST	绝大多数 Web / 移动 API
SOAP	银行等严格契约场景（scenario）
GraphQL	前端（front end）需求多变、关系复杂
gRPC	微服务（microservices）间低延迟（latency）（二进制）
WebSocket	实时推送（push）、聊天
Webhook	事件回调客户端 URL

API 产品思维： 由外而内（outside-in）；开发者同理心；粒度（Granularity）（SRP / ISP）；端点（Endpoint）稳定；文档用 OpenAPI。**网关**（gateway）**职责：** 路由、聚合（aggregate）、协议转换、认证（Authentication）授权（Authorisation）、限流（rate limiting）配额（Quota）、缓存监控（monitoring）、BFF。网关管南北向，服务网格（service mesh）管东西向（East-West）；反向代理（reverse proxy）主要做转发、负载均衡（Load Balancing）、TLS 终止，网关在此之上做认证、限流、版本（version）、聚合等 API 管理。**安全**（security）：校验参数、全程 TLS、限流；OAuth 2.0、JWT、SAML、CORS。

github.com/bibi45c · 免费分享，禁止售卖

易错回顾

题目	常见错答	正确要点
书店 4 个操作	`GET book/7`、`POST order`	`/books/7`、`/orders`：复数、以 `/` 开头；电话放请求体（request body）✓
`GET /orders/5/cancel`	说是"URI 带动作"	**主要错**是 GET 改状态
`POST /orders/get` 等	写"第二级"	**Level 1**（从 0 编号）
当前 1.1.0 新增接口	1.1.0	**1.2.0**（先看当前版本）
扫码枪 PUT 整个包裹	按**枪 ID** 分区（partition）；URI 放分区 ID；例子漏返回码	按**包裹 ID**；`POST /parcels/{id}/scan-events`；四项写全
书店订单 Level 3	改地址用 PUT；links 写 update-timestamp	**PATCH**；links 写下一步能做的；已发货改地址 409

第 6 章 平台管理与可观测性

工具	解决什么
服务发现（service discovery）	服务找到彼此（注册 → 查找 → 连接）
部署（deployment）策略	无缝发布、在线试验（见 T17）
熔断（circuit breaking）与限流（rate limiting）	优雅处理错误、防冲垮
分布式追踪（AWS X-Ray）	快速排障（trace / span，跟踪 ID 放 HTTP 头）
指标（metric）采集（capture）	数据驱动决策
日志（logs）聚合（aggregate）	集中查询（带关联（Association） ID）

服务发现： 注册中心 = 内部 DNS，3 或 5 个奇数节点（node）（Raft / Paxos）；客户端发现（自己查、自己负载均衡（Load Balancing））vs 服务端发现（网关（gateway）/ 负载均衡器查）。**Saga：** 本地事务（transaction）+ 补偿事务；编排（orchestration）（中心有单点）vs 协同（choreography）（逻辑分散）；没有读隔离（isolation）。**指标两类：** 基础设施（infrastructure）（CPU、请求量、响应时间（response time）看百分位（percentile））；应用（任务成功、清晰度、满意度）。

易错回顾

题目	常见错答	正确要点
外卖"下单顺畅"指标	组件写日志组件；频率"按需"；存 OpenSearch → RDS	服务发事件 + 流处理计算；**每 1 分钟汇总**（aggregation）；Timestream + S3
打车"平均接单等待"	问"依赖指标（dependent metric）是什么"	就是公式里的**两个时间戳**，各写采集、保存、位置
新推荐模型先给部分用户	—	**金丝雀**（canary）（不影响用户用真实流量测 → 影子流量（Shadowing））
Yield vs Uptime	—	没请求时停机不影响人；Yield 衡量真实影响

第 7 章 云计算与迁移

服务模型 = 你管到哪一层（越往下运维越少、控制越少、锁定越强）：

模型	你管	厂商管	AWS 例子
IaaS	操作系统往上全部	硬件、虚拟化、网络	EC2
CaaS	容器（Container）镜像（container image）、K8s 配置（configuration）	容器编排（Container Orchestration）平台	EKS、ECS
PaaS	只交代码和配置	运行时、中间件、操作系统	Elastic Beanstalk
FaaS	只写函数（function）（事件触发、按调用计费）	所有服务器	Lambda
SaaS	直接用	全部	Microsoft 365

部署（deployment）**模型：** 公有 / 私有 / 社区 / **混合云**（私有 + 公有互联，敏感数据留本地）/ **多云**（Multi-Cloud）（多家公有云，避免锁定；额外组件见 T5 卡）/ VPC（公有云里逻辑隔离（isolation）的私有网络）。**NIST 五特征：** On-demand self-service、Broad network access、Resource pooling、Rapid elasticity、Measured service。

迁移（migration）： 7R 判断见预备卡 C p.10。课件另有 **Encapsulate 封装**（Encapsulation）（把旧系统的数据和功能包成 API）。Lift and shift 的两个缺点（drawback）：用不上云原生（Cloud-Native）特性；云上运行可能更贵。别先迁最关键、最复杂的系统（如 ERP）。**IaC 好处：** 环境一致、版本（version）控制 + 评审、代码即文档、少人为错误、灾难后在新区域（zone）重建。

“可用句式｜__ 处填案例内容 **第 7 章句子：混合云 · 为什么上云 · IaC**

A hybrid cloud keeps the sensitive _____ data on-premises to meet _____ regulations, while the elastic _____ workload runs in the public cloud and scales with demand.

> 混合云把敏感的 _____ 数据留在本地以满足 _____ 法规，弹性的 _____ 负载放在公有云上，随需求扩缩。

The cloud gives rapid elasticity and measured service: capacity is added in minutes for the peak of _____ and we pay only for what we use, instead of buying servers for the peak.

> 云提供快速弹性和按量计费：高峰 _____ 时几分钟内加容量，只为用掉的部分付费，不用按高峰买服务器。

With Infrastructure as Code (Terraform), every environment is created from version-controlled templates, so test and production are identical, changes are reviewed like code, and the whole stack can be rebuilt in another region after a disaster.

> 用基础设施即代码（Terraform），每个环境都从版本控制的模板创建，测试和生产完全一致，变更像代码一样评审，灾难后能在另一个区域重建整套系统。

第 8 章 云原生架构

宠物 vs 牛群： 垂直扩展（Vertical Scaling）、粘性会话（sticky session）、单体（monolith）vs 水平扩展（horizontal scaling）、共享状态存储、松耦合（coupling）。**Google 五原则：** 为自动化设计；聪明处理状态；优先托管服务（managed service）；纵深防御（defence in depth）；持续做架构。**十二要素**里和扩展相关：**6 进程（无状态**（stateless）**）**、**8 并发**（concurrency）**（进程模型水平扩展）**；其余：代码库、依赖、配置（configuration）、后端（backend）服务、构建发布运行分离、端口绑定、易处置、环境等价、日志（logs）即事件流（event streaming）、管理进程。

状态 = 会影响下一次请求结果、需要记住的信息；**无状态** = 不依赖上次请求留在自己身上的东西，需要时去共享存储查。好处：可扩展、一致、优雅关闭、可恢复。

状态放哪	评价
粘性会话	不推荐：实例挂了丢会话、负载不均、妨碍缩容（scale in）
共享状态存储	Redis / 数据库存会话、计数
StatefulSet + PV / PVC	集群（Cluster）内自己跑有状态组件
外部状态存储（external state store）（托管）	**生产首选**

风格	一句话	缺点
单体	一个整体	紧耦合，只能整体扩
微服务（microservices）	小、独立部署（deployment）	运维复杂、网络延迟（latency）
六边形	端口与适配器（代码结构）	结构复杂
Serverless	事件触发函数（function）	冷启动（cold start）、锁定、时长上限
事件驱动	通过事件通信	难追踪（tracing）、最终一致（eventually consistent）
Lambda / Kappa	批 + 流 / 只用流	两套逻辑 / 重处理难

易错回顾

题目	常见错答	正确要点
在线考试改造：答题交卷	微服务 ✓，理由（reason）"每场考试独立部署"	理由写**负载特点**（开考暴涨十倍）
同上：交卷发邮件	想到六边形、可复用（reusable）	**Serverless**（突发、短小）；六边形不是部署风格
同上：每晚统计	批处理 ✓	CronJob
粘性会话为什么不好	只答"那台挂了会丢"	补**负载不均、妨碍缩容**，并点名 Redis
外卖四件事分配风格	全对	每项加一句理由

github.com/bibi45c · 免费分享，禁止售卖

第 9 章 微服务与服务间通信

事件流（event streaming）**"按 X 分区**（partition）**"（PAST2 例）：** 车载单元的位置事件按班次号 busTripId 分区。同一班次永远进同一分区 → 一个实例**按顺序**处理；不同分区 → 不同实例**并行**（Parallel）。一个分区里会混着好几个班次。消费者组（Consumer Group） 1 = ArrivalTimeComputer，结果写进 DynamoDB / Redis，**乘客 App 经 API 读存储**，不直连实例；消费者组 2 = DesirableSpeedComputer，再读同一批事件，2 秒内回传车速。

```
17号班次 第1条 ┐
05号班次 第1条 ├→ 分区 0 → 实例 1（按顺序）┐
17号班次 第2条 ┘                            ├→ 到站时间存储 ← 乘客 App
42号班次 第1、2条 → 分区 1 → 实例 2（并行）  │
88号班次 第1、2条 → 分区 2 → 实例 3（并行）  ┘
组 2 DesirableSpeedComputer：再读全部分区 → 2 秒内回传车速
```

消息队列（message queue） **（抢票例）：** 用户是**生产者**（producer）（点抢票 = 放一张订单）；后台出票实例是**消费者**（consumer），一单只给一个实例，出完即删，不保序、不能重放。

词	一句话
同步（synchronous） vs 异步（asynchronous）	要立刻拿结果 → 同步 REST / gRPC；状态变化、多方要读、削峰（peak shaving） → 异步事件
扩展立方体 Scale Cube	X 克隆副本（replica）；Y 按功能拆（先拆开，再只给忙的服务加副本）；Z 按数据分片（sharding）
每服务一个数据库（Database per Service）	松耦合（coupling）、独立扩展（scale independently）；跨服务（cross-service）查询和事务（transaction）变难
CQRS	写模型和读模型分开，读预先算好的视图
事件溯源 Event Sourcing	存事件序列而不是当前状态，可审计（audit）、可重算

句子见 T10 补充框 p.**35**、T11 补充框 p.**38**。

易错回顾

题目	常见错答	正确要点
模拟卷 1 Q1(a)(ii) 队列还是事件流	两个理由（reason）都落在"事件会保留"	第二个理由换成"按 connectorId 分区保序"
模拟卷 1 Q2(a)(ii) 内部通信	只写服务发现（service discovery）	同步 + 异步两句都写（T5 (ii) 句）
"按 xx 分区"没懂	以为实例 1 直接服务乘客	分区间并行、分区内有序；实例写存储，乘客读存储

第 10 章 容器与 Kubernetes

	虚拟机	容器
隔离（isolation）	硬件级（Hypervisor）	进程级（共享内核（Shared Kernel））
大小 / 启动	GB / 分钟	MB / **秒**
适合	遗留、不同 OS、强隔离	微服务（microservices）、快速扩缩、CI/CD

K8s 调度容器（Container），**但节点**（node）**通常是虚拟机**（Virtual Machine (VM)）**（EC2）**，不矛盾。控制平面（Control Plane）：kube-apiserver（入口）、etcd（集群（Cluster）状态）、scheduler（放哪个节点）、controller-manager（让实际靠近期望）；节点：kubelet、kube-proxy、容器运行时；附加：CoreDNS、Metrics Server。

Service 怎么找 Pod： 靠**标签选择器**（Label Selector） + 就绪探针（Readiness Probe），不是 Deployment 通知。Deployment 的 Pod 没身份没盘，缩了再扩是新 Pod；StatefulSet 同名 Pod 挂回自己的盘。**可用区**（availability zone）= 同一地区的几个机房，不是异地备份。

托管 vs 自建（self-hosted）见 T10 卡。

易错回顾

题目	常见错答	正确要点
无状态（stateless） voting 用什么对象	Pod	**Deployment**（+ Service）
集群里跑 PostgreSQL 主从 / 每晚统计 / 每节点日志（logs）	StatefulSet + PVC / cronjob / daemonset	✓（写对象名大写）
在线考试部署（deployment）计划	三个服务都配 HPA	**HPA 只给 exam-svc**；补 Ingress、托管 Redis / DB、CronJob
"加一台 EC2 是加入 namespace 吗"	—	加进**整个集群**；namespace 是逻辑分组

第 11 章 数据管理与云存储

模型	擅长	短板
键值	按键极快：会话、购物车、计数	范围查询贵
文档	JSON 嵌套、schema 灵活	少 join、少复杂事务（transaction）
列族	写密集、稀疏、时序	无 join，二级索引贵
搜索引擎（Search Engine）	全文检索、日志（logs）	插入慢，不作主存储
图	关系遍历、推荐	难水平扩展（horizontal scaling）
SQL 查文件	大批量分析	不适合在线事务

RDBMS 强 ACID，难点是保持一致还要水平扩展；**NewSQL**（Spanner）兼顾。**老师的选型**（technology selection）**四问：** 数据形态、量级、关系复杂度、一致性要求；财务账本要强一致（strongly consistent），别被 NoSQL 带偏。**购物车作业：** 登录 → RDBMS；页头 → 键值；商品列表（字段各异）→ 文档；下单 → RDBMS 事务；推荐 → 图。

易错回顾

题目	常见错答	正确要点
写是瓶颈（bottleneck），加读副本（read replica）？	没用 ✓	写仍进一个主库（Primary） → 分片（sharding）
选分片方式	哈希 ✓	范围分片（Range Sharding）易热点
分片受影响组件 / 副作用（side effect）	不知道怎么写	见 T11 卡：写入方、存储方、用户服务、汇总（aggregation）方 + 热点、跨片汇总慢
打车：海量写 + 毫秒读	DynamoDB + ElastiCache ✓	每个服务写**满足哪个属性、靠什么机制**
外卖 AWS 链路走一遍	菜品图片存 **Neptune**	图片存 **S3**（+ CloudFront）；Neptune 是图数据库（Graph Database）

第 12 章 性能与容量规划

类别	战术
控制需求	限流（rate limiting）、排队（queueing）、流量控制、排优先级
管理资源（Resource）	加资源、提高效率、多份计算副本（replica）、多份数据副本、减少开销、调度
设计启发	负载均衡（Load Balancing）、缓存（cache）、预计算（precomputation）、降低算法复杂度、消除瓶颈（bottleneck）

缓存看命中率（hit ratio）（幂律分布时小缓存也有高命中率）；预计算例：Twitter 发推时写进粉丝时间线（扇出（Fan-out）），名人仍用查询。**容量规划**（capacity planning）：存储看累积、计算看峰值（peak）；趋势分析（Trend Analysis）（线性回归、移动平均、指数平滑；注意季节性）；Amdahl / USL：加机器不线性；算上采购提前期（Procurement Lead Time）。**弹性**（Elasticity）**四准则：** 主动；扩激进缩保守；分析定阈值；避免乒乓。

github.com/bibi45c · 免费分享，禁止售卖

易错回顾

题目	常见错答	正确要点
串行（Serial）1000/500/800/300，提到600	网关（gateway）负载均衡 + 数据库分片（sharding） ✓	**500 的网关也要改**；写前提（assumption）"写是瓶颈"
为什么阈值 60–70%	"性价比区间"	补机制：R = S/(1−U)，70% 后排队急增；启动要时间
抢票工作负载模型（workload model）	曲线写"观察"；没数字	选 **Spike**；100 万 ÷ 300 秒 ≈ 3,300 / 秒

第 13 章 高可用、可维护与安全

算题和战术见 T13–T17 p.**40**，可维护性（maintainability）四战术见预备卡 H p.**11**；这里只记卡上没有的。

四个扣分点： ① 可用性（availability）（需要时能不能用）≠ 可靠性（reliability）（一段时间内能否持续正确工作）；② 冗余（redundancy）要配**故障检测 + 自动切换**；③ 重试（retry）要配**超时**（timeout） **+ 指数退避 + 熔断**（circuit breaking）；④ 副本（replica）在同一可用区（availability zone）或同一宿主机上，**不能按并联**（in parallel）**算。**

备份	每次备份什么	恢复需要
完全 Full	全部（最慢、最占空间）	1 份
差异 Differential	上次**完全**备份后的变化	完全 + 最近一次差异
增量（delta）Incremental	上次**任意**备份后的变化（最快）	完全 + 之后所有增量（恢复最慢）

例：周日完全 + 其余每天增量，周五早上数据坏了 → 周日完全 + 周一至周四的增量，按顺序恢复。**RAID：** 1 = 镜像（container image）；5 = 坏一块盘能恢复。

易错回顾

题目	常见错答	正确要点
模拟卷 1 Q3(c) 可用性计算	没算出 99.25%；外部网关（gateway）写"配多可用区（Multi-AZ）"	一定写出数字；外部依赖改为第二家并联切换，或异步（asynchronous）移出同步（synchronous）路径

github.com/bibi45c · 免费分享，禁止售卖

PART 5 速查与索引

常用数据与性能指标：案例数字 • 公式 • 默认值

怎么用这一章

答题要"引用数字"时来这里查：**案例里的数字照抄附录**；案例没给时，用本章的"答题默认数字"并写一句 **Assume _____**。标【通用知识】的是课程没讲、业界常用的量级，只用来帮助论证（justification），不当课件原话。

三套样卷的关键数字（照抄用）

项目	PAST • Vote Anywhere	PAST2 • MLTTP 公交	PAST3 • ONE Health
主要并发用户（concurrent users）	**100,000 Voters**、1,000 Proctors、1,000 Observers、10 Election Admins、5 Platform Admins	**100,000 Public Users**、**10,000 Registered Users**、5 System Admins	文本未写具体人数；要求随数据增长保持响应
设备 / 写入量	10 万选民 5 分钟内投完票（Q3(d)）	**10,000 Vehicle Onboard Units**；车队最多 **3,000** 辆；第一阶段每分钟 **10,000** 次更新	运动轨迹持续写入
交互（interaction）响应	交互请求 **90% < 3 秒**	查看到站 / 载客量 **90% < 3 秒**	交互操作（如查配送状态）**90% < 1 秒**
实时更新	简要结果平均 **1 分钟**内；详细结果平均 **3 分钟**内	变化 **5 秒**内展示；理想车速 **2 秒**内送达车载设备；每 **30 秒**同步（synchronous）一次（Q1(a) 题干（question stem），非附录）	—
测试参数	30 秒调用时间窗、最大响应 < **5 秒**（Q1(a)(i)）	负载测试（load test）中延迟（latency） **20–30 秒**（Q1(c)）	—
多租户（multitenancy）	**不预期**	**不预期**	**必须支持**，按医疗机构、供应商、服务类型分片（sharding） / 分区（partition）
安全（security）	DMZ 反向代理（reverse proxy）、个人信息加密（encryption）、GDPR	租户专属密钥（Tenant-Specific Key）、PCI-DSS	DMZ、加密、GDPR、**租户专属密钥自动轮换**、**PCI-DSS 支付只在最少子系统**（subsystem）

性能与可用性指标：定义和写法

中文	English	定义 / 公式	答题时这样写
响应时间	Response Time	发出请求到收到响应 = 服务时间 + 等待时间 + 传输时间	**P90 response time below 3 s**
服务时间	Service Time	组件真正处理一个请求的时间	**reduce the service time by caching**
等待 / 排队时间	Waiting / Queueing Time	排在别的请求后面等的时间	**queueing time grows sharply above 70% utilisation**
延迟	Latency	数据从一个组件传到另一个组件的时间	**inter-process calls add latency**
首字节时间	Time to First Byte (TTFB)	发出请求到收到第一个字节	Web 前端性能
吞吐量	Throughput	单位时间完成的请求数（req/s、TPS）	**write throughput of 10,000 updates per minute**
每秒事务数 / 查询数	TPS / QPS	Transactions / Queries Per Second	**the app server reaches 6 TPS**
利用率	Utilisation	忙碌时间 / 可用时间 = λ × S	**keep CPU below 60–70%**
到达率	Arrival Rate (λ)	每秒到达的请求数	**about 3,300 requests per second**
并发数	Concurrency	同时在系统里的请求或用户数	**100,000 concurrent voters**
百分位	Percentile (P50 / P90 / P95 / P99)	P90 = 90% 的请求不超过这个值	**90% of requests within 3 s (P90 < 3 s)**
最大响应时间	Maximum Response Time	最慢那个请求	**maximum below 5 s**
错误率	Error Rate	失败请求 / 总请求	**error rate below 0.1%**
产出率	Yield	成功请求 / 总请求（比 uptime 更重要）	**payment success rate (yield)**
收获率	Harvest	返回的数据 / 全部数据	**partial results when a shard is down**
可用时间	Uptime	(MTBF − MTTR) / MTBF（PE 课件；这里 MTBF 指两次故障起点的间隔，含修复时间）	**three nines of uptime**
可用性	Availability	MTBF / (MTBF + MTTR)	**99.9% availability**
平均故障间隔	MTBF	总运行时间 / 故障次数	
平均恢复时间	MTTR	总修复时间 / 修复次数	**reduce MTTR with health checks and auto-restart**
平均无故障时间	MTTF	总运行小时 / 单元数	

github.com/bibi45c · 免费分享，禁止售卖

中文	English	定义 / 公式	答题时这样写
恢复时间目标	Recovery Time Objective (RTO)	故障后多久必须恢复【通用知识】	**RTO of 1 hour**
恢复点目标	Recovery Point Objective (RPO)	最多能丢多少时间的数据【通用知识】	**RPO of 5 minutes**
缓存命中率	Cache Hit Ratio	命中次数 / 总读取次数	**a cache only helps with a high hit ratio**
复制延迟	Replication Lag	主库写入到副本可读的延迟	**replicas may return stale data**
事件流积压	Consumer Lag	还没被消费的事件数	**scale consumers on stream lag**
队列长度	Queue Length	等待处理的消息数	**scale workers on queue length**
平均绝对误差	Mean Absolute Error (MAE)	平均 |预测 − 实际|	**a falling MAE shows better prediction**
服务等级目标 / 协议	SLO / SLA	内部目标 / 对客户承诺（含赔偿）【通用知识】	**the SLA is 99.9%**
DORA 指标	DORA Metrics	部署频率、变更前置时间、MTTR、变更失败率	平台工程题

利用率 → 响应时间（R = S / (1 − U)）

利用率 U	50%	60%	70%	80%	90%	95%	99%
响应时间（response time）是服务时间（Service Time）的	**2 倍**	2.5 倍	**3.3 倍**	5 倍	10 倍	**20 倍**	100 倍

结论：**超过约 70% 排队**（queueing）**急剧增长 → 自动扩容**（scale out）**阈值设 60–70%。**课件原话只给了 50% = 2 倍、70% > 3 倍，其余是按同一公式算的。

可用性：几个 9 → 允许停机多久

可用性	每年	每月（30 天）	每周
99%（两个 9）	3.65 天	7.2 小时	1.68 小时
99.5%	1.83 天	3.6 小时	50.4 分钟
99.9%（三个 9）	**8.76 小时**	43.2 分钟	10.1 分钟
99.95%	4.38 小时	21.6 分钟	5.0 分钟
99.99%（四个 9）	**52.6 分钟**	4.3 分钟	1.0 分钟
99.999%（五个 9）	5.3 分钟	26 秒	6 秒

组合	算式	结果
两个 99% 串联（in series）	0.99 × 0.99	98.01%
三个 99.9% 串联	0.999^3	≈ 99.7%
三个 99% 串联	0.99^3	≈ 97.0%
两个 99% 并联（in parallel）	$1 - 0.01^2$	99.99%
两个 99.9% 并联	$1 - 0.001^2$	99.9999%
两个 98% 并联	$1 - 0.02^2$	99.96%
课程作业现状	$0.999^2 \times 0.99^4 \times 0.98$	93.95%
课程作业改造后	$0.999999^2 \times 0.9999^4 \times 0.9996$	99.92%

年停机分钟 = (1 − A) × 525,600。老师说：**2% 的不可用大多来自软件更新，不是硬件。**

换算：人数和时间 → 每秒多少请求

题目说	每秒	怎么用
10 万选民 5 分钟内投票	100,000 ÷ 300 ≈ **333 / 秒**（平均，开始时更高）	分片（sharding）、写吞吐（write throughput）论证（justification）
10 万次调用 30 秒内	100,000 ÷ 30 ≈ **3,333 / 秒**	负载测试（load test）到达率（arrival rate）
100 万人 5 分钟内进入	1,000,000 ÷ 300 ≈ **3,333 / 秒**	抢票测试
每分钟 1 万次更新	10,000 ÷ 60 ≈ **167 / 秒**	遥测写入
假设（assumption）例（非案例数字）：每台设备每 30 秒一次 × 1 万台	10,000 ÷ 30 ≈ **333 / 秒**	设备上报
每天 324,000 次，60% 集中在 4 个忙时	324,000 × 60% ÷ 240 = **810 / 分钟**（平均 675 / 分钟）	课程作业峰值（peak）估算

github.com/bibi45c · 免费分享，禁止售卖

换算	数
1 分钟 / 1 小时 / 1 天	60 秒 / 3,600 秒 / **86,400 秒**
1 年	525,600 分钟 ≈ 3,150 万秒
Little 定律 n = λ × R	333 / 秒 × 3 秒 ≈ **1,000 个请求同时在系统里**；1,000 / 秒 × 0.2 秒 = 200
每 Pod 承载 → Pod 数	100,000 ÷ 2,000（假设）= **50 个 Pod**
复合增长 10% / 月，24 个月	1.1^{24} ≈ **9.85 倍**
存储单位	1 KB = 1,024 B；1 MB = 1,024 KB；1 GB = 1,024 MB；1 TB = 1,024 GB

延迟量级：为什么"跨服务就慢"

操作	大约	来源
进程内函数（function）调用	纳秒到微秒	【通用知识】
读内存	约 100 纳秒	【通用知识】
Redis / ElastiCache 读	**亚毫秒**（< 1 ms）	笔记 11
DynamoDB 读写	**个位数毫秒**	复习书第 16 章
同一数据中心内一次网络往返	约 0.5 毫秒	【通用知识】
SSD 随机读 / 机械硬盘寻道	约 0.1 毫秒 / 约 10 毫秒	【通用知识】
跨洲网络往返	约 100–150 毫秒	【通用知识】
容器（Container）启动 / 虚拟机（Virtual Machine (VM)）启动	**秒级 / 分钟级**	笔记 10
Serverless 冷启动（cold start）	几百毫秒到几秒	【通用知识】
日志（logs）和追踪（tracing）数据可查到	常有约 15 分钟延迟（latency）	老师答疑

一句话： 跨进程（网络）调用比进程内调用（in-process call）慢上千倍，所以领域模型（domain model）里**一起用的实体**（entity）**分在两个服务**就会慢；共置（collocation）、合并服务能把网络调用变成进程内调用。

答题默认数字（案例没给时用，写 Assume）

用在哪	默认值	依据
HPA 扩容（scale out）阈值	CPU **60–70%**	排队（queueing）拐点（笔记 12）
最少副本（replica）/ 可用区（availability zone）	**2–3 个 Pod，跨 2–3 个可用区**	可用性（availability）（笔记 10）
最多副本	峰值（peak）÷ 单 Pod 承载量；**Assume one pod handles about 2,000 users**	笔记 10
负载测试（load test）通过标准（pass criteria）	**P90 < 3 s、最大 < 5 s、错误率**（Error Rate）**< 0.1%**	PAST 题干（question stem）Q1(a)(i) + 笔记 12（错误率是通用做法）
指标（metric）采集（capture）/ 汇总（aggregation）频率	每个事件实时采集，**每 1 分钟汇总**；分析型每天	笔记 6
指标保存时长（retention period）	原始 **30–90 天**；分钟汇总 **90 天**；日汇总 **3 年**	笔记 6
金丝雀（canary）流量	**5–10%**（95/5、90/10）	笔记 6
重试（retry）指数退避	**50 ms → 100 ms → 200 ms**	笔记 6
容量余量（headroom）	**30%**	课程作业
峰值估算	日量 × 60% ÷ 4 个忙时	课程作业
事件流（event streaming）分区键（partition key）	顺序必须一致的那个对象的 ID（班次、包裹、poll）	笔记 5
分片键（shard key）	唯一 ID + **哈希**	笔记 11
服务注册中心（service registry）节点（node）数	**3 或 5 个**（奇数，共识）	笔记 6

HTTP 状态码全表

码	名称	什么时候用
200	OK	GET 成功；PUT / PATCH 更新成功
201	Created	POST 当场创建了资源（Resource）（带新资源位置）
202	Accepted	已收下，后台异步（asynchronous）处理
204	No Content	DELETE 成功，无返回体
301 / 302	Moved Permanently / Found	永久 / 临时重定向
304	Not Modified	缓存（cache）仍有效【通用知识】

github.com/bibi45c · 免费分享，禁止售卖

码	名称	什么时候用
400	Bad Request	请求格式或参数错
401	Unauthorized	没有认证（Authentication）（没登录 / 令牌无效）
403	Forbidden	认证了但没权限
404	Not Found	资源不存在
405	Method Not Allowed	这个资源不支持该方法【通用知识】
409	Conflict	和当前状态冲突（已发货还改地址）
415 / 422	Unsupported Media Type / Unprocessable Entity	格式不支持 / 内容校验不过【通用知识】
429	Too Many Requests	超过限流（rate limiting）
500	Internal Server Error	服务器内部错误
502 / 504	Bad Gateway / Gateway Timeout	网关拿不到下游响应 / 下游超时【通用知识】
503	Service Unavailable	暂时不可用（过载、维护）

云服务常用数字

服务	数字	来源
Amazon S3	设计耐久性 **11 个 9**（99.999999999%）	笔记 11
Amazon Aurora	最多 **15** 个读副本（read replica）	笔记 11
Amazon RDS / Aurora	**Multi-AZ** 自动故障转移（failover）	笔记 11
AWS Lambda	单次最长运行 **15 分钟**；按调用和运行时间付费	【通用知识】
Amazon DynamoDB	单个 item 最大 **400 KB**；按需容量自动扩展	【通用知识】
托管服务（managed service） SLA	常见 **99.9%+**	笔记 10
Google IoT Core	已于 **2023 年**停用	笔记 11
平台 API 收入	Salesforce 约 **50%**、eBay 约 **60%** 来自 API 生态	笔记 5
维护成本	占生命周期成本 **80–90%**；改进型维护 70–75%	复习书第 13 章、老师答疑

github.com/bibi45c · 免费分享，禁止售卖

术语词典

PART 5　速查与索引

术语词典：题干难词 + 14 个主题

怎么用这一章

按主题排的术语词典，**每一行都自动收进书末的英文 A–Z 索引和中文拼音索引。**题干里的非技术难词在第一节。解释尽量一句话，细节看对应题型卡或学习笔记。

题干常见英文词（非技术）

English	中文	题干里怎么理解
adequately	充分地	要写够、写全
alternative	替代方案	另一种可选做法；"给出替代"就是再列一个产品或方案
appropriate	合适的	要说明为什么合适
assume / assumption	假设	题目没给的条件，自己写一句前提
briefly	简要地	两三句即可，别展开
concrete	具体的	要带名字、数字、请求示例
consideration	考虑因素	列出影响决定的因素
constraint	约束	必须遵守的限制（许可证、PCI-DSS）
crucial / critical	最关键的	只选最重要的一两个
deliberately omitted	刻意省略	没画的部分不算错
derive / derived	推导出	由前面的结果得出；"派生指标"是计算出来的
describe / outline	描述 / 概述	说清是什么、怎么做
drastic	大幅的	drastic adaptation = 大改
effect / side effect	效果 / 副作用	措施带来的不良后果
elaborate	详细说明	要展开写理由
ensure	确保	要说明靠什么机制保证
evaluate	评估	先给判断，再给依据
existing	现有的	以附录原图为准
facilitate	促进、便于	
feasible	可行的	
identify	指出	点名具体对象
illustrate	图示 / 举例说明	要画图或给具体例子
impact / implication	影响	"对后端的影响"要单独写一段
incomplete	不完整的	图不全；可能要你补全
justify / justification	论证 / 理由	引用案例数字或规则
leverage	利用	
mitigate	缓解	降低风险或代价
most likely cause	最可能的原因	给一个主要原因
novice	新手	"新手画的图"= 里面有错
optimal / suboptimal	最优 / 不够优	suboptimal = 不理想
propose	提出	写具体改法
rationale	理由	= justification
recommend	推荐	写具体产品名
respectively	分别	前后一一对应
scenario	场景	质量属性场景要六部分
stakeholder	利益相关者	提需求的人
suitable	合适的	
trade-off	取舍	得到什么、失去什么
undesirable	不理想的	T4 找错
with respect to / in terms of	就……而言	在某个方面比较
viable	可行的	
whereby	借此	说明机制

github.com/bibi45c · 免费分享，禁止售卖

术语词典

架构基础与质量属性

中文	English	一句话解释
软件架构	Software Architecture	部件、部件关系、与环境关系、设计和演进原则
企业架构	Enterprise Architecture (EA)	整个企业的业务和 IT 规则
解决方案架构	Solution Architecture (SA)	一个系统的整体蓝图
详细设计	Detailed Design	一个部件内部怎么实现
非功能需求	Non-Functional Requirement (NFR)	系统"做得多好"的要求
质量属性	Quality Attribute	可度量的非功能特性
质量属性场景	Quality Attribute Scenario	刺激源、刺激、制品、环境、响应、响应度量
刺激源 / 刺激	Source / Stimulus	谁 / 发生了什么
制品	Artifact	系统被作用的部分
响应度量	Response Measure	带数字的判定标准
效用树	Utility Tree	按重要性和难度给场景打分，(H, H) 先做
架构上重要的需求	Architecturally Significant Requirement (ASR)	对架构影响大的需求
性能	Performance	响应时间、吞吐量
可扩展性	Scalability	负载增加时加资源仍能满足要求
弹性	Elasticity	按负载自动扩缩
可用性	Availability	需要时能不能用
可靠性	Reliability	一段时间内能否正确工作
可维护性	Maintainability	修改软件的难易程度
可修改性	Modifiability	不引入缺陷地修改
可演进性	Evolvability	持续发布新功能的能力
易用性	Usability	用户完成任务的顺畅程度
可移植性	Portability	换平台、换云的难易
互操作性	Interoperability	与其他系统交换数据的能力
可测试性	Testability	容易验证
可分析性	Analyzability	容易找到问题根因
关注点分离	Separation of Concerns	不同职责放不同部件
高内聚 / 低耦合	High Cohesion / Low Coupling	内部紧密相关 / 部件之间少依赖
抽象 / 封装	Abstraction / Encapsulation	隐藏细节 / 把变化限制在部件内
单一职责原则	Single Responsibility Principle (SRP)	一个部件只负责一件事
4+1 视图	4+1 View Model	逻辑、进程、开发、物理 + 场景
参考架构	Reference Architecture	可复用的架构模板
架构风格	Architectural Style	结构型（分层、C/S、P2P）与通信型（调用返回、事件）
客户端-服务器	Client-Server	一方请求，一方提供服务
分层架构	Layered / N-Tier Architecture	每层只调用下一层
点对点网络	Peer-to-Peer (P2P)	每个节点既是客户端又是服务器
调用-返回	Call-Return	同步调用并等待结果
面向服务架构	Service-Oriented Architecture (SOA)	以服务和 ESB 集成为核心
企业服务总线	Enterprise Service Bus (ESB)	集中的协议转换、路由、编排
门户服务器	Portal Server	把多个系统整合到一个界面
反向代理	Reverse Proxy	代表内部服务器接收外部请求
防火墙	Firewall	按 IP、端口、协议放行或拦截
非军事区	Demilitarized Zone (DMZ)	内外网之间放对外入口的网段
响应式宣言	Reactive Manifesto	及时响应、有韧性、有弹性、消息驱动

逻辑 / 物理架构与打包部署

中文	English	一句话解释
逻辑架构	Logical Architecture	与技术无关的功能和部署元素
物理架构	Physical Architecture	具体产品、数量、规格
边界 / 控制 / 实体	Boundary / Control / Entity (BCE)	界面或接口 / 流程 / 数据
子系统	Subsystem	同一服务、同一层的打包单元
打包	Packaging	把组件和实体装进子系统
体现（包含）	Manifest	子系统包含某个组件或实体
部署	Deploy	子系统安装到某个节点
部署单元	Deployment Unit	被部署的打包好的子系统
位置	Location	Internet、DMZ、Internal Zone
节点	Node	能运行软件的设备或服务器
内部区	Internal Zone	放应用和数据库的内网区域
表现层	Presentation Layer	用户交互
控制层	Control Layer	用例流程逻辑
数据访问层	Data Access Layer	访问持久存储
外部集成层	External / Enterprise Integration Layer	对接外部系统
平台服务层	Platform Service Layer	多个应用共享的服务
共享库层	Shared Library Layer	可复用的库
系统上下文	System Context	系统与角色、外部系统的边界
规模估算	Sizing	按 NFR 确定数量和规格
集成端点	Integration Endpoint	源、目标、协议、格式
架构决策记录	Architecture Decision Record	问题、决策、假设、备选、理由
概念验证	Proof of Concept (PoC)	用原型降低技术风险

github.com/bibi45c · 免费分享，禁止售卖

术语词典

平台思维与平台工程

中文	English	一句话解释
平台	Platform	让外部各方构建产品、促成交换的基础
框架	Framework	它调用你的代码
库	Library	你调用它
种子	Seed	用户来平台想要的核心价值
生产者 / 消费者	Producer / Consumer	提供 / 使用种子的角色
交互	Interaction	围绕种子的动作
磁铁	Magnet	吸引双方加入
工具箱	Toolbox	让交互发生的工具
撮合者	Matchmaker	用数据配对生产者与消费者
网络效应	Network Effect	用户越多平台越有价值【通用知识】
最小可行产品	Minimum Viable Product (MVP)	先做最小范围验证价值
多租户	Multitenancy	多个客户共用平台、数据隔离
租户	Tenant	平台上的一个客户组织
生态系统	Ecosystem	合作与竞争共同交付价值的组织网络
平台工程	Platform Engineering	构建自助式工具链
内部开发者平台	Internal Developer Platform (IDP)	给开发者的自助平台
黄金路径	Golden Path	预设的标准做法
基础设施即代码	Infrastructure as Code (IaC)	用代码管理基础设施
GitOps	GitOps	以 Git 为唯一事实来源发布
持续集成 / 持续交付	CI / CD	自动构建测试 / 自动发布

领域驱动设计 DDD

中文	English	一句话解释
领域模型	Domain Model	业务概念和关系的模型
领域实体	Domain Entity	有唯一标识、会变化的对象
值对象	Value Object	没有标识、不可变
聚合	Aggregate	必须一起保持一致的一组实体
聚合根	Aggregate Root	聚合唯一的入口
业务不变量	Business Invariant	任何时候都必须成立的规则
小聚合	Small Aggregate	只装必须一起改的实体
按标识引用	Reference by Identity	只存另一个聚合的 ID
仓储	Repository	一个聚合的持久化
领域事件	Domain Event	过去时命名的业务事件
最终一致性	Eventual Consistency	暂时不一致，稍后一致
强一致性	Strong Consistency	写完马上读到最新值
限界上下文	Bounded Context	术语和模型一致的边界
通用语言	Ubiquitous Language	一个 BC 里共用的语言
核心域 / 支撑域 / 通用域	Core / Supporting / Generic Subdomain	竞争优势 / 辅助 / 各行业通用
上下文映射	Context Mapping	BC 之间的集成关系
防腐层	Anticorruption Layer	下游把上游模型翻译成自己的
遵奉者	Conformist	下游照搬上游模型
开放主机服务	Open Host Service	上游给所有下游的统一协议
共享内核	Shared Kernel	两个团队共享一小块模型
大泥球	Big Ball of Mud	没有边界的模型
贫血模型	Anemic Domain Model	只有 getter/setter 没有行为
可复用服务 / 框架 / 共享库	Reusable Service / Framework / Shared Library	复用规模从大到小
共享服务	Shared Service	被多个应用复用的服务
类图	Class Diagram	UML 静态结构图
关联 / 聚合 / 组合	Association / Aggregation / Composition	普通关系 / 部分可独立 / 部分随整体
泛化（继承）	Generalisation (Inheritance)	子类是一种父类
多重性	Multiplicity	关联两端的数量
业务专属实体	Business-Specific Entity	只属于某业务的实体
只读副本（领域）	Read-Only Copy	使用方保存的、由事件同步的数据

github.com/bibi45c · 免费分享，禁止售卖

术语词典

API 设计

中文	English	一句话解释
资源	Resource	URI 标识的名词对象
统一资源标识符	URI	资源的地址
端点	Endpoint	一个可调用的 URI + 方法
请求体 / 响应体	Request / Response Body	传输的数据
请求头	Request Header	元数据，如版本、令牌
查询参数	Query Parameter	过滤、排序、分页
路径参数	Path Parameter	URI 里的
安全方法	Safe Method	不改变状态（GET）
幂等	Idempotent	多次执行结果相同
幂等键	Idempotency Key	识别重复请求
理查森成熟度模型	Richardson Maturity Model (RMM)	Level 0–3
超媒体	Hypermedia (HATEOAS)	响应带下一步链接
链接关系	rel (link relation)	这个链接干什么
超文本引用	href	链接指向的地址
语义化版本	Semantic Versioning	MAJOR.MINOR.PATCH
不兼容的修改	Breaking Change	会让旧客户端出错
向后兼容	Backward Compatible	不破坏旧客户端
URI 版本 / 请求头版本	URI / Header Versioning	/v1/... / Accept-version
内容协商	Content Negotiation	用 Accept 头选格式和版本
由外而内	Outside-in Design	从调用方需要出发
粒度	Granularity	一个 API 承担的职责大小
分页	Pagination	offset 或 cursor
批量接口	Batch Endpoint	一次请求带多条数据
部分响应	Partial Response	只返回需要的字段
限流 / 配额	Rate Limiting / Quota	限制调用速度和总量
网关	API Gateway	统一入口
面向前端的后端	Backend for Frontend (BFF)	为某类客户端定制的后端
开放 API 规范	OpenAPI Specification	描述 REST API 的标准
远程过程调用	gRPC / RPC	像本地函数一样调远程
图查询 API	GraphQL	客户端指定要哪些字段
双向长连接	WebSocket	服务器可主动推送
回调	Webhook	事件发生时调用客户端 URL
授权框架	OAuth 2.0	授权标准
JSON 令牌	JSON Web Token (JWT)	签名的身份令牌
跨域资源共享	CORS	允许浏览器跨域

云原生与架构风格

中文	English	一句话解释
云原生	Cloud-Native	容器、微服务、自动化、可观测
微服务	Microservices	小的、可独立部署的服务
单体	Monolith	一个整体部署
无服务器	Serverless	事件触发的函数，厂商管服务器
函数即服务	Function as a Service (FaaS)	如 Lambda
冷启动	Cold Start	函数首次调用的初始化延迟
混合架构	Hybrid Architecture	微服务 + Serverless + 定时任务 + 事件流
事件驱动架构	Event-Driven Architecture (EDA)	通过事件间接通信
六边形架构	Hexagonal Architecture	端口与适配器，代码内部结构
Lambda / Kappa 架构	Lambda / Kappa Architecture	批 + 流两层 / 只用流
十二要素应用	Twelve-Factor App	云原生应用的十二条实践
无状态	Stateless	不在进程里保存需要记住的状态
有状态	Stateful	需要保存状态
粘性会话	Sticky Session	同一客户端总到同一实例
外部状态存储	External State Store	状态放托管存储
外部化配置	Externalised Configuration	配置和代码分离
不可变基础设施	Immutable Infrastructure	不在线改，整体替换
宠物与牛群	Pets vs Cattle	精心照顾 vs 随时替换
托管服务	Managed Service	云厂商运维的服务
厂商锁定	Vendor Lock-in	难以换云
多云	Multi-Cloud	同时用多家云
责任共担模型	Shared Responsibility Model	厂商管云本身，用户管云里面
IaaS / PaaS / SaaS	IaaS / PaaS / SaaS	基础设施 / 平台 / 软件即服务
渐进式 Web 应用	Progressive Web App (PWA)	可安装、可离线的网页应用
原生应用	Native App	深度访问硬件
客户端编排	Client-Side Orchestration	客户端组合多个 API
内容分发网络	Content Delivery Network (CDN)	就近分发静态内容
Web 应用防火墙	Web Application Firewall (WAF)	过滤恶意 Web 请求
扩展立方体	Scale Cube	X 克隆 / Y 功能拆分 / Z 数据分区
水平 / 垂直扩展	Horizontal / Vertical Scaling	加机器 / 加配置
康威定律	Conway's Law	系统结构复制组织沟通结构
迁移策略 7R	7R Migration Strategies	Rehost、Replatform、Rearchitect 等

github.com/bibi45c · 免费分享，禁止售卖

术语词典

容器与 Kubernetes

中文	English	一句话解释
容器	Container	隔离的进程，共享宿主内核
虚拟机	Virtual Machine (VM)	完整操作系统，硬件级隔离
虚拟机监控器	Hypervisor	运行多个虚拟机
镜像 / 镜像仓库	Image / Container Registry	打包好的应用 / 存镜像的地方
容器编排	Container Orchestration	自动部署、扩缩、自愈
集群	Cluster	控制平面 + 工作节点
控制平面	Control Plane	API server、etcd、调度器、控制器
工作节点	Worker Node	运行 Pod 的机器（云上是 EC2）
命名空间	Namespace	逻辑分组，不是机器
Pod	Pod	最小部署单元
副本集	ReplicaSet	保持 Pod 数量
部署对象	Deployment	管无状态 Pod、滚动更新
有状态集	StatefulSet	固定名字、自己的盘
守护进程集	DaemonSet	每节点一个 Pod
任务 / 定时任务	Job / CronJob	一次性 / 按时间表
服务（K8s）	Service	一组 Pod 的稳定地址 + 负载均衡
入口	Ingress	外部 HTTP 请求按路径分发
配置 / 密钥	ConfigMap / Secret	配置 / 敏感信息
持久卷 / 持久卷申领	PersistentVolume / PersistentVolumeClaim	存储 / 存储申请
水平 Pod 自动扩缩	Horizontal Pod Autoscaler (HPA)	改副本数
集群自动扩缩器	Cluster Autoscaler	加减节点
存活 / 就绪探针	Liveness / Readiness Probe	要不要重启 / 能不能接流量
标签选择器	Label Selector	Service 靠标签找 Pod
边车代理	Sidecar Proxy	Pod 旁的网络助手容器
服务网格	Service Mesh	管东西向流量（Istio）
节点组	Node Group	一批同规格节点
运维器	Operator	把运维知识编码管理有状态中间件
声明式 / 期望状态	Declarative / Desired State	描述目标，系统去达成
自愈	Self-Healing	Pod 挂了自动重建

数据与存储

中文	English	一句话解释
关系数据库	Relational Database (RDBMS)	表、SQL、ACID 事务
键值存储	Key-Value Store	按键快速查找
文档数据库	Document Database	JSON 文档、schema 灵活
宽列 / 列族数据库	Wide-Column / Column-Family Database	写密集、一主键多列
图数据库	Graph Database	擅长遍历关系
时序数据库	Time-Series Database (TSDB)	按时间存指标
搜索引擎	Search Engine	全文检索、日志分析
对象存储	Object Storage	图片、文件、备份
块存储 / 文件存储	Block / File Storage	磁盘卷 / 共享文件系统
数据仓库	Data Warehouse	列式存储做分析
数据湖	Data Lake	原始数据大量存放（S3）
内存缓存	In-Memory Cache	数据放内存，亚毫秒
读穿透 / 写穿透	Read-Through / Write-Through	缓存未命中自动读库 / 写时同步写库
读副本	Read Replica	只扩展读
主库	Primary	接受写入的库
分片	Sharding	按分片键把数据分到多台机器
分片键	Shard Key / Partition Key	决定数据去哪片
哈希分片 / 范围分片	Hash / Range Sharding	均匀 / 易热点
热点分片	Hot Shard	负载集中在一片
重新分片	Re-sharding	调整分片数量
垂直分区	Vertical Partitioning	按列或功能拆表
多语言持久化	Polyglot Persistence	不同数据用不同存储
每服务一个数据库	Database per Service	服务数据解耦
共享数据库	Shared Database	简单但耦合
ACID	ACID	原子性、一致性、隔离性、持久性
BASE	BASE	基本可用、软状态、最终一致
CAP 定理	CAP Theorem	网络分区时在一致性和可用性之间选
新型 SQL	NewSQL	SQL + 水平扩展（Spanner）
命令查询职责分离	CQRS	读模型和写模型分开
事件溯源	Event Sourcing	存事件序列而不是当前状态
多可用区部署	Multi-AZ Deployment	主备在不同可用区
数据脱敏 / 令牌化	Data Masking / Tokenisation	隐藏或替换敏感数据

github.com/bibi45c · 免费分享，禁止售卖

术语词典

通信与消息

中文	English	一句话解释
同步 / 异步	Synchronous / Asynchronous	等结果 / 不等结果
请求-响应	Request-Response	期待一个结果
单向通知	One-Way Notification	不期待响应
发布 / 订阅	Publish / Subscribe	一条消息多个订阅者都收到
点对点（消息）	Point-to-Point Messaging	一条消息一个消费者
消息队列	Message Queue	投递后删除，竞争消费
事件流	Event Streaming	有序、持久、可重放
分区	Partition	事件流里并行的"车道"
消费者组	Consumer Group	组内一人处理，组间各一份
消息代理	Message Broker	生产者和消费者之间的中间件
至少一次投递	At-Least-Once Delivery	可能重复
幂等接收者	Idempotent Receiver	重复收到结果不变
削峰	Peak Shaving / Load Levelling	用队列吸收峰值
编排 / 协同	Orchestration / Choreography	中心指挥 / 各自订阅事件
Saga 模式	Saga Pattern	本地事务 + 补偿事务
两阶段提交	Two-Phase Commit (2PC)	准备 + 提交的分布式事务
服务注册中心	Service Registry	记录服务实例位置
服务发现	Service Discovery	客户端发现 / 服务端发现
东西向 / 南北向流量	East-West / North-South Traffic	服务之间 / 用户与平台
通信亲和度	Communication Affinity	两服务间消息数量和大小
共置	Collocation	多个服务放一个容器
智能端点哑管道	Smart Endpoints and Dumb Pipes	逻辑在服务，管道简单

性能、容量与测试

中文	English	一句话解释
瓶颈	Bottleneck	限制整体吞吐的组件
服务需求	Service Demand	访问次数 × 服务时间
利用率定律	Utilisation Law	U = λ × S
Little 定律	Little's Law	n = λ × R
串行 / 并行	Serial / Parallel	时间相加吞吐取最小 / 时间取最长吞吐相加
负载均衡	Load Balancing	把请求分到多台
预计算	Precomputation	先算好存起来
扇出	Fan-out	一次写入推给很多读者
负载测试	Load Testing	恒定负载
压力测试	Stress Testing	阶梯递增到极限
突发测试	Spike Testing	突然涌入
耐久测试	Soak / Endurance Testing	长时间稳定负载
工作负载模型	Workload Model	曲线、用户与到达率、场景组合、通过标准
场景组合	Workload Mix	各用例占比
思考时间	Think Time	用户两次操作的间隔
施压机	Load Generator	发出模拟负载的机器
通过标准	Pass Criteria	如 P90 < 3 s
基准测试	Benchmark	标准测试比较性能
容量规划	Capacity Planning	预测并准备资源
趋势分析	Trend Analysis	用历史数据预测
阿姆达尔定律	Amdahl's Law	串行部分限制加速上限
通用扩展定律	Universal Scalability Law (USL)	加串扰，曲线先升后降
冷却时间	Cooldown Period	两次扩缩的最短间隔
乒乓效应	Ping-Pong Effect	扩了又缩、缩了又扩
余量	Headroom	为增长预留的容量
采购提前期	Procurement Lead Time	申请到可用的时间

github.com/bibi45c · 免费分享，禁止售卖

术语词典

可用性、韧性与发布

中文	English	一句话解释
单点故障	Single Point of Failure (SPOF)	一坏全坏的组件
冗余	Redundancy	多份部件
主动 / 被动冗余	Active / Passive Redundancy	热备 / 温备
故障转移	Failover	切到备用
心跳	Heartbeat	定期存活信号
健康检查	Health Check	判断实例是否正常
优雅降级	Graceful Degradation	出错时提供部分功能
超时	Timeout	等太久就放弃
指数退避重试	Retry with Exponential Backoff	等待时间成倍增加
重试风暴	Retry Storm	大量重试压垮下游
熔断器	Circuit Breaker	关闭 → 打开 → 半开
舱壁	Bulkhead	分区隔离，限制故障范围
减载	Load Shedding	主动丢弃超额请求
背压	Backpressure	下游让上游放慢
级联故障	Cascading Failure	故障沿调用链传播
缓存踩踏	Dogpile / Cache Stampede	重启后同时回源压垮数据库
吵闹邻居	Noisy Neighbour	共享硬件被别人抢资源
混沌工程	Chaos Engineering	在生产中做故障实验
蓝绿部署	Blue/Green Deployment	两套环境一键切换
滚动更新	Rolling Update	逐台替换
金丝雀发布	Canary Release	新版本先接少量流量
影子流量	Shadowing	同时发新旧，只用旧结果
回滚	Rollback	退回上一个版本
先扩后缩	Expand–Migrate–Contract	不兼容表结构变更的分步做法
墓碑标记	Tombstone	删除时打标记而不是真删
RAID	RAID	磁盘冗余阵列（0、1、5、6、10）
完全 / 差异 / 增量备份	Full / Differential / Incremental Backup	全部 / 自上次完全 / 自上次任意
适应度函数	Fitness Function	自动检查架构规则

安全

中文	English	一句话解释
保密性 / 完整性 / 可用性	Confidentiality / Integrity / Availability (CIA)	信息安全三目标
真实性	Authenticity	确认是本人或真实来源
不可否认	Non-Repudiation	做过的事不能抵赖
认证 / 授权 / 审计	Authentication / Authorisation / Accounting (AAA)	你是谁 / 能做什么 / 做了什么
威胁 / 漏洞 / 风险	Threat / Vulnerability / Risk	T + V = R
资产	Asset	要保护的东西
皇冠明珠	Crown Jewels	最关键的数据
跳板	Stepping Stones	被用来攻击关键资产的系统
威胁分类	STRIDE	仿冒、篡改、抵赖、泄露、拒绝服务、提权
风险打分	DREAD	可能性和影响打分
风险处置	Risk Treatment	缓解、规避、转移、接受
纵深防御	Defence in Depth	多层控制
最小权限	Least Privilege	最少权限、最短时间
完全仲裁	Complete Mediation	每次访问都检查
设计即安全	Secure by Design	需求阶段就纳入安全
传输层安全	Transport Layer Security (TLS)	加密传输
双向 TLS	Mutual TLS (mTLS)	双方互相验证证书
静态 / 传输中加密	Encryption at Rest / in Transit	存储时 / 传输时加密
密钥管理服务	Key Management Service (KMS)	管理和轮换密钥
硬件安全模块	Hardware Security Module (HSM)	硬件保存长期密钥
身份与访问管理	Identity and Access Management (IAM)	谁能访问什么
单点登录	Single Sign-On (SSO)	一次登录多系统通用
入侵检测 / 防御	IDS / IPS	发现 / 阻止入侵
分布式拒绝服务	Distributed Denial of Service (DDoS)	大量流量打垮服务
通用数据保护条例	General Data Protection Regulation (GDPR)	欧盟个人数据法规
支付卡行业数据安全标准	PCI-DSS	支付和持卡数据要求
租户专属密钥	Tenant-Specific Key	每个租户自己的加密密钥

github.com/bibi45c · 免费分享，禁止售卖

术语词典

可观测性与指标

中文	English	一句话解释
可观测性	Observability	指标、日志、追踪三件套
指标	Metrics	随时间变化的数字
日志	Logs	事件的文本记录
追踪	Traces	一个请求经过的服务和耗时
跟踪 ID / 关联 ID	Trace ID / Correlation ID	串起一个请求
跨度	Span	追踪里的一个工作单元
集中式日志	Centralised Logging	ELK / EFK
仪表盘	Dashboard	Grafana、Kibana
告警	Alert	超阈值通知
应用指标	Application Metric	任务成功、清晰度、满意度
基础设施指标	Infrastructure Metric	CPU、内存、请求量
派生（计算）指标	Derived Metric	由基础指标计算
依赖指标	Dependent / Base Metric	计算用到的原始数据
保存时长	Retention Period	数据留多久
流处理任务	Stream-Processing Job	实时计算（Kinesis + Lambda）

AWS 常用产品：名字 → 干什么

中文	English	一句话解释
虚拟机	Amazon EC2	计算实例，K8s 的节点
托管 Kubernetes	Amazon EKS	跑容器化微服务
容器服务 / 无服务器容器	Amazon ECS / AWS Fargate	AWS 自有容器编排 / 不管机器
函数	AWS Lambda	事件触发、按调用付费
应用托管 PaaS	AWS Elastic Beanstalk	上传代码自动部署
API 网关	Amazon API Gateway	统一入口、限流、认证
负载均衡	Elastic Load Balancing (ALB / NLB)	分发流量
CDN	Amazon CloudFront	就近分发
DNS	Amazon Route 53	域名解析、故障转移
Web 防火墙	AWS WAF / AWS Shield	挡恶意请求 / 防 DDoS
对象存储	Amazon S3	文件、媒体、数据湖
关系数据库	Amazon RDS / Amazon Aurora	事务、多可用区
键值数据库	Amazon DynamoDB	海量写、毫秒级
文档数据库	Amazon DocumentDB	MongoDB 兼容
宽列数据库	Amazon Keyspaces	Cassandra 兼容
内存缓存	Amazon ElastiCache (Redis)	亚毫秒读、计数、会话
图数据库	Amazon Neptune	关系遍历
时序数据库	Amazon Timestream	指标、遥测
搜索	Amazon OpenSearch Service	全文检索、日志
数据仓库	Amazon Redshift	历史分析
SQL 查文件	Amazon Athena	直接查 S3
大数据集群	Amazon EMR	批处理分析
消息队列	Amazon SQS	任务分发、削峰
推送 / 发布订阅	Amazon SNS	短信、推送、多订阅者
事件总线	Amazon EventBridge	事件路由、定时调度
事件流	Amazon Kinesis Data Streams	实时摄取、分片有序
托管 Kafka	Amazon MSK	Kafka 事件流
工作流	AWS Step Functions	编排多步流程
物联网接入	AWS IoT Core	设备数据接入
机器学习	Amazon SageMaker	训练和推理
生成式 AI	Amazon Bedrock	调用大模型
用户认证	Amazon Cognito	注册登录
权限 / 密钥	AWS IAM / AWS Secrets Manager / AWS KMS	访问控制 / 存密码 / 管密钥
监控	Amazon CloudWatch	指标、日志、告警
分布式追踪	AWS X-Ray	调用链
镜像仓库	Amazon ECR	存容器镜像
基础设施即代码	AWS CloudFormation / AWS CDK	用代码建资源

github.com/bibi45c · 免费分享，禁止售卖

PART 5 速查与索引

索引：题干关键词 · 中文拼音 · 英文 A–Z

题干关键词索引：看到这个英文词 → 是什么题 → 翻到哪页

读题时遇到下面的词，按字母顺序找，直接翻到题型卡或速查表对应的页。数字开头的放在最前面。

题干里的词	中文意思	题型 / 用途	页
200 / 201 / 202 / 204	状态码	T3；201 做完了，202 收下了	p.69
409 Conflict, 429	状态冲突、超过限流	T3 Level 3	p.69
4+1 views	4+1 视图	架构基础	p.72
7R, rehost, replatform	上云迁移策略	T17	p.42
99.9%, nines	几个 9	T13 可用性	p.68
accuracy, prediction	准确度、预测	T8 指标（MAE）	p.33
ACID, BASE, CAP	事务特性、最终一致、CAP	T6 / T11 选存储	p.75
actuator, IoT sensor	执行器、物联网传感器	T10 原生 vs PWA	p.35
added, removed, changed	新增、删除、修改	T5 改造本地架构	p.23
aggregate (red circle)	聚合（图上红圈）	T1 诊断	p.16
aggregate root, invariant	聚合根、不变量	T1 数据不一致	p.73
alternatives	替代选择	T6 托管 ML	p.51
anticorruption layer, conformist	防腐层、遵奉者	DDD 上下文映射	p.73
API Gateway	API 网关	T5 组件	p.23
application metric	应用指标	T8	p.33
arrival rate, virtual users	到达率、虚拟用户	T11 测试	p.68
assume, assumption	假设	写一句前提	p.69
asynchronous, 202 Accepted	异步	T3 性能重构	p.21
at-least-once, idempotent receiver	至少一次、幂等接收	消息可靠性	p.76
auto-scaling, threshold	自动扩缩、阈值	T5 / T11（60–70%）	p.68
availability, downtime	可用性、停机	T13	p.40
availability zone, Multi-AZ	可用区、多可用区	T5 K8s / T6	p.24
backup (full / differential / incremental)	备份（完全 / 差异 / 增量）	第 13 章；恢复要哪几份	p.66
backward compatible, breaking change	向后兼容、不兼容修改	T3 版本	p.20
batch endpoint	批量接口	T3 性能	p.21
blue/green, rolling, canary	部署策略	T17	p.42
bottleneck	瓶颈	T11	p.38
boundary, control, entity	边界、控制、实体	T4 分层	p.13
bounded context	限界上下文	T1 / T4 论证	p.16
bulkhead, backpressure	舱壁、背压	T17	p.77
cache, hit ratio	缓存、命中率	T11	p.38
capacity, sizing, growth	容量、规模、增长	T14	p.41
CDN, CloudFront	内容分发网络	T5 / T6	p.27
circuit breaker, cascading failure	熔断器、级联故障	T17	p.42
class diagram, UML, DDD	类图	T2	p.18
client-side orchestration	客户端编排	T10 后端影响	p.46
cohesion, coupling	内聚、耦合	T4 / T1 论证	p.72
cold start	冷启动	T10 微服务 vs Serverless	p.35
collocate, same container instance	放进同一个容器	T12	p.36
communication affinity	通信亲和度	T12	p.36
complete the packaging	补全打包	T4	p.13
composition, inheritance, multiplicity	组合、继承、多重性	T2 类图符号	p.18
concrete example, illustrate	具体例子、举例说明	T3（方法+URI+体+码）	p.21
concurrent users	并发用户	引用 §4 数字	p.67
consistency, invariant, rule	一致性、不变量、规则	T1 数据不一致	p.16
container vs virtual machine	容器 vs 虚拟机	T10	p.35
countermeasure	对策	T1	p.16
CQRS, event sourcing	读写分离、事件溯源	第 9 章；T11 补充句	p.65
CronJob, scheduled job	定时任务	T5	p.23
cross-shard aggregation	跨分片汇总	T11 副作用	p.38

github.com/bibi45c · 免费分享，禁止售卖

题干里的词	中文意思	题型 / 用途	页
crucial architectural aspect	最关键的架构方面	T9	p.**31**
DaemonSet, StatefulSet	K8s 对象	T5 (iv)	p.**24**
database per service	每服务一个库	T5 / T6	p.**75**
deliberately omitted	刻意省略	T4 不算错	p.**13**
deploy, deployment link	部署、部署连线	T4	p.**13**
deterministic and repeatable	确定且可重复	T10 事件流	p.**35**
Do you agree...?	你同意吗	先表态再给理由	p.**5**
DMZ, reverse proxy	非军事区、反向代理	T4 区域规则	p.**13**
domain event	领域事件	T1 副本同步	p.**16**
domain model, `<<service>>`	领域模型、服务框	T1	p.**16**
drawback, side effect	缺点、副作用	没有也写 no major drawback	p.**5**
DynamoDB	键值数据库	T6 写多	p.**27**
elasticity	弹性	T7 / T11	p.**72**
ElastiCache, Redis	内存缓存	T6 读快 / T5 状态	p.**27**
encapsulate, lift and shift	封装、直接搬迁	迁移；预备卡 C 补充	p.**10**
error rate	错误率	通过标准 < 0.1%	p.**67**
event streaming vs message queue	事件流 vs 消息队列	T10	p.**35**
eventual consistency	最终一致	T1 缺点	p.**16**
expand–migrate–contract, schema change	表结构变更	T17	p.**42**
expose ... as API to ...	把……作为 API 开放给……	T9	p.**31**
External Integration layer	外部集成层	T4 清单 ⑥	p.**13**
GDPR, PCI-DSS	数据保护、支付安全标准	T16 / PAST3	p.**77**
generic subdomain	通用域	T1 复用 / T2 共享	p.**18**
GET vs POST vs PUT vs PATCH	HTTP 方法	T3	p.**20**
HATEOAS, links, rel, href	超媒体链接	T3 Level 3	p.**21**
hot shard	热点分片	T11 副作用	p.**38**
HPA, Cluster Autoscaler	Pod 扩缩、节点扩缩	T5 (iv)	p.**24**
hybrid cloud, NIST	混合云、云的五特征	第 7 章	p.**64**
hybrid, serverless, scheduled jobs	混合、无服务器、定时任务	T5	p.**23**
IaaS, CaaS, PaaS, FaaS, SaaS	云服务模型	第 7 章；预备卡 F	p.**64**
idempotent, safe method	幂等、安全方法	T3	p.**20**
identify	指出	点名具体对象	p.**5**
in-process vs inter-process	进程内 vs 跨进程	T1 慢 / T12	p.**69**
Ingress, Service (K8s)	入口、服务	T5 (iv)	p.**24**
interaction, seed, producer, consumer	交互、种子、生产者、消费者	T9	p.**31**
internal communication	内部通信	T5 (ii)	p.**23**
IoT Core, Kinesis	设备接入、事件流	T6 / PAST2	p.**51**
ISO 25010, utility tree	质量模型、效用树	T15	p.**41**
JMeter, load generator	负载测试工具、施压机	T11	p.**38**
justify	论证	引用 §4 数字	p.**5**
Kubernetes deployment plan	K8s 部署计划	T5 (iv)	p.**24**
Lambda (AWS), function	函数	T5 / T6 事件驱动	p.**27**
latency of the API, concurrent updates	API 延迟、并发更新	T3 性能	p.**21**
latency numbers	延迟量级	跨服务为什么慢	p.**69**
Little's Law	并发 = 到达率 × 响应时间	T11 / T14	p.**68**
load test, performance test, Postman	负载测试、性能测试	T11	p.**38**
local storage, mobile device	本地存储、手机	T4 清单 ⑤	p.**13**
logical detailed deployment	逻辑详细部署图	T4（四个维度）	p.**13**
managed service	托管服务	T6 / T10	p.**35**
manifest	打包（包含）连线	T4	p.**13**
memory, footprint	内存、占用	T1 聚合太大	p.**16**
metric, measure and track	指标、衡量和跟踪	T8	p.**33**
microservices vs monolith	微服务 vs 单体	T5 / T10	p.**23**
migrate to the cloud	上云迁移	T17	p.**42**
MLTTP, onboard unit	公交遥测案例	PAST2	p.**48**
monolith, MVP prototype	单体、最小原型	T5 改造	p.**23**

github.com/bibi45c · 免费分享，禁止售卖

题干里的词	中文意思	题型 / 用途	页
MTBF, MTTR, RTO, RPO	故障间隔、恢复时间	T13	p.67
multi-cloud	多云	T5 (i) 额外组件	p.23
multitenancy, tenant	多租户、租户	T9（PAST3）	p.54
namespace, pod, node	K8s 层次	T5 (iv)	p.75
native app vs PWA	原生 vs 渐进式网页应用	T10	p.35
non-functional requirement	非功能需求	T7	p.30
notification	通知	T1 复用 / T5 Serverless	p.44
OAuth 2.0, JWT	授权、令牌	T3 收尾 / T16	p.74
on-premises, legacy architecture	本地部署、传统架构	T5 改造	p.23
ONE Health, prescription, supplier	医疗健身案例	PAST3	p.54
orchestration vs choreography	编排 vs 协同	T10	p.35
P90, P99, percentile	百分位	通过标准	p.67
partition key	分区键	T3 / T11	p.21
physical deployment diagram	物理部署图	T6	p.27
polyglot persistence	多语言持久化	T6 决策 3	p.27
precomputation	预计算	T11	p.38
promptly, timely, quickly	及时、快速	T1 慢	p.16
proctor, polling room, Vote Anywhere	投票案例	PAST	p.43
public / private subnet, VPC	公有 / 私有子网	T6 物理图	p.27
quality attribute scenario	质量属性场景	T7 / T15	p.30
quality attributes	质量属性	T7	p.30
queue length, stream lag	队列长度、积压	T10 扩展决策	p.67
RDS, Aurora	关系数据库	T6 事务	p.27
read replica	只读副本	T11 分片	p.38
read-only copy	只读副本（领域）	T1 慢	p.16
refactor the API, outside-in	重构 API、由外而内	T3	p.21
reliability vs availability	可靠性 vs 可用性	第 13 章；T13 补充句	p.66
response time, throughput	响应时间、吞吐量	T11	p.67
retention, frequency, location	保存时长、频率、位置	T8 七字段	p.33
retry, exponential backoff	重试、指数退避	T17	p.42
reuse, other platforms, drastic adaptation	复用、其他平台、大改	T1 难以复用	p.16
RMM, Richardson, Level 2 / 3	成熟度模型	T3	p.20
Saga, compensating transaction	Saga、补偿事务	T17	p.42
SageMaker, managed ML	托管机器学习	T6	p.51
scale cube, X / Y / Z axis	扩展立方体	第 9 章；T11 补充句	p.65
scale independently	独立扩展	T1	p.16
semantic versioning, MAJOR.MINOR.PATCH	语义化版本	T3	p.20
service mesh, Istio, sidecar	服务网格	T5 通信组件	p.23
service registry, discovery	服务注册、发现	T5	p.23
shard, partition	分片、分区	T11	p.38
shared services	共享服务	T2	p.18
stateful, state management	有状态、状态管理	T5 (iii)	p.23
sticky session	粘性会话	T5（别用）	p.23
STRIDE, threat, vulnerability	威胁、漏洞	T16	p.42
subsystem	子系统（服务名 + 层名）	T4	p.13
synchronous vs asynchronous	同步 vs 异步	T10 / T3	p.35
think time, workload mix	思考时间、场景组合	T11	p.38
throughput	吞吐量	T11	p.38
undesirable architectural decision	不理想的架构决策	T4	p.13
usability	易用性	T7	p.30
utilisation	利用率	T11（R = S/(1−U)）	p.68
versioning, deprecated, Accept-version	版本、废弃、版本请求头	T3	p.20
workload model	工作负载模型	T11	p.38
write throughput	写吞吐	T11 分片	p.38
yield, harvest, uptime	产出率、收获率、可用时间	T8 / T13	p.67
zero downtime, rollback	零停机、回滚	T17	p.42

github.com/bibi45c · 免费分享，禁止售卖

题干里的词	中文意思	题型 / 用途	页
zone: Internet, DMZ, Internal	区域	T4 清单 ⑦	p.13

中文术语索引（按拼音首字母）

中文 → 英文 → 页。按拼音首字母分组（个别生僻字归到 # 组）。

#

4+1 视图 4+1 View Model **72**

A

ACID ACID **75**
API 网关 Amazon API Gateway **78**
阿姆达尔定律 Amdahl's Law **76**
安全方法 Safe Method **74**
按标识引用 Reference by Identity **73**

B

BASE BASE **75**
百分位 Percentile (P50 / P90 / P95 / P99) **67**
保存时长 retention period / Retention Period **61,78**
保密性 / 完整性 / 可用性 Confidentiality / Integrity / Availability (CIA) **77**
背压 Backpressure **77**
边车代理 Sidecar Proxy **75**
边界 / 控制 / 实体 Boundary / Control / Entity (BCE) **72**
编排 / 协同 Orchestration / Choreography **76**
标签选择器 Label Selector **75**
表现层 Presentation Layer **72**
并发数 Concurrency **67**
并发用户 concurrent users **61**
不兼容的修改 breaking change / Breaking Change **61,74**
不可变基础设施 Immutable Infrastructure **74**
不可否认 Non-Repudiation **77**
不理想的 undesirable **71**
不完整的 incomplete **71**
部分响应 Partial Response **74**
部署 Deploy **72**
部署单元 Deployment Unit **72**
部署对象 Deployment **75**

C

CAP 定理 CAP Theorem **75**
CDN Amazon CloudFront **78**
采购提前期 Procurement Lead Time **76**
参考架构 Reference Architecture **72**
舱壁 Bulkhead **77**
仓储 Repository **73**
查询参数 Query Parameter **74**
产出率 Yield **67**
场景 scenario **71**
场景组合 Workload Mix **76**
厂商锁定 Vendor Lock-in **74**
超媒体 Hypermedia (HATEOAS) **74**
超媒体链接 hypermedia link **61**
超时 Timeout **77**
超文本引用 href **74**
吵闹邻居 Noisy Neighbour **77**
持久卷 / 持久卷申领 PersistentVolume / PersistentVolumeClaim **75**
持续集成 / 持续交付 CI / CD **73**
充分地 adequately **71**
宠物与牛群 Pets vs Cattle **74**
抽象 / 封装 Abstraction / Encapsulation **72**
传输层安全 Transport Layer Security (TLS) **77**
串行 / 并行 Serial / Parallel **76**
垂直分区 Vertical Partitioning **75**
磁铁 Magnet **73**
刺激源 / 刺激 Source / Stimulus **72**
促进、便于 facilitate **71**
存活 / 就绪探针 Liveness / Readiness Probe **75**
撮合者 Matchmaker **73**
错误率 Error Rate **67**

D

DNS Amazon Route 53 **78**
DORA 指标 DORA Metrics **68**
打包 Packaging **72**
大幅的 drastic **71**
大泥球 Big Ball of Mud **73**
大数据集群 Amazon EMR **78**
单点登录 Single Sign-On (SSO) **77**
单点故障 Single Point of Failure (SPOF) **77**
单体 Monolith **74**
单向通知 One-Way Notification **76**
单一职责原则 Single Responsibility Principle (SRP) **72**
弹性 Elasticity **72**
到达率 Arrival Rate (λ) **67**
等待 / 排队时间 Waiting / Queueing Time **67**
第 90 百分位 90th percentile (P90) **61**
点对点（消息） Point-to-Point Messaging **76**
点对点网络 Peer-to-Peer (P2P) **72**
调用-返回 Call-Return **72**
东西向 / 南北向流量 East-West / North-South Traffic **76**
读穿透 / 写穿透 Read-Through / Write-Through **75**
读副本 Read Replica **75**
端点 Endpoint **74**
队列长度 Queue Length **68**
对象存储 Object Storage / Amazon S3 **75,78**
多可用区部署 Multi-AZ Deployment **75**
多语言持久化 polyglot persistence / Polyglot Persistence **61,75**
多云 Multi-Cloud **74**
多重性 Multiplicity **73**
多租户 Multitenancy **73**

F

发布 / 订阅 Publish / Subscribe **76**
反向代理 Reverse Proxy **72**
泛化（继承） Generalisation (Inheritance) **73**
防腐层 Anticorruption Layer **73**
防火墙 Firewall **72**
非功能需求 Non-Functional Requirement (NFR) **72**
非军事区 Demilitarized Zone (DMZ) **72**
分别 respectively **71**
分布式拒绝服务 Distributed Denial of Service (DDoS) **77**
分布式追踪 AWS X-Ray **78**
分层架构 Layered / N-Tier Architecture **72**
分片 Sharding **75**
分片键 Shard Key / Partition Key **75**
分区 Partition **76**
分页 Pagination **74**
风险处置 Risk Treatment **77**
风险打分 DREAD **77**
服务（K8s） Service **75**
服务等级目标 / 协议 SLO / SLA **68**
服务发现 Service Discovery **76**
服务时间 Service Time **67**
服务网格 Service Mesh **75**
服务需求 Service Demand **76**
服务注册中心 Service Registry **76**
副本集 ReplicaSet **75**
复制延迟 Replication Lag **68**
负载测试 Load Testing **76**
负载均衡 Load Balancing / Elastic Load Balancing (ALB / NLB) **76,78**

G

GitOps GitOps **73**
概念验证 Proof of Concept (PoC) **72**
高峰负载 peak load **60**
高内聚 / 低耦合 High Cohesion / Low Coupling **72**
告警 Alert **78**
跟踪 ID / 关联 ID Trace ID / Correlation ID **78**
工具箱 Toolbox **73**
工作负载模型 workload model / Workload Model **61,76**
工作节点 Worker Node **75**
工作流 AWS Step Functions **78**
公有 / 私有子网 public / private subnet **61**
共享服务 Shared Service **73**
共享库层 Shared Library Layer **72**
共享内核 Shared Kernel **73**
共享数据库 Shared Database **75**
共置 Collocation **76**
故障转移 Failover **77**
关联 / 聚合 / 组合 Association / Aggregation / Composition **73**
关系数据库 Relational Database (RDBMS) / Amazon RDS / Amazon Aurora **75,78**
关注点分离 Separation of Concerns **72**
规模估算 Sizing **72**
滚动更新 Rolling Update **77**

H

哈希分片 / 范围分片 Hash / Range Sharding **75**
函数 AWS Lambda **78**
函数即服务 Function as a Service (FaaS) **74**
核心域 / 支撑域 / 通用域 Core / Supporting / Generic Subdomain **73**
合适的 appropriate / suitable **71**
互操作性 Interoperability **72**
缓存踩踏 Dogpile / Cache Stampede **77**
缓存命中率 Cache Hit Ratio **68**
缓解 mitigate **71**
黄金路径 Golden Path **73**
皇冠明珠 Crown Jewels **77**
恢复点目标 Recovery Point Objective (RPO) **68**
恢复时间目标 Recovery Time Objective (RTO) **68**
回调 Webhook **74**
回滚 Rollback **77**
混合架构 Hybrid Architecture **74**
混沌工程 Chaos Engineering **77**

I

IaaS / PaaS / SaaS IaaS / PaaS / SaaS **74**

J

JSON 令牌 JSON Web Token (JWT) **74**
基础设施即代码 Infrastructure as Code (IaC) / AWS CloudFormation / AWS CDK **73,78**
基础设施指标 Infrastructure Metric **78**
基准测试 Benchmark **76**
机器学习 Amazon SageMaker **78**
集成端点 Integration Endpoint **72**
集群 Cluster **75**
集群自动扩缩器 Cluster Autoscaler **75**
集中式日志 Centralised Logging **78**
级联故障 Cascading Failure **77**
假设 assume / assumption **71**

github.com/bibi45c · 免费分享，禁止售卖

架构风格 Architectural Style **72**
架构决策记录 Architecture Decision Record **72**
架构上重要的需求 Architecturally Significant Requirement (ASR) **72**
监控 Amazon CloudWatch **78**
简要地 briefly **71**
减载 Load Shedding **77**
键值存储 Key-Value Store **75**
键值数据库 Amazon DynamoDB **78**
健康检查 Health Check **77**
渐进式 Web 应用 Progressive Web App (PWA) **74**
交互 Interaction **73**
节点 Node **72**
节点组 Node Group **75**
解决方案架构 Solution Architecture (SA) **72**
借此 whereby **71**
金丝雀发布 Canary Release **77**
进程内调用 in-process call **60**
静态 / 传输中加密 Encryption at Rest / in Transit **77**
镜像 / 镜像仓库 Image / Container Registry **75**
镜像仓库 Amazon ECR **78**
就……而言 with respect to / in terms of **71**
聚合 Aggregate **73**
聚合根 Aggregate Root **73**
具体的 concrete **71**

K

开放 API 规范 OpenAPI Specification **74**
开放主机服务 Open Host Service **73**
康威定律 Conway's Law **74**
考虑因素 consideration **71**
可测试性 Testability **72**
可分析性 Analyzability **72**
可复用服务 / 框架 / 共享库 Reusable Service / Framework / Shared Library **73**
可观测性 Observability **78**
可靠性 Reliability **72**
可扩展性 Scalability **72**
可维护性 Maintainability **72**
可行的 feasible / viable **71**
可修改性 Modifiability **72**
可演进性 Evolvability **72**
可移植性 Portability **72**
可用区 availability zone **61**
可用时间 Uptime **67**
可用性 Availability **67,72**
刻意省略 deliberately omitted **71**
客户端-服务器 Client-Server **72**
客户端编排 client-side orchestration / Client-Side Orchestration **61,74**
控制层 Control Layer **72**
控制平面 Control Plane **75**
库 Library **73**
跨度 Span **78**
跨进程通信 inter-process communication **60**
跨域资源共享 CORS **74**
块存储 / 文件存储 Block / File Storage **75**
宽列 / 列族数据库 Wide-Column / Column-Family Database **75**
宽列数据库 Amazon Keyspaces **78**
框架 Framework **73**
扩展立方体 Scale Cube **74**

L

Lambda / Kappa 架构 Lambda / Kappa Architecture **74**
Little 定律 Little's Law **76**
蓝绿部署 Blue/Green Deployment **77**
类图 Class Diagram **73**
冷启动 Cold Start **74**
冷却时间 Cooldown Period **76**
理查森成熟度模型 Richardson Maturity Model (RMM) **74**
理由 rationale **71**
利益相关者 stakeholder **71**
利用 leverage **71**
利用率 Utilisation **67**
利用率定律 Utilisation Law **76**
粒度 Granularity **74**
链接关系 rel (link relation) **74**
两阶段提交 Two-Phase Commit (2PC) **76**
领域模型 Domain Model **73**
领域实体 Domain Entity **73**
领域事件 domain event / Domain Event **60,73**
流处理任务 Stream-Processing Job **78**
六边形架构 Hexagonal Architecture **74**
路径参数 Path Parameter **74**
论证 / 理由 justify / justification **71**
逻辑架构 Logical Architecture **72**

M

每服务一个数据库 Database per Service **75**
每秒事务数 / 查询数 TPS / QPS **67**
门户服务器 Portal Server **72**
密钥管理服务 Key Management Service (KMS) **77**
幂等 Idempotent **74**
幂等键 Idempotency Key **74**
幂等接收者 Idempotent Receiver **76**
面向服务架构 Service-Oriented Architecture (SOA) **72**
面向前端的后端 Backend for Frontend (BFF) **74**
描述 / 概述 describe / outline **71**
命令查询职责分离 CQRS **75**
命名空间 Namespace **75**
墓碑标记 Tombstone **77**

N

耐久测试 Soak / Endurance Testing **76**
内部开发者平台 Internal Developer Platform (IDP) **73**
内部区 Internal Zone **72**
内存缓存 In-Memory Cache / Amazon ElastiCache (Redis) **75,78**
内容分发网络 Content Delivery Network (CDN) **74**
内容协商 Content Negotiation **74**

P

Pod Pod **75**
派生（计算）指标 Derived Metric **78**
配置 / 密钥 ConfigMap / Secret **75**
批量接口 Batch Endpoint **74**
贫血模型 Anemic Domain Model **73**
乒乓效应 Ping-Pong Effect **76**
平均故障间隔 MTBF **67**
平均恢复时间 MTTR **67**
平均绝对误差 Mean Absolute Error (MAE) **68**
平均无故障时间 MTTF **67**
平台 Platform **73**
平台服务层 Platform Service Layer **72**
平台工程 Platform Engineering **73**
瓶颈 Bottleneck **76**
评估 evaluate **71**

Q

企业服务总线 Enterprise Service Bus (ESB) **72**
企业架构 Enterprise Architecture (EA) **72**
迁移策略 7R 7R Migration Strategies **74**
强一致性 Strong Consistency **73**
请求-响应 Request-Response **76**
请求体 / 响应体 Request / Response Body **74**
请求头 Request Header **74**
趋势分析 Trend Analysis **76**
取舍 trade-off **71**
权限 / 密钥 AWS IAM / AWS Secrets Manager / AWS KMS **78**
确保 ensure **71**

R

RAID RAID **77**
热点分片 hot shard / Hot Shard **61,75**
任务 / 定时任务 Job / CronJob **75**
认证 / 授权 / 审计 Authentication / Authorisation / Accounting (AAA) **77**
日志 Logs **78**
熔断器 Circuit Breaker **77**
容量规划 Capacity Planning **76**
容器 Container **75**
容器编排 Container Orchestration **75**
容器服务 / 无服务器容器 Amazon ECS / AWS Fargate **78**
冗余 Redundancy **77**
入口 Ingress **75**
入侵检测 / 防御 IDS / IPS **77**
软件架构 Software Architecture **72**

S

Saga 模式 Saga Pattern **76**
SQL 查文件 Amazon Athena **78**
扇出 Fan-out **76**
上下文映射 Context Mapping **73**
设计即安全 Secure by Design **77**
身份与访问管理 Identity and Access Management (IAM) **77**
声明式 / 期望状态 Declarative / Desired State **75**
生产者 / 消费者 Producer / Consumer **73**
生成式 AI Amazon Bedrock **78**
生态系统 Ecosystem **73**
施压机 Load Generator **76**
十二要素应用 Twelve-Factor App **74**
时序数据库 Time-Series Database (TSDB) / Amazon Timestream **75,78**
事件流 Event Streaming / Amazon Kinesis Data Streams **76,78**
事件流积压 Consumer Lag **68**
事件驱动架构 Event-Driven Architecture (EDA) **74**
事件溯源 Event Sourcing **75**
事件总线 Amazon EventBridge **78**
事务范围 transaction scope **60**
适应度函数 Fitness Function **77**
收获率 Harvest **67**
首字节时间 Time to First Byte (TTFB) **67**
守护进程集 DaemonSet **75**
授权框架 OAuth 2.0 **74**
数据仓库 Data Warehouse / Amazon Redshift **75,78**
数据访问层 Data Access Layer **72**
数据湖 Data Lake **75**
数据脱敏 / 令牌化 Data Masking / Tokenisation **75**
双向 TLS Mutual TLS (mTLS) **77**
双向长连接 WebSocket **74**
水平 / 垂直扩展 Horizontal / Vertical Scaling **74**
水平 Pod 自动扩缩 Horizontal Pod Autoscaler (HPA) **75**
思考时间 Think Time **76**
搜索 Amazon OpenSearch Service **78**
搜索引擎 Search Engine **75**

T

提出 propose **71**
体现（包含） Manifest **72**
替代方案 alternative **71**
跳板 Stepping Stones **77**
通过标准 Pass Criteria **76**
通信亲和度 communication affinity / Communication Affinity **61,76**
通用服务 generic service **60**
通用扩展定律 Universal Scalability Law (USL) **76**
通用数据保护条例 General Data Protection Regulation (GDPR) **77**
通用语言 Ubiquitous Language **73**
同步 / 异步 Synchronous / Asynchronous **76**
统一资源标识符 URI **74**
突发测试 Spike Testing **76**

github.com/bibi45c · 免费分享，禁止售卖

图查询 API GraphQL **74**
图示 / 举例说明 illustrate **71**
图数据库 Graph Database / Amazon Neptune **75,78**
推导出 derive / derived **71**
推荐 recommend **71**
推送 / 发布订阅 Amazon SNS **78**
吞吐量 Throughput **67**
托管 Kafka Amazon MSK **78**
托管 Kubernetes Amazon EKS **78**
托管服务 Managed Service **74**

U

URI 版本 / 请求头版本 URI / Header Versioning **74**

W

Web 防火墙 AWS WAF / AWS Shield **78**
Web 应用防火墙 Web Application Firewall (WAF) **74**
外部化配置 Externalised Configuration **74**
外部集成层 External / Enterprise Integration Layer **72**
外部状态存储 external state store / External State Store **61,74**
完全 / 差异 / 增量备份 Full / Differential / Incremental Backup **77**
完全仲裁 Complete Mediation **77**
网关 API Gateway **74**
网络效应 Network Effect **73**
威胁 / 漏洞 / 风险 Threat / Vulnerability / Risk **77**
威胁分类 STRIDE **77**
微服务 Microservices **74**
位置 Location **72**
文档数据库 Document Database / Amazon DocumentDB **75,78**
无服务器 Serverless **74**
无状态 Stateless **74**
物理架构 Physical Architecture **72**
物联网接入 AWS IoT Core **78**

X

系统上下文 System Context **72**
先扩后缩 Expand–Migrate–Contract **77**
现有的 existing **71**
限界上下文 Bounded Context **73**
限流 / 配额 Rate Limiting / Quota **74**
详细设计 Detailed Design **72**
详细说明 elaborate **71**
响应度量 Response Measure **72**
响应时间 Response Time **67**
响应式宣言 Reactive Manifesto **72**
向后兼容 backward compatible / Backward Compatible **61,74**
削峰 Peak Shaving / Load Levelling **76**
消费者组 Consumer Group **76**
消息代理 Message Broker **76**
消息队列 Message Queue / Amazon SQS **76,78**
小聚合 Small Aggregate **73**
效果 / 副作用 effect / side effect **71**
效用树 Utility Tree **72**
写吞吐 write throughput **61**
新手 novice **71**
新型 SQL NewSQL **75**
心跳 Heartbeat **77**
性能 Performance **72**
虚拟机 Virtual Machine (VM) / Amazon EC2 **75,78**
虚拟机监控器 Hypervisor **75**

Y

压力测试 Stress Testing **76**
延迟 Latency **67**
业务不变量 Business Invariant **73**
业务专属实体 business-specific entity / Business-Specific Entity **60,73**
依赖指标 dependent metric / Dependent / Base Metric **61,78**
仪表盘 Dashboard **78**
易用性 Usability **72**
应用托管 PaaS AWS Elastic Beanstalk **78**
应用指标 Application Metric **78**
影响 impact / implication **71**
影子流量 Shadowing **77**
硬件安全模块 Hardware Security Module (HSM) **77**
用户认证 Amazon Cognito **78**
优雅降级 Graceful Degradation **77**
由外而内 Outside-in Design **74**
有状态 Stateful **74**
有状态集 StatefulSet **75**
余量 Headroom **76**
语义化版本 Semantic Versioning **74**
预计算 Precomputation **76**
原生应用 Native App **74**
远程过程调用 gRPC / RPC **74**
约束 constraint **71**
云原生 Cloud-Native **74**
运维器 Operator **75**

Z

责任共担模型 Shared Responsibility Model **74**
粘性会话 Sticky Session **74**
真实性 Authenticity **77**
支付卡行业数据安全标准 PCI-DSS **77**
值对象 Value Object **73**
指标 Metrics **78**
指出 identify **71**
指数退避重试 Retry with Exponential Backoff **77**
只读副本 read-only copy **60**
只读副本（领域） Read-Only Copy **73**
至少一次投递 At-Least-Once Delivery **76**
制品 Artifact **72**
智能端点哑管道 Smart Endpoints and Dumb Pipes **76**
质量属性 Quality Attribute **72**
质量属性场景 Quality Attribute Scenario **72**
种子 Seed **73**
重试风暴 Retry Storm **77**
重新分片 Re-sharding **75**
主动 / 被动冗余 Active / Passive Redundancy **77**
主库 Primary **75**
追踪 Traces **78**
资产 Asset **77**
资源 Resource **74**
子系统 Subsystem **72**
自愈 Self-Healing **75**
纵深防御 Defence in Depth **77**
租户 Tenant **73**
租户专属密钥 Tenant-Specific Key **77**
最大响应时间 Maximum Response Time **67**
最关键的 crucial / critical **71**
最可能的原因 most likely cause **71**
最小可行产品 Minimum Viable Product (MVP) **73**
最小权限 Least Privilege **77**
最优 / 不够优 optimal / suboptimal **71**
最终一致 eventually consistent **60**
最终一致性 Eventual Consistency **73**
遵奉者 Conformist **73**

英文术语 A–Z 索引

收录学习笔记各章术语表、PART 3 常用短语、常用数据和术语词典，页码指向解释所在页。

#

4+1 View Model 4+1 视图 **72**
7R Migration Strategies 迁移策略 7R **74**
90th percentile (P90) 第 90 百分位 **61**

A

Abstraction / Encapsulation 抽象 / 封装 **72**
ACID ACID **75**
Active / Passive Redundancy 主动 / 被动冗余 **77**
adequately 充分地 **71**
Aggregate 聚合 **73**
Aggregate Root 聚合根 **73**
Alert 告警 **78**
alternative 替代方案 **71**
Amazon API Gateway API 网关 **78**
Amazon Athena SQL 查文件 **78**
Amazon Bedrock 生成式 AI **78**
Amazon CloudFront CDN **78**
Amazon CloudWatch 监控 **78**
Amazon Cognito 用户认证 **78**
Amazon DocumentDB 文档数据库 **78**
Amazon DynamoDB 键值数据库 **78**
Amazon EC2 虚拟机 **78**
Amazon ECR 镜像仓库 **78**
Amazon ECS / AWS Fargate 容器服务 / 无服务器容器 **78**
Amazon EKS 托管 Kubernetes **78**
Amazon ElastiCache (Redis) 内存缓存 **78**
Amazon EMR 大数据集群 **78**
Amazon EventBridge 事件总线 **78**
Amazon Keyspaces 宽列数据库 **78**
Amazon Kinesis Data Streams 事件流 **78**
Amazon MSK 托管 Kafka **78**
Amazon Neptune 图数据库 **78**
Amazon OpenSearch Service 搜索 **78**
Amazon RDS / Amazon Aurora 关系数据库 **78**
Amazon Redshift 数据仓库 **78**
Amazon Route 53 DNS **78**
Amazon S3 对象存储 **78**
Amazon SageMaker 机器学习 **78**
Amazon SNS 推送 / 发布订阅 **78**
Amazon SQS 消息队列 **78**
Amazon Timestream 时序数据库 **78**
Amdahl's Law 阿姆达尔定律 **76**
Analyzability 可分析性 **72**
Anemic Domain Model 贫血模型 **73**
Anticorruption Layer 防腐层 **73**
API Gateway 网关 **74**
Application Metric 应用指标 **78**
appropriate 合适的 **71**
Architecturally Significant Requirement (ASR) 架构上重要的需求 **72**
Architectural Style 架构风格 **72**
Architecture Decision Record 架构决策记录 **72**
Arrival Rate (λ) 到达率 **67**
Artifact 制品 **72**
Asset 资产 **77**
Association / Aggregation / Composition 关联 / 聚合 / 组合 **73**
assume / assumption 假设 **71**
At-Least-Once Delivery 至少一次投递 **76**
Authentication / Authorisation / Accounting (AAA) 认证 / 授权 / 审计 **77**
Authenticity 真实性 **77**
Availability 可用性 **67,72**
availability zone 可用区 **61**
AWS CloudFormation / AWS CDK 基础设施即代码 **78**
AWS Elastic Beanstalk 应用托管 PaaS **78**
AWS IAM / AWS Secrets Manager / AWS KMS 权限 / 密钥 **78**

github.com/bibi45c · 免费分享，禁止售卖

AWS IoT Core 物联网接入 **78**
AWS Lambda 函数 **78**
AWS Step Functions 工作流 **78**
AWS WAF / AWS Shield Web 防火墙 **78**
AWS X-Ray 分布式追踪 **78**

B

Backend for Frontend (BFF) 面向前端的后端 **74**
Backpressure 背压 **77**
backward compatible 向后兼容 **61,74**
BASE BASE **75**
Batch Endpoint 批量接口 **74**
Benchmark 基准测试 **76**
Big Ball of Mud 大泥球 **73**
Block / File Storage 块存储 / 文件存储 **75**
Blue/Green Deployment 蓝绿部署 **77**
Bottleneck 瓶颈 **76**
Boundary / Control / Entity (BCE) 边界 / 控制 / 实体 **72**
Bounded Context 限界上下文 **73**
breaking change 不兼容的修改 **61,74**
briefly 简要地 **71**
Bulkhead 舱壁 **77**
Business Invariant 业务不变量 **73**
business-specific entity 业务专属实体 **60,73**

C

Cache Hit Ratio 缓存命中率 **68**
Call-Return 调用-返回 **72**
Canary Release 金丝雀发布 **77**
Capacity Planning 容量规划 **76**
CAP Theorem CAP 定理 **75**
Cascading Failure 级联故障 **77**
Centralised Logging 集中式日志 **78**
Chaos Engineering 混沌工程 **77**
CI / CD 持续集成 / 持续交付 **73**
Circuit Breaker 熔断器 **77**
Class Diagram 类图 **73**
Client-Server 客户端-服务器 **72**
client-side orchestration 客户端编排 **61,74**
Cloud-Native 云原生 **74**
Cluster 集群 **75**
Cluster Autoscaler 集群自动扩缩器 **75**
Cold Start 冷启动 **74**
Collocation 共置 **76**
communication affinity 通信亲和度 **61,76**
Complete Mediation 完全仲裁 **77**
concrete 具体的 **71**
Concurrency 并发数 **67**
concurrent users 并发用户 **61**
Confidentiality / Integrity / Availability (CIA) 保密性 / 完整性 / 可用性 **77**
ConfigMap / Secret 配置 / 密钥 **75**
Conformist 遵奉者 **73**
consideration 考虑因素 **71**
constraint 约束 **71**
Consumer Group 消费者组 **76**
Consumer Lag 事件流积压 **68**
Container 容器 **75**
Container Orchestration 容器编排 **75**
Content Delivery Network (CDN) 内容分发网络 **74**
Content Negotiation 内容协商 **74**
Context Mapping 上下文映射 **73**
Control Layer 控制层 **72**
Control Plane 控制平面 **75**
Conway's Law 康威定律 **74**
Cooldown Period 冷却时间 **76**
Core / Supporting / Generic Subdomain 核心域 / 支撑域 / 通用域 **73**
CORS 跨域资源共享 **74**
CQRS 命令查询职责分离 **75**
Crown Jewels 皇冠明珠 **77**
crucial / critical 最关键的 **71**

D

DaemonSet 守护进程集 **75**
Dashboard 仪表盘 **78**
Data Access Layer 数据访问层 **72**
Database per Service 每服务一个数据库 **75**
Data Lake 数据湖 **75**
Data Masking / Tokenisation 数据脱敏 / 令牌化 **75**
Data Warehouse 数据仓库 **75**
Declarative / Desired State 声明式 / 期望状态 **75**
Defence in Depth 纵深防御 **77**
deliberately omitted 刻意省略 **71**
Demilitarized Zone (DMZ) 非军事区 **72**
Dependent / Base Metric 依赖指标 **78**
dependent metric 依赖指标 **61**
Deploy 部署 **72**
Deployment 部署对象 **75**
Deployment Unit 部署单元 **72**
derive / derived 推导出 **71**
Derived Metric 派生（计算）指标 **78**
describe / outline 描述 / 概述 **71**
Detailed Design 详细设计 **72**
Distributed Denial of Service (DDoS) 分布式拒绝服务 **77**
Document Database 文档数据库 **75**
Dogpile / Cache Stampede 缓存踩踏 **77**
Domain Entity 领域实体 **73**
domain event 领域事件 **60,73**
Domain Model 领域模型 **73**
DORA Metrics DORA 指标 **68**
drastic 大幅的 **71**
DREAD 风险打分 **77**

E

East-West / North-South Traffic 东西向 / 南北向流量 **76**
Ecosystem 生态系统 **73**
effect / side effect 效果 / 副作用 **71**
elaborate 详细说明 **71**
Elasticity 弹性 **72**
Elastic Load Balancing (ALB / NLB) 负载均衡 **78**
Encryption at Rest / in Transit 静态 / 传输中加密 **77**
Endpoint 端点 **74**
ensure 确保 **71**
Enterprise Architecture (EA) 企业架构 **72**
Enterprise Service Bus (ESB) 企业服务总线 **72**
Error Rate 错误率 **67**
evaluate 评估 **71**
Event-Driven Architecture (EDA) 事件驱动架构 **74**
Event Sourcing 事件溯源 **75**
Event Streaming 事件流 **76**
Eventual Consistency 最终一致性 **73**
eventually consistent 最终一致 **60**
Evolvability 可演进性 **72**
existing 现有的 **71**
Expand–Migrate–Contract 先扩后缩 **77**
External / Enterprise Integration Layer 外部集成层 **72**
Externalised Configuration 外部化配置 **74**
external state store 外部状态存储 **61,74**

F

facilitate 促进、便于 **71**
Failover 故障转移 **77**
Fan-out 扇出 **76**
feasible 可行的 **71**
Firewall 防火墙 **72**
Fitness Function 适应度函数 **77**
Framework 框架 **73**
Full / Differential / Incremental Backup 完全 / 差异 / 增量备份 **77**
Function as a Service (FaaS) 函数即服务 **74**

G

General Data Protection Regulation (GDPR) 通用数据保护条例 **77**
Generalisation (Inheritance) 泛化（继承） **73**
generic service 通用服务 **60**
GitOps GitOps **73**
Golden Path 黄金路径 **73**
Graceful Degradation 优雅降级 **77**
Granularity 粒度 **74**
Graph Database 图数据库 **75**
GraphQL 图查询 API **74**
gRPC / RPC 远程过程调用 **74**

H

Hardware Security Module (HSM) 硬件安全模块 **77**
Harvest 收获率 **67**
Hash / Range Sharding 哈希分片 / 范围分片 **75**
Headroom 余量 **76**
Health Check 健康检查 **77**
Heartbeat 心跳 **77**
Hexagonal Architecture 六边形架构 **74**
High Cohesion / Low Coupling 高内聚 / 低耦合 **72**
Horizontal Pod Autoscaler (HPA) 水平 Pod 自动扩缩 **75**
Horizontal / Vertical Scaling 水平 / 垂直扩展 **74**
hot shard 热点分片 **61,75**
href 超文本引用 **74**
Hybrid Architecture 混合架构 **74**
Hypermedia (HATEOAS) 超媒体 **74**
hypermedia link 超媒体链接 **61**
Hypervisor 虚拟机监控器 **75**

I

IaaS / PaaS / SaaS IaaS / PaaS / SaaS **74**
Idempotency Key 幂等键 **74**
Idempotent 幂等 **74**
Idempotent Receiver 幂等接收者 **76**
identify 指出 **71**
Identity and Access Management (IAM) 身份与访问管理 **77**
IDS / IPS 入侵检测 / 防御 **77**
illustrate 图示 / 举例说明 **71**
Image / Container Registry 镜像 / 镜像仓库 **75**
Immutable Infrastructure 不可变基础设施 **74**
impact / implication 影响 **71**
incomplete 不完整的 **71**
Infrastructure as Code (IaC) 基础设施即代码 **73**
Infrastructure Metric 基础设施指标 **78**
Ingress 入口 **75**
In-Memory Cache 内存缓存 **75**
in-process call 进程内调用 **60**
Integration Endpoint 集成端点 **72**
Interaction 交互 **73**
Internal Developer Platform (IDP) 内部开发者平台 **73**
Internal Zone 内部区 **72**
Interoperability 互操作性 **72**
inter-process communication 跨进程通信 **60**

J

Job / CronJob 任务 / 定时任务 **75**
JSON Web Token (JWT) JSON 令牌 **74**
justify / justification 论证 / 理由 **71**

K

Key Management Service (KMS) 密钥管理服务 **77**
Key-Value Store 键值存储 **75**

L

Label Selector 标签选择器 **75**
Lambda / Kappa Architecture Lambda / Kappa 架构 **74**
Latency 延迟 **67**
Layered / N-Tier Architecture 分层架构 **72**
Least Privilege 最小权限 **77**

github.com/bibi45c · 免费分享，禁止售卖

leverage 利用 **71**
Library 库 **73**
Little's Law Little 定律 **76**
Liveness / Readiness Probe 存活 / 就绪探针 **75**
Load Balancing 负载均衡 **76**
Load Generator 施压机 **76**
Load Shedding 减载 **77**
Load Testing 负载测试 **76**
Location 位置 **72**
Logical Architecture 逻辑架构 **72**
Logs 日志 **78**

M

Magnet 磁铁 **73**
Maintainability 可维护性 **72**
Managed Service 托管服务 **74**
Manifest 体现（包含） **72**
Matchmaker 撮合者 **73**
Maximum Response Time 最大响应时间 **67**
Mean Absolute Error (MAE) 平均绝对误差 **68**
Message Broker 消息代理 **76**
Message Queue 消息队列 **76**
Metrics 指标 **78**
Microservices 微服务 **74**
Minimum Viable Product (MVP) 最小可行产品 **73**
mitigate 缓解 **71**
Modifiability 可修改性 **72**
Monolith 单体 **74**
most likely cause 最可能的原因 **71**
MTBF 平均故障间隔 **67**
MTTF 平均无故障时间 **67**
MTTR 平均恢复时间 **67**
Multi-AZ Deployment 多可用区部署 **75**
Multi-Cloud 多云 **74**
Multiplicity 多重性 **73**
Multitenancy 多租户 **73**
Mutual TLS (mTLS) 双向 TLS **77**

N

Namespace 命名空间 **75**
Native App 原生应用 **74**
Network Effect 网络效应 **73**
NewSQL 新型 SQL **75**
Node 节点 **72**
Node Group 节点组 **75**
Noisy Neighbour 吵闹邻居 **77**
Non-Functional Requirement (NFR) 非功能需求 **72**
Non-Repudiation 不可否认 **77**
novice 新手 **71**

O

OAuth 2.0 授权框架 **74**
Object Storage 对象存储 **75**
Observability 可观测性 **78**
One-Way Notification 单向通知 **76**
OpenAPI Specification 开放 API 规范 **74**
Open Host Service 开放主机服务 **73**
Operator 运维器 **75**
optimal / suboptimal 最优 / 不够优 **71**
Orchestration / Choreography 编排 / 协同 **76**
Outside-in Design 由外而内 **74**

P

Packaging 打包 **72**
Pagination 分页 **74**
Partial Response 部分响应 **74**
Partition 分区 **76**
Pass Criteria 通过标准 **76**
Path Parameter 路径参数 **74**
PCI-DSS 支付卡行业数据安全标准 **77**
peak load 高峰负载 **60**
Peak Shaving / Load Levelling 削峰 **76**
Peer-to-Peer (P2P) 点对点网络 **72**
Percentile (P50 / P90 / P95 / P99) 百分位 **67**
Performance 性能 **72**
PersistentVolume / PersistentVolumeClaim 持久卷 / 持久卷申领 **75**
Pets vs Cattle 宠物与牛群 **74**
Physical Architecture 物理架构 **72**
Ping-Pong Effect 乒乓效应 **76**
Platform 平台 **73**
Platform Engineering 平台工程 **73**
Platform Service Layer 平台服务层 **72**
Pod Pod **75**
Point-to-Point Messaging 点对点（消息） **76**
polyglot persistence 多语言持久化 **61,75**
Portability 可移植性 **72**
Portal Server 门户服务器 **72**
Precomputation 预计算 **76**
Presentation Layer 表现层 **72**
Primary 主库 **75**
Procurement Lead Time 采购提前期 **76**
Producer / Consumer 生产者 / 消费者 **73**
Progressive Web App (PWA) 渐进式 Web 应用 **74**
Proof of Concept (PoC) 概念验证 **72**
propose 提出 **71**
public / private subnet 公有 / 私有子网 **61**
Publish / Subscribe 发布 / 订阅 **76**

Q

Quality Attribute 质量属性 **72**
Quality Attribute Scenario 质量属性场景 **72**
Query Parameter 查询参数 **74**
Queue Length 队列长度 **68**

R

RAID RAID **77**
Rate Limiting / Quota 限流 / 配额 **74**
rationale 理由 **71**
Reactive Manifesto 响应式宣言 **72**
read-only copy 只读副本 **60**
Read-Only Copy 只读副本（领域） **73**
Read Replica 读副本 **75**
Read-Through / Write-Through 读穿透 / 写穿透 **75**
recommend 推荐 **71**
Recovery Point Objective (RPO) 恢复点目标 **68**
Recovery Time Objective (RTO) 恢复时间目标 **68**
Redundancy 冗余 **77**
Reference Architecture 参考架构 **72**
Reference by Identity 按标识引用 **73**
Relational Database (RDBMS) 关系数据库 **75**
Reliability 可靠性 **72**
rel (link relation) 链接关系 **74**
ReplicaSet 副本集 **75**
Replication Lag 复制延迟 **68**
Repository 仓储 **73**
Request Header 请求头 **74**
Request-Response 请求-响应 **76**
Request / Response Body 请求体 / 响应体 **74**
Re-sharding 重新分片 **75**
Resource 资源 **74**
respectively 分别 **71**
Response Measure 响应度量 **72**
Response Time 响应时间 **67**
retention period 保存时长 **61,78**
Retry Storm 重试风暴 **77**
Retry with Exponential Backoff 指数退避重试 **77**
Reusable Service / Framework / Shared Library 可复用服务 / 框架 / 共享库 **73**
Reverse Proxy 反向代理 **72**
Richardson Maturity Model (RMM) 理查森成熟度模型 **74**
Risk Treatment 风险处置 **77**
Rollback 回滚 **77**
Rolling Update 滚动更新 **77**

S

Safe Method 安全方法 **74**
Saga Pattern Saga 模式 **76**
Scalability 可扩展性 **72**
Scale Cube 扩展立方体 **74**
scenario 场景 **71**
Search Engine 搜索引擎 **75**
Secure by Design 设计即安全 **77**
Seed 种子 **73**
Self-Healing 自愈 **75**
Semantic Versioning 语义化版本 **74**
Separation of Concerns 关注点分离 **72**
Serial / Parallel 串行 / 并行 **76**
Serverless 无服务器 **74**
Service 服务（K8s） **75**
Service Demand 服务需求 **76**
Service Discovery 服务发现 **76**
Service Mesh 服务网格 **75**
Service-Oriented Architecture (SOA) 面向服务架构 **72**
Service Registry 服务注册中心 **76**
Service Time 服务时间 **67**
Shadowing 影子流量 **77**
Sharding 分片 **75**
Shard Key / Partition Key 分片键 **75**
Shared Database 共享数据库 **75**
Shared Kernel 共享内核 **73**
Shared Library Layer 共享库层 **72**
Shared Responsibility Model 责任共担模型 **74**
Shared Service 共享服务 **73**
Sidecar Proxy 边车代理 **75**
Single Point of Failure (SPOF) 单点故障 **77**
Single Responsibility Principle (SRP) 单一职责原则 **72**
Single Sign-On (SSO) 单点登录 **77**
Sizing 规模估算 **72**
SLO / SLA 服务等级目标 / 协议 **68**
Small Aggregate 小聚合 **73**
Smart Endpoints and Dumb Pipes 智能端点哑管道 **76**
Soak / Endurance Testing 耐久测试 **76**
Software Architecture 软件架构 **72**
Solution Architecture (SA) 解决方案架构 **72**
Source / Stimulus 刺激源 / 刺激 **72**
Span 跨度 **78**
Spike Testing 突发测试 **76**
stakeholder 利益相关者 **71**
Stateful 有状态 **74**
StatefulSet 有状态集 **75**
Stateless 无状态 **74**
Stepping Stones 跳板 **77**
Sticky Session 粘性会话 **74**
Stream-Processing Job 流处理任务 **78**
Stress Testing 压力测试 **76**
STRIDE 威胁分类 **77**
Strong Consistency 强一致性 **73**
Subsystem 子系统 **72**
suitable 合适的 **71**
Synchronous / Asynchronous 同步 / 异步 **76**
System Context 系统上下文 **72**

T

Tenant 租户 **73**
Tenant-Specific Key 租户专属密钥 **77**
Testability 可测试性 **72**
Think Time 思考时间 **76**
Threat / Vulnerability / Risk 威胁 / 漏洞 / 风险 **77**
Throughput 吞吐量 **67**
Timeout 超时 **77**
Time-Series Database (TSDB) 时序数据库 **75**
Time to First Byte (TTFB) 首字节时间 **67**
Tombstone 墓碑标记 **77**

github.com/bibi45c · 免费分享，禁止售卖

Toolbox 工具箱 **73**
TPS / QPS 每秒事务数 / 查询数 **67**
Trace ID / Correlation ID 跟踪 ID / 关联 ID **78**
Traces 追踪 **78**
trade-off 取舍 **71**
transaction scope 事务范围 **60**
Transport Layer Security (TLS) 传输层安全 **77**
Trend Analysis 趋势分析 **76**
Twelve-Factor App 十二要素应用 **74**
Two-Phase Commit (2PC) 两阶段提交 **76**

U

Ubiquitous Language 通用语言 **73**
undesirable 不理想的 **71**
Universal Scalability Law (USL) 通用扩展定律 **76**
Uptime 可用时间 **67**
URI 统一资源标识符 **74**
URI / Header Versioning URI 版本 / 请求头版本 **74**
Usability 易用性 **72**
Utilisation 利用率 **67**
Utilisation Law 利用率定律 **76**
Utility Tree 效用树 **72**

V

Value Object 值对象 **73**
Vendor Lock-in 厂商锁定 **74**
Vertical Partitioning 垂直分区 **75**
viable 可行的 **71**
Virtual Machine (VM) 虚拟机 **75**

W

Waiting / Queueing Time 等待 / 排队时间 **67**
Web Application Firewall (WAF) Web 应用防火墙 **74**
Webhook 回调 **74**
WebSocket 双向长连接 **74**
whereby 借此 **71**
Wide-Column / Column-Family Database 宽列 / 列族数据库 **75**
with respect to / in terms of 就……而言 **71**
Worker Node 工作节点 **75**
Workload Mix 场景组合 **76**
workload model 工作负载模型 **61,76**
write throughput 写吞吐 **61**

Y

Yield 产出率 **67**

github.com/bibi45c · 免费分享，禁止售卖